

MODELLI E ARCHITETTURE AVANZATI DI BASI DI DATI

Alessandro Zappatore

Università degli studi di Torino
Anno accademico 2025/2026, 2° semestre

Contents

1	Introduzione alle Basi di Dati e ai Modelli Storici	5
1.1	Concetti Fondamentali	5
1.2	Livelli di Astrazione (Architettura Standard)	5
1.3	DBMS (Data Base Management System)	5
2	Carrellata Storica dei Modelli di Dati	7
2.1	Prima Generazione: Modelli Gerarchici e Reticolari (Anni '60)	7
2.2	Seconda Generazione: Modello Relazionale (Anni '70)	7
2.3	Terza Generazione: Anni '80 e il Modello ad Oggetti	8
2.4	Quarta Generazione: Anni '90 e 2000 - Dati Semistrutturati e Grafi	8
2.5	Modelli Ad-Hoc per Domini Specifici	9
3	Introduzione all'Architettura	10
3.1	Gestione dello Storage (Dati su Disco)	10
3.1.1	Organizzazione dei File	10
3.2	Gestione della Memoria e dei File	11
3.2.1	Disk Space Manager	11
3.2.2	Buffer Manager	11
3.3	Recovery Management	12
3.3.1	Recovery Manager (Gestore del Ripristino)	12
3.4	Concurrency Control	13
3.4.1	Lock Manager	13
3.4.2	Transaction Manager e Proprietà ACID	13
3.4.3	Dettagli sull'implementazione ACID e Log	14
3.5	Query Evaluation Engine	14
4	Dischi e Gestione della Memoria Secondaria	16
4.1	Architettura Fisica del Disco	16
4.2	Tempo di Accesso e Componenti di Costo	17
4.3	Strategie di Ottimizzazione e "Next Block"	17
4.4	Pre-fetching e Prevedibilità	17
4.5	File Manager e Organizzazione dei Dati	18
4.5.1	Strategie di Organizzazione dei File	18
5	Struttura Interna: Pagine e Record	20
5.1	Record	20
5.1.1	Record a Lunghezza Fissa (Fixed-Length)	20
5.1.2	Record a Lunghezza Variabile (Variable-Length)	20
5.2	Organizzazione di una pagina	20

5.2.1	Record a lunghezza fissa	21
5.2.2	Record a lunghezza variabile (Slotted Page)	21
5.3	Heap File	22
5.3.1	Heap File: Implementazione a Lista Concatenata	22
5.3.2	Heap File: Implementazione a Directory	23
6	Indici	24
6.1	Ricerca per intervallo (Range Search)	24
6.2	ISAM (Indexed Sequential Access Method)	24
6.3	Indici B+ Tree	26
6.3.1	Impatto della Dimensione della Chiave sulle Prestazioni	27
6.3.2	Compressione delle chiavi basata su prefissi (Prefix Key Compression)	28
6.3.3	Inizializzazione di un B+ Tree: Bulk Loading	29
6.4	Classificazione degli indici	29
6.5	Indici Clusterizzati e Non Clusterizzati	30
6.5.1	Indice Clusterizzato (<i>Clustered Index</i>)	31
6.5.2	Indice Non Clusterizzato (<i>Unclustered Index</i>)	32
6.5.3	Casi di Studio: Domande d'Esame	32
6.5.4	Gestione dei Duplicati e Struttura delle Voci (Data Entries)	33
6.6	Indici con chiavi di ricerca composte	33
6.6.1	Strategie di esecuzione per query multi-attributo	33
6.7	Indici Hash-Based	34
6.7.1	Hashing Statico	35
6.7.2	Hashing Estendibile (Extendible Hashing)	36
6.7.3	Hashing Lineare (Linear Hashing)	37
7	Analisi dei Costi e Prestazioni	39
7.1	Parametri del Modello di Costo	39
7.2	Assunzioni sulle Strutture Dati	39
7.3	Tabella Riassuntiva dei Costi (in operazioni di I/O)	40
7.4	Analisi Dettagliata per Struttura	40
7.4.1	Heap File	40
7.4.2	Sorted File	41
7.4.3	Indice ad Albero Clustered (Clustered Tree)	41
7.4.4	Indice ad Albero Non Clustered (Unclustered Tree)	41
7.4.5	Indice Hash Non Clustered (Unclustered Hash)	42
7.5	Come scegliere l'indice appropriato	42
7.5.1	Il Trade-off Fondamentale e la Scelta dell'Indice	43
7.5.2	Linee Guida per la Costruzione degli Indici	43
7.5.3	Esempi di Scelta per Indici Clustered	44
7.5.4	Esempi di Scelta per Query Index Only	45
8	Gestione del Buffer (Buffer Manager)	47
8.1	Obiettivi e Metriche di Valutazione	47
8.2	Politiche di Rimpiazzo (Replacement Policies)	47
8.3	Differenza tra OS e DBMS: Il problema del Sequential Flooding	47
8.4	Algoritmo di Rimpiazzo a Orologio (Clock Algorithm)	48
8.5	Evoluzione delle Strategie di Allocazione	49
8.5.1	Domain Separation (Separazione per Dominio)	49
8.5.2	Working Set Model	49
8.5.3	HotSet Model	50
8.6	Query Locality Set Model (QLSM)	50
8.6.1	Algoritmo Globale di Gestione QLSM	51

9	Valutazione degli operatori	52
9.1	Operatori Relazionali e Operazioni Fisiche	52
9.1.1	Operatori dell'Algebra Relazionale	52
9.1.2	Operazioni Fisiche Implicite	52
9.2	Classificazione degli Algoritmi di Valutazione	53
9.3	Implementazione dell'Operatore di Selezione (σ)	53
9.3.1	Query con Singola Condizione	53
9.3.2	Query Complesse (Condizioni Congiunte)	53
9.4	Implementazione dell'Operatore di Proiezione (π)	54
9.5	L'Operatore di Ordinamento	54
9.5.1	2-Way External Merge Sort	55
9.5.2	General External Merge Sort (con (B) Buffer)	56
9.5.3	Ottimizzazione del Passo 0: Replacement Selection (Internal Sort)	57
9.6	L'Operatore di Join ($\bowtie$)	58
9.7	Algoritmi basati su Cicli Annidati (Nested Loops)	58
9.7.1	Simple Nested Loop Join (Tuple-oriented)	58
9.7.2	Page-oriented Nested Loop Join	59
9.7.3	Block Nested Loop Join (BNLJ)	59
9.7.4	Index Nested Loop Join	59
9.8	Algoritmi basati su Ordinamento e Partizionamento	60
9.8.1	Sort-Merge Join	60
9.8.2	Hash-Join	61
10	Ottimizzazione delle query	62
10.1	Alberi di esecuzione: Modello Logico e Modello Fisico	62
10.1.1	Modello logico	62
10.1.2	Modello fisico	63
10.2	Approcci all'Ottimizzazione	63
10.3	Modello basato sui costi (Stima)	63
10.4	Principio di ottimalità delle sotto-query	64
10.4.1	Esempio applicativo: Principio di Ottimalità delle Sotto-query	64
10.5	Ordine di Valutazione dei Join	66
10.6	Approccio Ricorsivo (Top-Down)	67
10.6.1	Il collo di bottiglia della ricorsione ingenua	67
10.7	Programmazione Dinamica (Bottom-Up)	67
11	Concorrenza e Transazioni	69
11.1	Il concetto di Transazione	69
11.2	Le Proprietà ACID	69
11.3	Esecuzione Concorrente e Anomalie	69
11.3.1	Esempi di Schedule	70
11.4	Conflitti e Anomalie di Concorrenza	70
11.4.1	Conflitto W-R: Lettura Sporca (Dirty Read)	70
11.4.2	Conflitto R-W: Lettura Non Ripetibile (Unrepeatable Read) o Update Fantasma	70
11.4.3	Conflitto W-W: Scrittura Cieca (Lost Update / Overwriting Uncommitted Data)	71
11.5	Teoria dello Scheduling e Serializzabilità	71
11.5.1	Schedule Conflict Serializable (Serializzabilità basata sui conflitti)	71
11.5.2	Gestione della Concorrenza basata sul Locking	72
11.5.3	Deadlock (Stallo)	73
11.5.4	Locking con Multigranularità (MGL)	73
11.5.5	Database Dinamici e il Problema dei Dati Fantasma (Phantom Reads)	74
11.5.6	Gestione della Concorrenza sugli Indici Gerarchici	75
11.6	Controllo della Concorrenza Ottimistico (Kung-Robinson)	76
11.7	Two-Phase Locking Ottimistico (O2PL)	79
11.8	Timestamp Concurrency Control (Timestamp CC)	79
11.8.1	View Serializability e Thomas Write Rule	80

11.9 Multiversion Timestamp Concurrency Control (MVCC)	80
12 Crash Recovery	82
12.1 Gestione del Buffer Pool: Politiche Steal e No-Force	82
12.2 Logging e Write-Ahead Logging (WAL)	83
12.2.1 Struttura del File di Log e LSN	83
12.2.2 La Regola del Write-Ahead Logging (WAL)	83
12.3 Strutture Dati di Supporto al Recovery	84
12.4 Checkpointing	84
12.5 La Procedura di Abort (Disfacimento Logico)	84
12.6 La Procedura di Crash Recovery	85
12.6.1 Fase di Analisi	85
12.6.2 Fase di REDO	86
12.6.3 Fase di UNDO	86
12.6.4 Il Paradigma "Repeating History": Perché il REDO precede l'UNDO?	87

1 Introduzione alle Basi di Dati e ai Modelli Storici

1.1 Concetti Fondamentali

Database (Base di Dati) Un database è una collezione di dati organizzati in modo strutturato, in funzione del modello di riferimento scelto. È fondamentale che i dati siano organizzati in modo efficiente per poterne garantire una gestione e un'interrogazione ottimale (reperimento, analisi e visualizzazione univoca dei risultati).

Differenza tra Dato e Informazione

- **Dato:** È un elemento grezzo, privo di contesto (es. un numero, una stringa, una data).
- **Informazione:** È il dato interpretato e contestualizzato.

Esempio: Il numero "10" è un dato. Se lo associo al numero di studenti presenti a lezione, diventa un'informazione. Se lo associo al numero di libri che ho letto, diventa un'informazione diversa. Il dato di partenza è lo stesso, ma il contesto ne cambia il significato. Le basi di dati organizzano i dati proprio per rendere efficiente e rapida l'estrazione di queste informazioni.

Modello dei Dati È un formalismo matematico e rigoroso utilizzato per descrivere la struttura dei dati stessi (es. modello relazionale, a oggetti, a grafo). Ciascun modello esprime un insieme di vincoli intrinseci che l'organizzazione dei dati deve rispettare. Nel modello relazionale, ad esempio, i dati devono essere divisi in tabelle normalizzate, collegate tramite chiavi primarie ed esterne.

Schema Rappresenta l'applicazione di un modello di dati a un dominio specifico. Definisce un insieme di vincoli strutturali e di integrità. In un database relazionale, lo schema descrive le tabelle (relazioni), i campi (attributi), i tipi di dato e i vincoli (es. chiavi primarie ed esterne).

I vincoli sono essenziali per garantire l'**integrità dei dati** (es. evitare problemi di integrità referenziale se una chiave esterna punta a un record inesistente) e semplificano l'ottimizzazione e la gestione, sebbene limitino la flessibilità del modello per certi scopi.

1.2 Livelli di Astrazione (Architettura Standard)

L'organizzazione dei dati può essere analizzata su tre livelli principali:

1. **Livello Concettuale:** È il livello più alto e astratto. Descrive "cosa" deve essere rappresentato (la conoscenza del dominio) senza preoccuparsi dell'implementazione. Utilizza formalismi comprensibili agli utenti, come i diagrammi ER (Entità-Relazione) o UML.
2. **Livello Logico:** Si posiziona a metà strada e colma il gap tra concettuale e fisico. Descrive i dati in modo formale attraverso le strutture previste dal modello scelto (es. tabelle nel modello relazionale o gerarchie nel modello ad oggetti), ma rimane indipendente dalla loro effettiva memorizzazione fisica. Alcuni modelli logici (come quello ad oggetti) sono più vicini al concettuale, altri (come il relazionale) sono più vicini al livello fisico.
3. **Livello Fisico:** Descrive "come" i dati sono effettivamente memorizzati su disco. Riguarda le strutture dati interne, l'allocazione dello spazio, la codifica in bit (es. come rappresentare fisicamente una data) e gli indici.

1.3 DBMS (Data Base Management System)

Cos'è un DBMS È un sistema software (che talvolta include componenti hardware) progettato per creare, gestire, manipolare e interrogare una base di dati in modo efficiente e sicuro. L'obiettivo principale di un DBMS è garantire la separazione tra la "business logic" (le applicazioni) e i dati fisici. Per fare ciò, deve garantire due livelli di astrazione:

- **Indipendenza Logica dei Dati:** Le applicazioni non devono risentire di modifiche fatte allo schema logico (es. se aggiungo una colonna a una tabella, un programma che interrogava le altre colonne deve continuare a funzionare senza essere riscritto).
- **Indipendenza Fisica dei Dati:** Le applicazioni e lo schema logico non devono risentire di modifiche a livello fisico (es. se aggiungo o rimuovo un indice su un disco per velocizzare una ricerca, il programma esterno non se ne deve accorgere).

Differenza e Interazione tra OS e DBMS Un Sistema Operativo (OS) offre funzionalità di base per la gestione delle risorse hardware e software, come la gestione dei file, della memoria centrale (buffer), della concorrenza e il recupero da malfunzionamenti (crash recovery). Sebbene un DBMS si appoggi all'OS per l'accesso fisico alle risorse, esso implementa versioni *ad hoc* e altamente specializzate di queste funzionalità. I meccanismi generici dell'OS risultano infatti inadeguati per le esigenze specifiche di una base di dati.

Le differenze principali si manifestano in quattro macro-aree:

- **Gestione del Buffer (Buffer Management):** Nei database, l'operazione di gran lunga più costosa è l'I/O su disco. Entrambi i sistemi utilizzano un buffer in RAM per minimizzare le letture/scritture fisiche, ma adottano strategie di *replacement* profondamente diverse per scegliere la "vittima" (la pagina di memoria da sovrascrivere quando la RAM è piena). L'OS utilizza politiche generiche e reattive, come l'LRU (Least Recently Used), che scarta la pagina non utilizzata da più tempo. Questa politica può essere disastrosa per un DBMS: ad esempio, durante la scansione sequenziale di una tabella molto grande, l'LRU svuoterebbe l'intero buffer scartando dati potenzialmente utili per altre query. Il DBMS, invece, ha un comportamento **predittivo**. Grazie al modulo di ottimizzazione delle query (Query Optimizer), il DBMS conosce in anticipo il piano di esecuzione e sa quali dati serviranno a breve, potendo così forzare il mantenimento in memoria di pagine strategiche o scartare quelle inutili.
- **Gestione dei File e Strutture Dati:** Per l'OS, un file è semplicemente una sequenza (stream) di byte non strutturata. Per il DBMS, i dati hanno una struttura semantica precisa: file fisici suddivisi in blocchi, contenenti tuple, formate da attributi. Spesso il DBMS bypassa del tutto le routine di caching dei file dell'OS per avere il controllo assoluto su *quando* un blocco viene effettivamente scritto su disco.
- **Gestione della Concorrenza (Concurrency Control):** L'OS gestisce la concorrenza a livello di processi e thread. Il DBMS deve invece gestire la concorrenza a **livello logico del dato** (es. applicando lock temporanei su una singola riga di una tabella). Questo permette a migliaia di utenti simultanei di leggere e aggiornare gli stessi dati senza creare inconsistenze.
- **Recupero dai Guasti (Crash Recovery):** Mentre l'OS preserva la coerenza del file system, il DBMS deve garantire a livello logico l'integrità dei dati e il rispetto delle proprietà *ACID*. Tramite file di log e protocolli come il **Write-Ahead Logging (WAL)**, il DBMS può eseguire l'UNDO (annullare transazioni a metà) e il REDO (riapplicare transazioni completate) dopo un crash.

Transazione È una sequenza di operazioni trattata come un'unità logica e **atomica**: o tutte le operazioni vengono eseguite con successo, o nessuna viene applicata (principio di "tutto o niente"). *Esempio:* In un bonifico bancario (prelievo da un conto e deposito in un altro), se il deposito fallisce, il prelievo deve essere annullato per mantenere la consistenza ed evitare la perdita di denaro.

2 Carrellata Storica dei Modelli di Dati

2.1 Prima Generazione: Modelli Gerarchici e Reticolari (Anni '60)

Modello Gerarchico Basato su una struttura ad albero. La radice rappresenta il concetto principale e i nodi figli le sue proprietà o entità dipendenti (es. Radice: "Persona" → Figli: "Nome", "Data di Nascita"). I puntatori erano bidirezionali (padre-figlio e figlio-padre).

- **Pro:** Molto efficiente per relazioni strettamente gerarchiche.
- **Contro:** Estremamente limitante per relazioni complesse o molti-a-molti, dove un nodo dovrebbe avere "più padri".

Modello Reticolare (Codasyl) Estende il modello gerarchico trasformando l'albero in un grafo. Questo permetteva ai nodi di avere molteplici nodi padre e di esprimere le relazioni tramite archi.

Criticità della Prima Generazione

1. **Distanza dal modello concettuale:** Erano funzionali solo per app di natura procedurale, ma inadeguati per sistemi come ospedali o banche.
2. **Navigazione a Puntatori Fisici:** Si sapeva da dove partiva il puntatore ma non dove andava a finire nel disco fisicamente.
3. **Linguaggio fortemente Procedurale:** L'utente doveva essere un bravo programmatore, scrivendo la procedura esatta (il percorso dei puntatori) per raggiungere il dato (es. "Vai a Università → segui puntatore Studenti...").
4. **Imprevedibilità e scarsa ottimizzazione:** L'ottimizzatore non poteva prevedere il percorso di accesso (e quindi allocare la memoria o posizionare la testina del disco in modo efficiente), rendendo impossibile un'ottimizzazione automatica.

2.2 Seconda Generazione: Modello Relazionale (Anni '70)

Introdotta da E.F. Codd, è il modello tuttora più diffuso. I dati sono organizzati in **frammenti omogenei** (le relazioni, o tabelle). Ciascun record (riga) ha la stessa struttura formale degli altri.

Vantaggi principali:

- **Prevedibilità e Ottimizzazione:** L'omogeneità permette di stimare quante pagine di memoria bisognerà leggere. "Prevedibilità implica ottimizzabilità" (si sfruttano al meglio i movimenti fisici del disco).
- **Assenza di Puntatori:** Si usano **puntatori logici** (le chiavi esterne per fare JOIN) eliminando l'imprevedibilità del puntatore fisico.
- **Linguaggio Dichiarativo (SQL):** L'utente dice *cosa* vuole estrarre, e non *come* trovarlo. Sarà il DBMS a tradurlo nei passi fisici necessari.
- **Algebra Relazionale:** Bastano pochi operatori matematici per implementare le richieste SQL. Attenzione: l'SQL è dichiarativo, mentre l'algebra relazionale è procedurale. Dato che gli operatori sono pochi, ci sono meno combinazioni possibili per ottimizzare (anche se l'ottimizzazione resta un problema matematico NP-completo).
L'utente dichiara cosa vuole ottenere non come ottenerlo, ci penserà il DBMS a tradurre la richiesta in un piano di esecuzione efficiente.
- **Vicinanza al Modello Fisico:** Le righe piatte nelle tabelle sono facilmente mappabili in blocchi contigui su disco.

Svantaggi e Limiti del Modello Relazionale:

- **Dati Piatti ("Particellati"):** Per ricomporre un'informazione complessa bisogna eseguire costosi e numerosi JOIN.
- **Difficoltà con Strutture Complesse:** Non è possibile rappresentare nativamente l'ereditarietà (es. Persona → Studente/Docente) o gerarchie complesse.
- **Incapacità di Match Parziali:** Il relazionale restituisce risultati esatti. Se cerco dati per "somiglianza" (match parziali), il relazionale fallisce perché questa logica andrebbe implementata nell'applicativo esterno, violando il principio di indipendenza dei dati.

2.3 Terza Generazione: Anni '80 e il Modello ad Oggetti

Per superare i limiti del relazionale (soprattutto riguardo all'ereditarietà), si sono sviluppate due scuole di pensiero, entrambe basate sul concetto di *Oggetto*:

- **Object-Oriented (OO) - Spinta dall'Accademia:** Si abbandona totalmente il relazionale. Si creano basi di dati che memorizzano nativamente non solo i dati, ma anche i metodi (comportamenti). Nativamente supporta gerarchie ed ereditarietà. Si interrogano con linguaggi come l'OQL (Object Query Language).
- **Object-Relational (OR) - Spinta dall'Industria:** Abbandonare il relazionale era economicamente folle visto il suo successo. Nasce così un approccio "ibrido": un motore relazionale di base arricchito da funzionalità ad oggetti utilizzabili solo "se necessarie", mantenendo la compatibilità con i vecchi sistemi.

2.4 Quarta Generazione: Anni '90 e 2000 - Dati Semistruutturati e Grafi

Con l'avvento di Internet, i database non sono più "isole" chiuse in una singola azienda, ma devono comunicare. Scambiarsi tabelle relazionali era problematico per colpa degli schemi rigidi.

Basi di Dati Semistrustrate (XML) Si cercava flessibilità (es. in un DB l'indirizzo è un campo unico, in un altro è diviso in via/civico). Nasce l'esigenza di documenti **autodescrittivi** come l'XML. In questi sistemi, **lo schema stesso diventa interrogabile**. *Esempio:* "Dimmi se esiste il campo 'Indirizzo' in questo nodo, e se sì, dammi il suo valore". Anche qui vinse un approccio ibrido: esistono DB nativamente XML, ma i giganti dell'industria hanno integrato il supporto XML sopra i propri DB Relazionali.

Database a Grafo (dal 2003) A differenza degli anni '60 in cui si forzava la struttura a grafo per tutto, ora i grafi vengono usati *esclusivamente* per domini in cui i concetti sono naturalmente legati da complesse reti di relazioni.

- **Web Semantico e RDF:** Permette di descrivere la conoscenza tramite **triplette (Soggetto, Predicato, Oggetto)**. Lo schema non è rigido né definito a priori (es. "Sapino" → gestisce → "Corso"). Posso effettuare **query di raggiungibilità** che nei modelli precedenti erano impossibili (es. "Trova il percorso relazionale più breve che collega il prof. X al dipartimento Y"). Si usa il linguaggio dichiarativo SPARQL.
- **Social Network:** Simili all'RDF ma con milioni di nodi (persone) e poche tipologie di archi (es. "Follow", "Like"). Se lo facessimo nel modello relazionale servirebbero infiniti JOIN. I DB a grafo permettono di calcolare facilmente il ruolo di un nodo (es. la "centralità" di un influencer) o di ricercare pattern ricorrenti (usati moltissimo ad esempio in biologia per studiare le catene di proteine).

2.5 Modelli Ad-Hoc per Domini Specifici

Oggi si è compreso che un "modello universale" non basta per tutto, e sono nati sistemi specializzati:

- **Database Spaziali (GIS):** L'elemento chiave è la *Location* (coordinate spaziali). Si usano per query impossibili in SQL tradizionale, come le *query di range / Nearest Neighbours* ("Trova i ristoranti entro 5km da me") e il **Join Spaziale** ("Abbina le scuole del comune con i supermercati più vicini a ciascuna di esse").
- **Database Documentali e Multimediali:** Permettono query basate sulla **somiglianza** e non sull'esatta uguaglianza (es. "Trova profili di lavoratori simili al mio").
- **Gestione dell'Incertezza:** I DB relazionali operano sotto l'assunzione di mondo chiuso (tutto ciò che è nel DB è vero al 100%, ciò che non c'è è falso). Alcuni modelli più moderni permettono di associare pesi di probabilità (es. "Lo studente X frequenta il corso Y con l'80% di confidenza").

3 Introduzione all'Architettura

Un Database Management System (DBMS) è un sistema software complesso suddiviso in moduli funzionali. L'architettura è tipicamente a livelli, dove ogni livello offre servizi a quello superiore nascondendo i dettagli implementativi del livello inferiore.

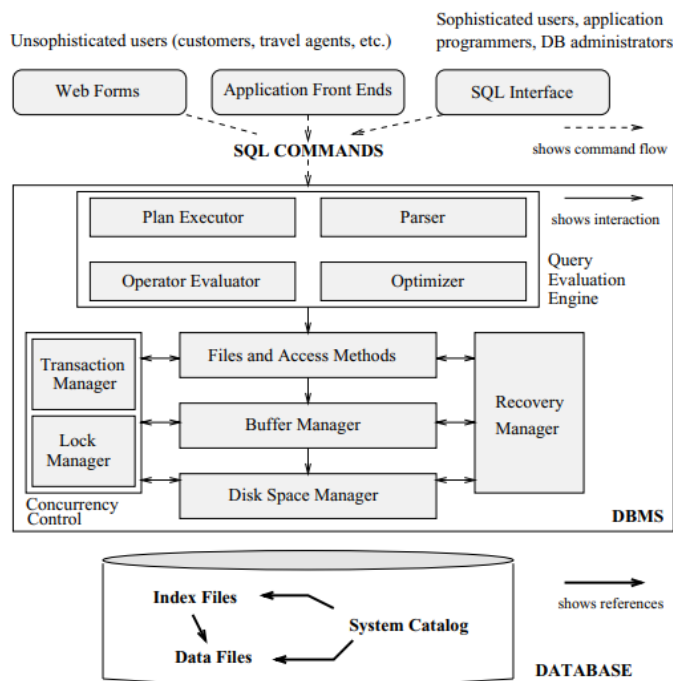


Figure 1: Panoramica dell'architettura a livelli di un DBMS

3.1 Gestione dello Storage (Dati su Disco)

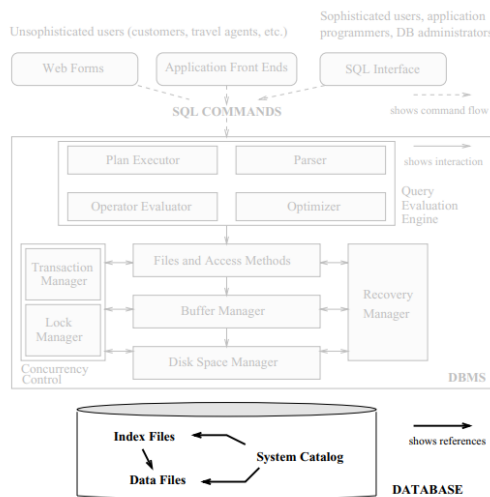


Figure 2: Rappresentazione dei dati su disco

Alla base del DBMS vi è la gestione fisica dei dati. I database risiedono su memoria secondaria (disco rigido o SSD) poiché la memoria volatile (RAM) non garantisce la persistenza e non è sufficientemente capiente per database di grandi dimensioni.

3.1.1 Organizzazione dei File

I dati su disco sono organizzati in file. A livello logico, il DBMS distingue tre categorie principali di file:

- **Data Files:** Contengono i dati veri e propri (i record delle tabelle). Possono essere organizzati come:
 - *Heap Files:* File non ordinati (mucchio), dove i nuovi record vengono inseriti nell'ultimo spazio disponibile. È la struttura più generica e veloce in scrittura.
 - *Sorted Files:* File in cui i record sono ordinati fisicamente in base a una chiave di ricerca. Ottimi per le letture sequenziali o ricerche su range.
- **Index Files:** Strutture ausiliarie (spesso alberi B+ o Hash) che velocizzano il recupero dei dati.
 - Contengono coppie (*chiave di ricerca, puntatore*).
 - Il puntatore indica l'indirizzo fisico (RID - Record ID) dove si trova il record completo nel data file.
- **System Catalogs (Dizionario dei Dati):** Contengono i metadati, ovvero "dati sui dati". Includono:
 - Schemi delle tabelle, nomi degli attributi e tipi di dato.
 - Statistiche per l'ottimizzatore (es. numero di record, cardinalità, distribuzione dei valori).
 - Informazioni sugli utenti e sui permessi di accesso.

3.2 Gestione della Memoria e dei File

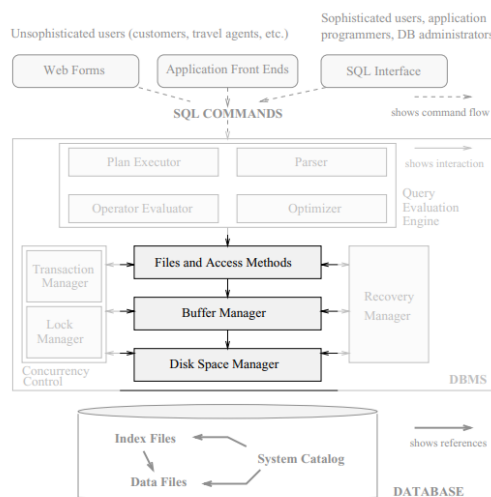


Figure 3: Interazione tra Disk Space Manager e Buffer Manager

Questo livello fa da "ponte" tra il disco (lento e persistente) e il processore (veloce e volatile).

3.2.1 Disk Space Manager

È il modulo di più basso livello che interagisce direttamente con il File System del Sistema Operativo.

- Gestisce l'allocazione e la deallocazione dello spazio su disco.
- Astrazione: Nasconde i dettagli hardware (tracce, settori) offrendo ai livelli superiori il concetto di **Pagina** (blocco di dati di dimensione fissa, es. 4KB o 8KB).

3.2.2 Buffer Manager

È uno dei componenti più critici per le prestazioni. Gestisce la memoria principale (RAM) riservata al DBMS, denominata **Buffer Pool**.

Funzionamento L'utente o i moduli superiori non accedono mai direttamente al disco. La procedura è la seguente:

1. **Richiesta:** Un livello superiore richiede una pagina.
2. **Check:** Il Buffer Manager controlla se la pagina è già nel Buffer Pool.
3. **Hit o Miss:**
 - Se presente (*Cache Hit*): restituisce l'indirizzo in memoria.
 - Se assente (*Cache Miss*): carica la pagina dal disco in un frame libero del buffer.

Strategie e Concetti Chiave

- **Unità di trasferimento:** Si legge/scrive sempre un'intera pagina, mai un singolo record. Questo sfrutta il principio di *località spaziale* (è probabile che dati vicini servano a breve).
- **Località temporale:** Se una pagina viene richiesta, è probabile che venga richiesta nuovamente a breve.
- **Page Replacement:** Quando il buffer è pieno, il gestore deve decidere quale pagina "scaricare" per fare spazio a quella nuova. Politiche comuni sono LRU (*Least Recently Used*) o Clock.
- **Dirty Page:** Una pagina modificata in memoria ma non ancora scritta su disco è detta "sporca". Viene scritta su disco solo quando necessario per minimizzare l'I/O (scrittura pigra o *lazy*).

3.3 Recovery Management

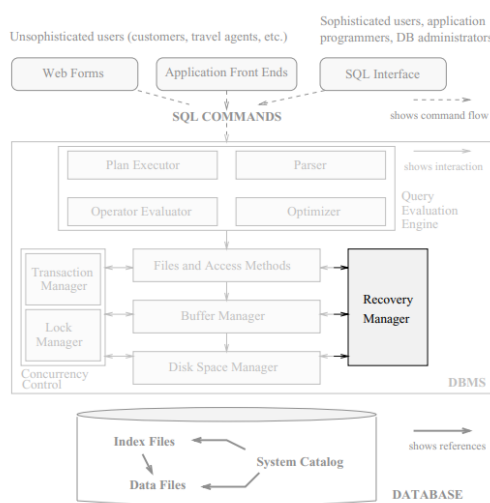


Figure 4: Recovery Manager e gestione dei crash

Questo livello garantisce l'integrità del database e la gestione degli accessi concorrenti tramite due moduli che collaborano strettamente.

3.3.1 Recovery Manager (Gestore del Ripristino)

Il suo obiettivo è garantire la durabilità e l'atomicità, proteggendo i dati da guasti (crash di sistema, blackout).

Disallineamento RAM/Disco Per efficienza, il Buffer Manager ritarda le scritture su disco. In caso di crash, il contenuto della RAM viene perso. Il Recovery Manager risolve questo problema utilizzando tecniche come il **WAL (Write-Ahead Logging)**: ogni modifica viene scritta in un file di Log (sicuro e sequenziale) *prima* che la modifica venga applicata al database vero e proprio.

Operazioni di Ripristino Al riavvio dopo un crash, il sistema analizza il Log ed esegue:

- **Undo (Annullamento):** Annulla le modifiche di transazioni che erano attive ma non completate al momento del crash (garantisce l'Atomicità).
- **Redo (Rifacimento):** Ripplica le modifiche di transazioni che erano state confermate (commit) ma le cui pagine di dati non erano ancora state scritte su disco (garantisce la Durabilità).

Esempio: In un bonifico bancario, se i soldi vengono tolti dal conto A (operazione 1) ma il sistema crasha prima di aggiungerli al conto B (operazione 2), il Recovery Manager tramite *Undo* annulla il prelievo su A, riportando il database a uno stato consistente.

3.4 Concurrency Control

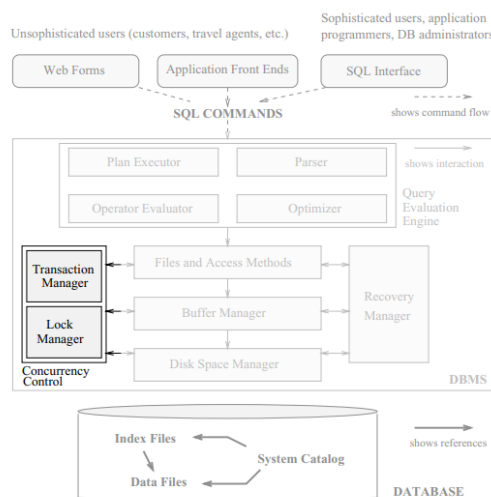


Figure 5: Concurrency Control e gestione dei Lock

3.4.1 Lock Manager

Gestisce l'accesso simultaneo di più utenti ai dati per evitare conflitti. Il modulo principale è il **Lock Manager**, che assegna permessi di accesso:

- **Lock Condiviso (S):** Permette la lettura. Più transazioni possono detenere un lock S sullo stesso oggetto.
- **Lock Esclusivo (X):** Permette la scrittura (e lettura). Solo una transazione alla volta può averlo.

I lock possono avere diversa *granularità*: Database, Tabella, Pagina o Record. Una granularità fine (record) aumenta la concorrenza ma anche l'overhead di gestione.

Deadlock Un rischio dei lock è lo stallo (*deadlock*): Transazione A aspetta una risorsa bloccata da B, mentre B aspetta una risorsa bloccata da A. Il Lock Manager deve rilevare e risolvere queste situazioni, solitamente abortendo una delle due transazioni.

3.4.2 Transaction Manager e Proprietà ACID

Il Transaction Manager coordina le operazioni per garantire che un insieme di azioni venga trattato come un'unica unità logica. Deve soddisfare le proprietà **ACID**:

- **A - Atomicità (All-or-Nothing):** La transazione è indivisibile. O vengono eseguite tutte le operazioni, o nessuna. In caso di errore, si esegue il *Rollback*.
- **C - Consistenza:** La transazione deve portare il DB da uno stato valido a un altro stato valido, rispettando tutti i vincoli di integrità (es. chiavi primarie, vincoli check).

- **I - Isolamento:** L'esecuzione concorrente di più transazioni non deve interferire l'una con l'altra. Ogni transazione deve "credere" di essere l'unica in esecuzione.
- **D - Durabilità:** Una volta che una transazione ha fatto *Commit* (conferma), le sue modifiche sono permanenti e non andranno perse, nemmeno in caso di crash catastrofico.

3.4.3 Dettagli sull'implementazione ACID e Log

Per garantire l'efficienza, il **Transaction Manager** gestisce un **File di Log** dove registra le operazioni per eventuale undo/redo. Tuttavia, per non rallentare il sistema, anche il log viene inizialmente scritto in memoria volatile. Questo introduce la necessità di protocolli avanzati per garantire che, al momento del *commit*, i dati siano effettivamente persistenti (Durabilità) nonostante il passaggio in RAM.

Le responsabilità delle proprietà ACID sono così ripartite:

- **Atomicità e Durabilità:** Gestite dal **Recovery Manager** (tramite log e ripristino).
- **Isolamento e Consistenza:** Gestite dal **Concurrency Control** (cooperazione tra Lock Manager e Transaction Manager).

Nota sul Locking: Esiste un trade-off fondamentale. Un locking molto rigido previene errori e dead-lock, ma riduce il *throughput* (transazioni per secondo) impedendo la parallelizzazione. I DBMS moderni cercano un equilibrio per massimizzare le prestazioni senza compromettere l'integrità.

Precisazione sull'Ottimizzatore L'**Optimizer** non cerca sempre la soluzione ottima in senso assoluto, poiché calcolarla richiederebbe troppo tempo. Utilizza invece delle **euristiche** per scartare i piani peggiori e sceglierne uno "sufficientemente buono" in tempi rapidi.

3.5 Query Evaluation Engine

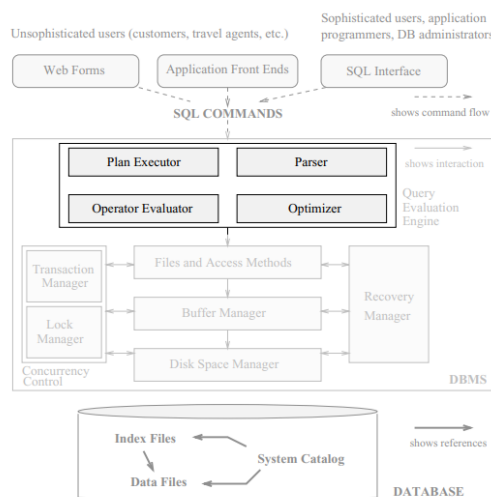


Figure 6: Flusso di esecuzione di una Query

È la parte "intelligente" del DBMS, responsabile dell'elaborazione delle richieste utente (solitamente in SQL).

Il processo si divide in quattro fasi principali:

1. Parser (Analizzatore):

- Verifica la correttezza sintattica (es. SQL scritto bene).
- Verifica la semantica (es. le tabelle e colonne esistono? L'utente ha i permessi?).
- Traduce la query in un albero di espressione dell'algebra relazionale.

2. Query Optimizer (Ottimizzatore):

- È il cervello del sistema. Per una stessa query esistono molti modi di eseguirla (piani di esecuzione).
- Utilizza le statistiche del *System Catalog* per stimare il costo (I/O, CPU) di ogni piano.
- Sceglie il piano di esecuzione più efficiente (es. decide se usare un indice o scansionare tutta la tabella).

3. Plan Executor:

- Prende il piano scelto dall'ottimizzatore e lo esegue, coordinando le operazioni.

4. Operator Evaluator:

- Implementa i singoli algoritmi per le operazioni relazionali (Join, Select, Project, Sort). Interagisce con il Buffer Manager per recuperare le pagine necessarie.

4 Dischi e Gestione della Memoria Secondaria

Il DBMS memorizza le informazioni su dischi magnetici (*hard disks*), dispositivi di memoria secondaria che garantiscono la **persistenza** dei dati: a differenza della RAM (volatile), i dati su disco vengono mantenuti anche dopo lo spegnimento del server.

Le implicazioni principali per il design di un DBMS derivano dalla differenza di velocità tra memoria centrale e disco:

- **Costo delle operazioni:** Le operazioni di I/O (Input/Output), ovvero *Read* (trasferimento Disco → RAM) e *Write* (trasferimento RAM → Disco), sono estremamente costose in termini di tempo rispetto alle operazioni in memoria centrale.
- **Obiettivo:** Il DBMS deve essere progettato per minimizzare il numero di accessi al disco, poiché sono il vero collo di bottiglia del sistema.

4.1 Architettura Fisica del Disco

Un disco rigido è costituito da una pila di piatti (superfici circolari magnetiche) che ruotano continuamente attorno a un perno centrale.

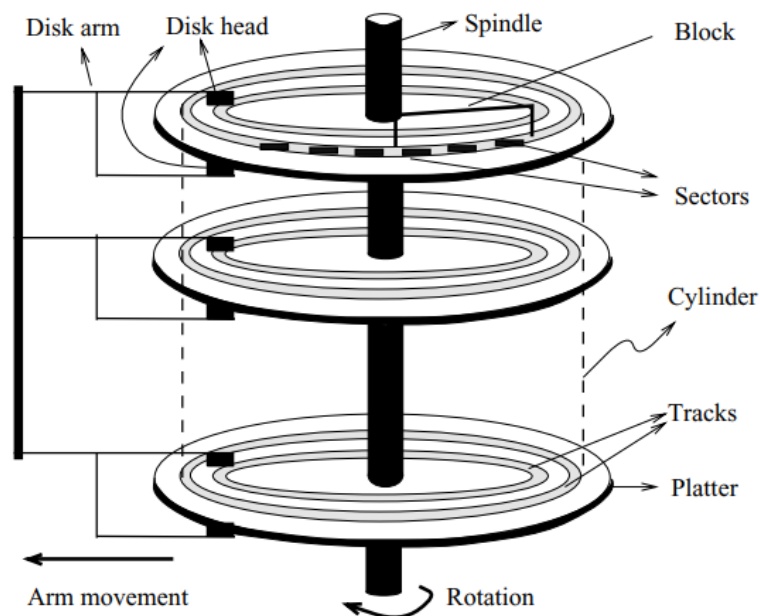


Figure 7: Struttura di un disco rigido: piatti, testine, braccio meccanico e cilindri.

L'organizzazione fisica è la seguente:

- **Piatti e Tracce:** I dati sono memorizzati su cerchi concentrici chiamati **tracce**.
- **Testine:** La lettura/scrittura avviene tramite testine montate su un unico braccio meccanico (attuatore). Le testine si muovono solidalmente: non possono accedere a punti diversi indipendentemente.
- **Cilindro:** L'insieme delle tracce che si trovano alla stessa distanza dal centro su tutti i piatti impilati forma un cilindro.
- **Settori:** Ogni traccia è divisa in settori. Un blocco logico (o **pagina**) corrisponde tipicamente a uno o più settori fisici ed è l'unità minima di lettura/scrittura (non è possibile leggere mezzo record, si legge sempre l'intera pagina).

4.2 Tempo di Accesso e Componenti di Costo

L'accesso ai dati su disco non è uniforme (come nella RAM, che è *Random Access Memory*), ma dipende dalla posizione fisica dei dati e dal movimento meccanico. Il tempo totale di accesso è la somma di tre componenti:

- **Seek Time** (1-20 ms): È il tempo necessario per spostare il braccio meccanico in modo che la testina si posizioni sulla traccia corretta.
 - È la componente più costosa (movimento meccanico macroscopico).
 - Dipende dalla distanza tra la posizione attuale della testina e quella di destinazione. Se i dati sono sulla stessa traccia, il seek time è nullo.
- **Rotational Latency** (0-10 ms): Una volta posizionata la testina sulla traccia, bisogna attendere che il disco ruoti finché il settore desiderato non passa sotto la testina.
 - Nel caso peggiore serve un giro completo; nel caso medio, mezzo giro.
 - I dischi ruotano costantemente per evitare l'inerzia di stop-and-go.
- **Transfer Time**: È il tempo necessario per il passaggio effettivo dei dati dalla superficie magnetica alla memoria principale (lettura vera e propria). È molto veloce rispetto ai primi due.

4.3 Strategie di Ottimizzazione e "Next Block"

Poiché il Seek Time è la fase più lenta, l'obiettivo principale nella memorizzazione dei file è massimizzare l'**accesso sequenziale** e minimizzare quello casuale (*random*).

Si introduce il concetto logico di **Next Block**: se un file occupa più pagine, il DBMS deve decidere dove allocare la pagina successiva per minimizzare i tempi di lettura futuri.

L'ordine di preferenza per l'allocazione ("Next Block") è:

1. **Stessa Traccia**: La soluzione ideale. Nessun movimento del braccio (Seek = 0) e attesa di rotazione minima.
2. **Stesso Cilindro**: Se la traccia è piena, si usa la traccia corrispondente su un altro piatto (stesso cilindro). Richiede solo il cambio elettronico della testina attiva, ma nessun movimento meccanico del braccio.
3. **Cilindro Adiacente**: Se il cilindro è pieno, si passa a quello fisicamente più vicino per ridurre al minimo il Seek Time.

4.4 Pre-fetching e Prevedibilità

Sfruttando il principio di località spaziale e l'organizzazione sequenziale (Next Block), il sistema può applicare il **Pre-fetching**:

- Quando viene richiesta una pagina, il sistema intuisce (prevedibilità) che probabilmente serviranno anche le pagine successive.
- Invece di fermare la lettura, il disco continua a leggere e carica in memoria (nel Buffer) anche le pagine adiacenti che passano sotto la testina.
- Questo trasforma molteplici potenziali richieste di I/O future in un'unica operazione sequenziale attuale, abbattendo drasticamente i costi.

4.5 File Manager e Organizzazione dei Dati

Il **File Manager** è il componente del DBMS responsabile della gestione dello spazio su disco e delle strutture dati utilizzate per memorizzare le informazioni. Possiamo classificare i file in due macro-categorie principali:

- **File di Dati:** Contengono i record effettivi delle tabelle (il contenuto del database). Possono essere organizzati come *Heap File* (non ordinati) o *Sorted File* (ordinati).
- **File di Indice:** Strutture ausiliarie (es. B+ Tree, Hash) che velocizzano l'accesso ai dati. Non contengono i dati completi, ma associano una chiave di ricerca alla posizione fisica del dato.

Fondamentale per il collegamento tra questi file è il concetto di **RID (Record ID)**.

RID (Record Identifier): È l'identificatore univoco e invariante di un record. Rappresenta l'indirizzo fisico o logico che permette al DBMS di recuperare esplicitamente un record specifico.

4.5.1 Strategie di Organizzazione dei File

La scelta di come organizzare i file su disco influenza drasticamente le prestazioni in lettura e scrittura. Non esiste una soluzione ottima universale, ma diverse alternative:

1. Heap Files (File Non Ordinati) È l'organizzazione più semplice. I record sono accumulati nel file senza un ordine specifico (spesso in ordine di arrivo/inserimento).

- **Inserimento:** Molto efficiente ($O(1)$). I nuovi dati vengono semplicemente accodati all'ultima pagina disponibile (*append-only*).
- **Ricerca:** Inefficiente ($O(N)$). Per trovare un record specifico è necessario scansionare l'intero file (linear scan), leggendo tutte le pagine.

2. Sorted Files (File Ordinati) I record sono memorizzati fisicamente in ordine sequenziale in base al valore di uno o più campi (*chiave di ordinamento*), nel caso di più campi l'ordine si fa su tutti i campi insieme non per il singolo. Un sorted files può essere ordinato per una **singola chiave di ordinamento!**

In presenza di più chiavi di ordinamento, devo scegliere una sola, solitamente quella più frequentemente usata nelle query di ricerca. Essendo i file solitamente memorizzati in modo contiguo, si prestano per la ricerca binaria che ha costo logaritmo in base al numero di record, scegliere come chiave di ordinamento un campo con un grande carico di lavoro di ricerca è fondamentale per le prestazioni. Es. `SELECT * WHERE cognome = "Rossi"`, se il file non è ordinato devo scansionare tutti i record, se è ordinato per `Cognome` posso usare la ricerca binaria e trovare tutti i record con cognome "Rossi" in modo efficiente e appena ne trovo uno diverso mi posso fermare perché sono certo che non ne troverò altri più avanti.

Es. se l'ordinamento è su (`Cognome`, `Nome`), i record sono ordinati prima per `Cognome` e in caso di parità per `Nome`.

- **Ricerca:** Molto efficiente. È possibile utilizzare la **Ricerca Binaria**, che ha costo $O(\log_2 N)$. Anche senza un indice, si accede alla pagina centrale e ci si sposta a destra o sinistra.
- **Manutenzione (Scrittura/Modifica):** Costosa. Per mantenere l'ordinamento fisico:
 - Bisogna trovare la posizione corretta per il nuovo record.
 - Se non c'è spazio nella pagina, è necessario effettuare uno **shifting** (spostamento) di tutti i record successivi per creare lo spazio. Questo può propagarsi a cascata sulle pagine seguenti.
- **Strategia del Buffer (Fill Factor):** Per mitigare il costo dello shifting, le pagine non vengono mai riempite al 100%. Si lascia una porzione di spazio libero (es. 20%) in ogni pagina per accomodare futuri inserimenti o espansioni dei record senza dover spostare dati tra pagine diverse.

3. Index Files (Indici) Sono strutture dati progettate per fornire **accesso diretto** ai record senza scansionare il file dati.

- Associano una *Chiave di Ricerca* a un *RID*.
- **Tipi comuni:**
 - *Hash Index*: Ottimi per ricerche di uguaglianza esatta (es. $ID = 5$).
 - *Tree Index* (es. *B+ Tree*): Ottimi per ricerche per intervallo (es. $Età > 18$).
- **Trade-off**: Gli indici occupano spazio aggiuntivo su disco e rallentano le operazioni di scrittura (poiché a ogni INSERT o DELETE nella tabella bisogna aggiornare anche l'indice), ma velocizzano drasticamente le letture.

5 Struttura Interna: Pagine e Record

La **pagina** è l'unità atomica di trasferimento I/O tra disco e memoria. Un file è una sequenza di pagine. All'interno delle pagine risiedono i **record**. Esistono due approcci principali per la rappresentazione dei record:

5.1 Record

5.1.1 Record a Lunghezza Fissa (Fixed-Length)

In fase di schema design, si stabilisce una dimensione fissa in byte per ogni attributo (es. CHAR(20), INTEGER). Tutti i record della tabella avranno la stessa dimensione totale.

- **Vantaggi (Prevedibilità):**

- L'accesso ai campi è immediato tramite aritmetica dei puntatori. Se so che i primi due campi occupano 10 byte totali, il terzo campo inizierà sempre al byte 11.
- Non serve memorizzare metadati sulla lunghezza.

- **Svantaggi:**

- **Frammentazione Interna (Spreco):** Se un campo CHAR(100) contiene solo "Ciao", vengono sprecati 96 byte. I dati nulli occupano comunque spazio fisico.
- Sprecare spazio significa memorizzare meno record per pagina, aumentando il numero di I/O necessari per leggere la tabella.

5.1.2 Record a Lunghezza Variabile (Variable-Length)

Necessari quando i dati hanno lunghezze eterogenee (es. VARCHAR, descrizioni testuali). Il record si adatta al contenuto effettivo.

Problematiche: viene meno la prevedibilità posizionale. Non so a priori dove inizia il terzo campo senza sapere quanto sono lunghi i primi due.

Esistono due tecniche implementative principali:

1. Uso di Separatori (Delimiters)

- I campi sono separati da un carattere speciale (es. |, \$).
- **Gestione NULL:** efficiente, basta mettere due separatori adiacenti (||).
- **Svantaggi:** accesso lento. Per leggere l'ultimo campo devo scansionare byte per byte tutto il record cercando i separatori. Inoltre, se il carattere separatore compare nei dati, deve essere gestito con sequenze di escape, aumentando la complessità.

2. Directory di Puntatori (Record Header) [Approccio consigliato]

- All'inizio del record (testa) viene inserita una piccola "directory" che contiene i puntatori (o *offset*) relativi alla fine di ogni campo variabile.
- **Vantaggi:**
 - **Accesso $O(1)$:** per leggere il campo X , leggo l'offset nell'header e salto direttamente a quella posizione di memoria, senza scansionare il resto.
 - **Overhead ridotto:** lo spazio occupato dagli offset è minimo rispetto allo spreco dei separatori o dei record fissi.
- **Vincoli:** la dimensione in bit riservata per i puntatori nell'header determina la dimensione massima teorica del record (es. se uso 16 bit per l'offset, il record non può superare 64KB).

5.2 Organizzazione di una pagina

La pagina è l'unità atomica di I/O. La sfida principale è come organizzare i record al suo interno per massimizzare lo spazio e gestire efficientemente inserimenti e cancellazioni.

5.2.1 Record a lunghezza fissa

In questo scenario, tutti i record hanno la stessa dimensione L . La pagina può essere vista come un array di slot di dimensione L .

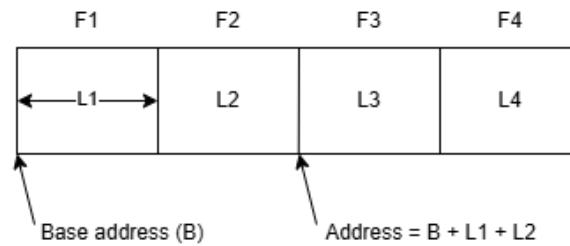


Figure 8: Record a lunghezza fissa

L'informazione essenziale da mantenere nell'header della pagina è il numero di record attualmente presenti. Esistono due strategie principali di gestione:

1. **Gestione Packed (Contigua):** i record occupano sempre le prime N posizioni (slot) della pagina senza lasciare buchi.
 - **Inserimento:** molto veloce, si scrive nel primo slot libero alla fine.
 - **Cancellazione:** quando si rimuove un record, si crea un "buco". Per mantenere la contiguità (packed), si sposta l'ultimo record della pagina nel buco appena creato (oppure si traslano tutti i successivi, ma è costoso).
 - **Problema dei RID:** spostando i record per colmare i buchi, il loro indirizzo fisico cambia. Questo invalida tutti i puntatori esterni (indici) che puntavano a quel record tramite il RID. È un approccio sconsigliato se ci sono indici esterni.
2. **Gestione Unpacked (Bitmap):** i record vengono inseriti nel primo slot libero, ma non vengono mai spostati dopo l'inserimento. I buchi rimangono vuoti finché non viene inserito un nuovo record.
 - **Gestione:** l'header della pagina contiene una **Bitmap** con un bit per ogni slot (1 = pieno, 0 = vuoto).
 - **Vantaggio (RID stabili):** poiché i record non si spostano mai, il RID (costituito da *PageID* + *SlotNumber*) rimane valido per tutta la vita del record. Non è necessario aggiornare gli indici esterni in caso di cancellazione di altri record nella stessa pagina.

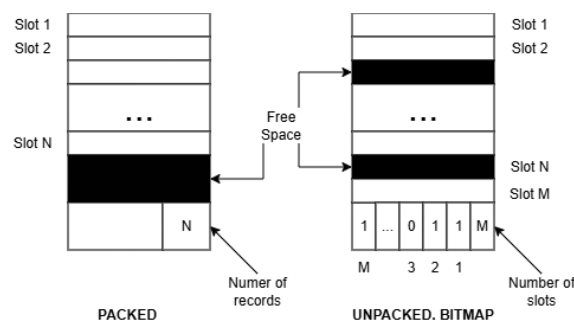


Figure 9: Organizzazione di una pagina con record a lunghezza fissa (con Bitmap)

5.2.2 Record a lunghezza variabile (Slotted Page)

Quando i record hanno lunghezza variabile, non possiamo dividere la pagina in slot fissi. La struttura standard utilizzata dai DBMS è la **Slotted Page Organization**.

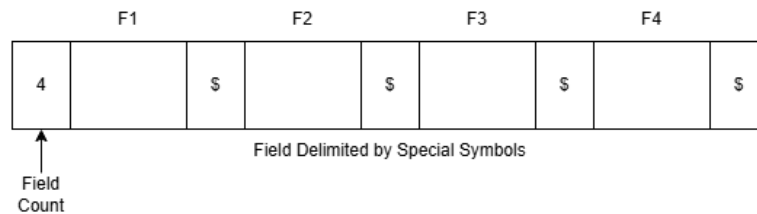


Figure 10: Record a lunghezza variabile

La gestione dello spazio libero diventa critica: un buco lasciato da un record cancellato di 20 byte non può ospitare un nuovo record di 30 byte (frammentazione).

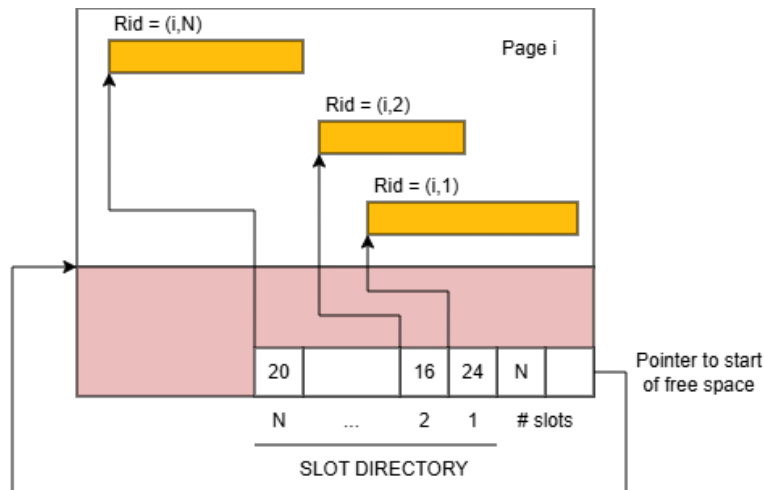


Figure 11: Slotted Page: Organizzazione di una pagina con record a lunghezza variabile

La struttura tipica prevede:

- **Footer/Directory (in basso):** contiene un array di slot. Ogni slot contiene una coppia $\langle \text{Offset}, \text{Lunghezza} \rangle$ che punta all'inizio del record corrispondente all'interno della pagina.
- **Dati (in alto):** i record vengono inseriti contigualmente dall'inizio della pagina verso il fondo.
- **Spazio Libero (al centro):** si trova tra l'ultimo record inserito e l'inizio della directory degli slot.

Compattazione e Indirizione: se un record viene cancellato, si crea un buco. Il sistema può spostare i record adiacenti per compattare lo spazio libero ("de-frammentazione"). Grazie alla directory degli slot, quando spostato un record fisicamente, aggiorniamo solo il suo **offset** nel footer. Il RID esterno punta all'indice dello slot (che non cambia), non all'indirizzo fisico del byte. Questo garantisce che i puntatori esterni rimangano validi anche se i dati si muovono all'interno della pagina.

5.3 Heap File

5.3.1 Heap File: Implementazione a Lista Concatenata

Un Heap File è un insieme di pagine senza ordinamento. Una possibile implementazione usa liste doppiamente concatenate gestite tramite una pagina header.

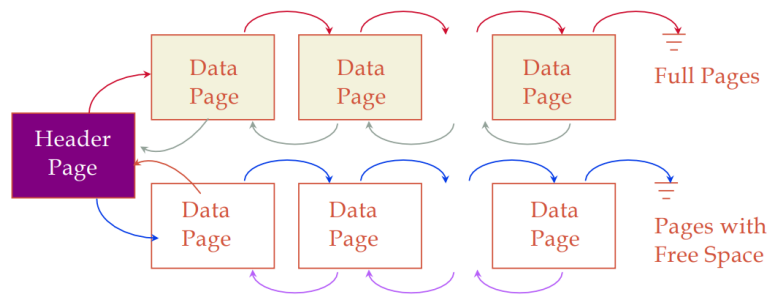


Figure 12: Organizzazione di un heap file come lista

Il DBMS mantiene due liste distinte:

1. **Full Pages List:** lista delle pagine completamente piene (o con spazio insufficiente per nuovi record). Non vengono consultate durante l'inserimento.
2. **Free Space List:** lista delle pagine che hanno almeno un po' di spazio libero.

Funzionamento:

- Per inserire un record, si scorre la lista delle pagine con spazio libero. Se la prima pagina ha spazio sufficiente, si inserisce. Se diventa piena, viene spostata nella lista delle "Full Pages".
- **Svantaggio:** se cerco spazio per un record molto grande, potrei dover scorrere molte pagine della "Free List" che hanno spazio libero ma insufficiente, generando I/O inutile.

5.3.2 Heap File: Implementazione a Directory

Questa è l'implementazione più efficiente e diffusa.

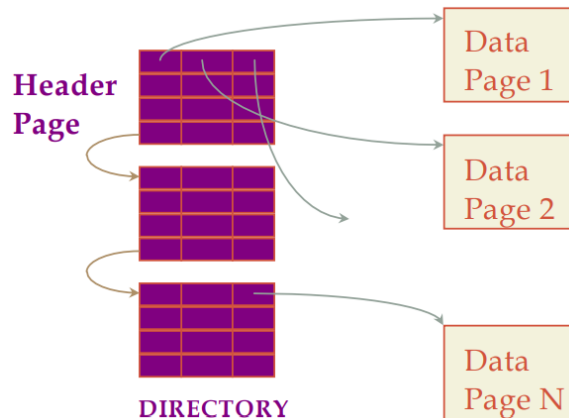


Figure 13: Organizzazione di un heap file usando una directory

Invece di liste, si usa una **Directory Page** (o un insieme di pagine directory) che mappa lo stato di ogni pagina del file dati.

Ogni voce della directory contiene:

- Puntatore alla pagina (PageID).
- **Free Space Count:** numero di byte liberi in quella pagina (o un indicatore approssimativo, es. 2 bit per indicare vuota, mezza piena, piena).

Vantaggi:

- La directory è molto compatta (una pagina di directory gestisce migliaia di pagine dati).
- Per inserire un record, il DBMS consulta la directory (in RAM o con 1 I/O) e trova subito una pagina con spazio sufficiente, senza dover caricare e controllare le pagine dati una per una.

6 Indici

Gli **indici** sono strutture ausiliarie progettate per velocizzare il recupero dei record in base a una chiave di ricerca, evitando la scansione sequenziale del file.

Un indice contiene una collezione di **Data Entries** (voci dei dati), denotate come k^* , che permettono di trovare i record con chiave k .

Esistono tre alternative principali per memorizzare le voci k^* all'interno di un indice:

- **Alternativa 1 (Indice Clusterizzato / Index-Organized):** la struttura dell'indice contiene il **record dati completo**.

$$k^* = \langle \text{Intero Record} \rangle$$

In questo caso non c'è un file dati separato; l'indice è il file. La ricerca è rapidissima, ma l'indice diventa molto grande.

- **Alternativa 2 (Indice $\langle K, RID \rangle$):** l'indice contiene la coppia chiave e RID del record.

$$k^* = \langle k, RID \rangle$$

Se ci sono più record con la stessa chiave k (duplicati), avrò più voci distinte nell'indice (es. $\langle k, RID_1 \rangle, \langle k, RID_2 \rangle$).

- **Alternativa 3 (Indice $\langle K, \text{ListaRID} \rangle$):** l'indice contiene la chiave e una lista di tutti i RID che soddisfano quella chiave.

$$k^* = \langle k, (RID_1, RID_2, \dots, RID_n) \rangle$$

Questo formato è più compatto per chiavi con molti duplicati, poiché la chiave k viene memorizzata una volta sola.

6.1 Ricerca per intervallo (Range Search)

Le ricerche per intervallo (ad esempio, `SELECT * WHERE Età > 18`) risultano particolarmente efficienti quando si utilizzano indici ordinati (come gli alberi B+ o le strutture ISAM). Questi indici, infatti, supportano l'accesso sequenziale ai record di dati (le voci k^*) seguendo l'ordine imposto dalla chiave di ricerca, minimizzando i salti sul disco.

6.2 ISAM (Indexed Sequential Access Method)

L'ISAM è una struttura dati ad albero ideata per ottimizzare la gestione di file ordinati tramite un indice primario. Il suo design è fortemente orientato alle prestazioni su disco: i vari nodi dell'albero sono memorizzati in modo contiguo per massimizzare l'efficienza degli accessi sequenziali, rendendo superflui i puntatori fisici tra nodi adiacenti dello stesso livello.

La creazione di un file ISAM avviene per fasi sequenziali:

1. **Allocazione delle pagine dati (foglie):** Vengono scritte su disco in sequenza, riempiendole con i record già ordinati secondo la chiave di ricerca.
2. **Allocazione delle pagine indice:** Vengono generate le pagine non foglia, contenenti coppie del tipo $\langle \text{chiave}, \text{PageID} \rangle$, che fungono da guida per la navigazione verso le pagine dati.
3. **Allocazione dello spazio di overflow:** Viene riservata un'area del disco destinata a ospitare i record delle future operazioni di inserimento.

Operazioni principali

- **Voci di indice (Index entries):** Formattate come $\langle \text{valore chiave}, \text{PageID} \rangle$, indirizzano la ricerca incanalandola verso il sotto-albero che contiene la pagina dati corretta.

- **Ricerca:** Inizia dalla radice e confronta la chiave cercata con quelle presenti nel nodo. Seguendo i puntatori, si scende fino a raggiungere la pagina foglia desiderata. Il costo computazionale è pari a $\mathcal{O}(\log_F N)$, dove F è il *fan-out* (numero di voci per pagina indice) ed N è il numero di pagine dati.
- **Inserimento:** Si individua la pagina dati di destinazione. Se la pagina dispone di spazio libero, il record viene inserito al suo interno; se è piena, il record viene collocato in una **pagina di overflow** concatenata alla foglia originale.
- **Cancellazione:** Si individua e si rimuove il record dalla pagina dati o di overflow in cui risiede. Se, a seguito di cancellazioni, una pagina di overflow si svuota completamente, questa viene deallocata.

Una caratteristica fondamentale dell'ISAM è la sua natura **statica**: una volta inizializzato il file, la struttura dell'albero delle pagine indice non subisce alcuna modifica (non avvengono né split né merge dei nodi). Le operazioni dinamiche (inserimenti e cancellazioni) sono gestite unicamente tramite le pagine dati e le catene di overflow. Questo approccio evita il costo di riarrangiamento dell'albero ma, in presenza di frequenti inserimenti, la crescita eccessiva delle catene di overflow porta a un drastico degrado delle prestazioni. In tal caso, si rende necessaria la ricostruzione periodica dell'intero file.

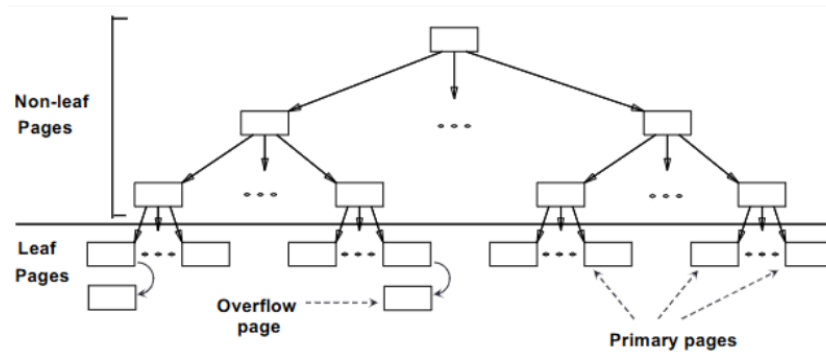


Figure 14: Rappresentazione della struttura di un albero ISAM

Vantaggi e Svantaggi

- **Vantaggi:** Struttura estremamente compatta e assenza di spazio sprecato. È la soluzione ideale per scenari statici o con rari aggiornamenti, poiché l'assenza di operazioni di bilanciamento rende l'accesso in sola lettura molto rapido.
- **Svantaggi:** Forte degrado delle prestazioni in scenari dinamici a causa dell'allungamento delle catene di overflow. Inoltre, presupponendo che il file dati sia ordinato fisicamente, è possibile costruire al massimo un solo indice ISAM primario (o *clustered*) per ogni relazione.

Esempio di inserimento Si consideri l'inserimento sequenziale dei record con chiave di ricerca 23*, 48*, 41*, 42*. Se le pagine foglia primarie sono piene, questi record andranno a popolare le relative catene di overflow.

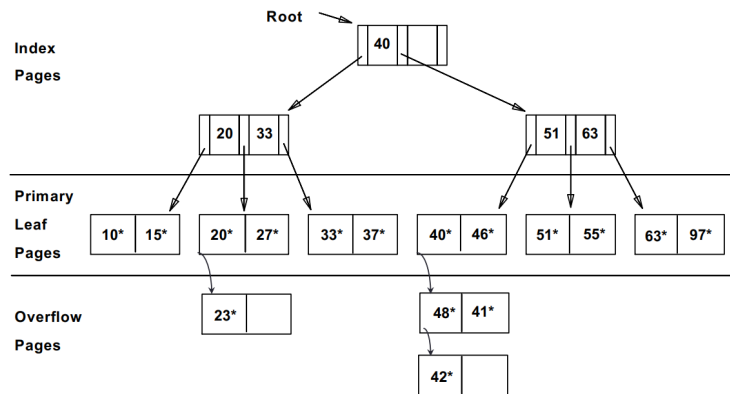


Figure 15: Evoluzione dell'albero ISAM a seguito di inserimenti (con utilizzo di pagine di overflow)

Esempio di cancellazione Consideriamo ora la cancellazione delle chiavi 42*, 51*, 97*.

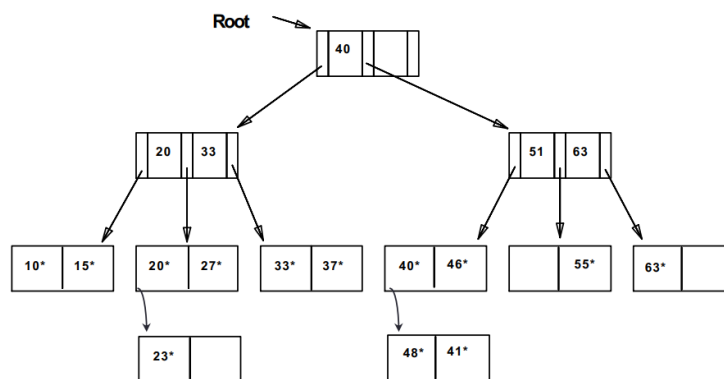


Figure 16: Stato dell'albero ISAM dopo l'operazione di cancellazione

Osservando la figura, è possibile notare che se la chiave 51* fungeva da separatore, essa **continuerà ad apparire nelle pagine indice** anche dopo essere stata rimossa dalle foglie. Questo è un effetto diretto della natura statica dell'ISAM, che non aggiorna la struttura di routing.

Inoltre, la gestione dello spazio vuoto differisce in base al tipo di pagina: se si cancella un record (ad esempio 41*) da una pagina di overflow, i record successivi (come 42*) vengono compattati traslando verso l'alto, senza lasciare "buchi". Al contrario, nelle pagine foglia primarie, la rimozione di un record lascia tipicamente uno spazio vuoto non ricompattato.

6.3 Indici B+ Tree

I B+ Tree sono strutture dati gerarchiche, dinamiche e bilanciate, ideate specificamente per superare le rigidità strutturali tipiche dell'ISAM. A differenza di quest'ultimo, i B+ Tree si adattano in modo dinamico ai cambiamenti dei dati, mantenendo un'efficienza costante senza la necessità di ricalcolare l'intera struttura o affidarsi a catene di overflow.

Come gli ISAM, i B+ Tree sono organizzati in livelli di nodi che hanno dimensione pari a quella di una pagina su disco. Un'altra similarità è il fatto che entrambi sono organizzati in strutture gerarchiche.

Nei B+ Tree, il file può essere come non ordinato, ma non è necessario che lo sia, poiché tutte le chiavi di ricerca vengono replicate e ordinate a livello delle foglie. Se il file non è ordinato deve avere un livello soprastante di foglie dove sono presenti tutte le chiavi di ricerca, in modo ordinato, ciascuno con il puntatore al record reale, perché non potrei sfruttare il maggiore e minore per navigare l'albero.

Struttura e Caratteristiche

- **Nodi interni (Pagine indice):** Contengono esclusivamente voci di routing (chiavi di separazione e puntatori) per indirizzare la ricerca verso il sotto-albero corretto. Non contengono i dati effettivi.

- **Nodi foglia (Pagine dati):** Contengono le voci di dato (o i puntatori ai record reali) e sono mantenuti rigorosamente ordinati in base alla chiave di ricerca.
- **Concatenazione delle foglie (Sequence Set):** Poiché nei B+ Tree i nodi dello stesso livello **non sono allocati contigualmente** su disco (vengono salvati dove c'è spazio), le pagine foglia sono collegate tra loro tramite puntatori (*next* e *previous*), formando una lista doppiamente concatenata. Questo garantisce un accesso sequenziale estremamente efficiente. Questi puntatori sono trasparenti all'utente e vengono gestiti dal sistema.
- **Indipendenza dall'ordinamento fisico:** Non è richiesto che il data file sottostante sia fisicamente ordinato. Questo permette di definire molteplici indici (B+ Tree) sulla stessa tabella per diverse chiavi di ricerca (*unclustered index*). Tutte le chiavi vengono infatti replicate e ordinate a livello delle foglie.
- **Vincolo di riempimento (Occupancy):** Per mantenere l'albero bilanciato e le prestazioni prevedibili, ogni nodo (fatta eccezione per la radice) deve mantenere un fattore di riempimento compreso tra il 50% e il 100% della sua capienza massima. Viene deliberatamente lasciato dello spazio libero per assorbire futuri inserimenti limitando le modifiche all'architettura dell'albero.

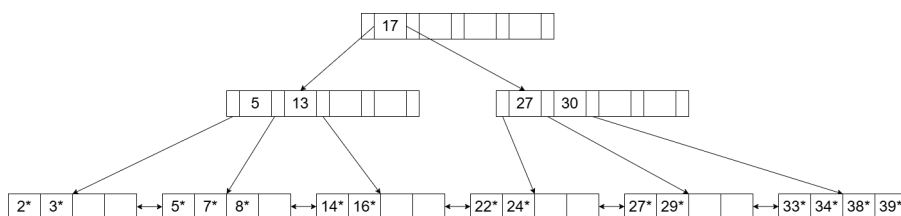


Figure 17: Rappresentazione della struttura e dei collegamenti a livello foglia in un albero B+ Tree

Gestione Dinamica: Inserimento e Cancellazione La forte dinamicità del B+ Tree si esprime nella sua capacità di autobilanciarsi a seguito di operazioni di aggiornamento:

- **Inserimento (Split):** Si individua la foglia corretta. Se il nodo è pieno, avviene uno *split*: il nodo viene diviso in due, i dati vengono equamente distribuiti e la chiave mediana viene **copiata** (se si divide una foglia) o **promossa** (se si divide un nodo interno) al nodo padre. Se anche il padre risulta pieno, l'operazione di split si propaga verso l'alto (*cascade*). Se lo split raggiunge la radice e la divide, viene creata una nuova radice, incrementando l'altezza dell'albero di un livello.
- **Cancellazione (Merge e Redistribuzione):** Si rimuove l'elemento dalla foglia. Se il livello di riempimento del nodo scende al di sotto del 50%, si viola il vincolo strutturale. L'albero tenta prima una **redistribuzione**, prendendo in prestito elementi da un nodo "fratello" adiacente che ha spazio in abbondanza. Se i nodi adiacenti sono a loro volta al limite minimo del 50%, si procede con un **merge** (fusione): i due nodi vengono accorpati e la chiave di routing associata nel padre viene eliminata. Anche il merge può propagarsi a cascata verso la radice, arrivando potenzialmente a ridurre l'altezza complessiva dell'albero.

Questa operazione è molto costosa, quindi si cerca di rimandarla il più possibile sperando che arrivino nuovi inserimenti che riempiano i nodi prima di doverli unire.

6.3.1 Impatto della Dimensione della Chiave sulle Prestazioni

La dimensione in byte della chiave di ricerca ha un impatto diretto e profondo sulle prestazioni complessive dell'albero B+ Tree, poiché determina il **fan-out** (il numero massimo di figli per nodo).

- **Chiavi semplici (es. Interi):** Occupano pochi byte. Questo permette di inserire un numero elevato di voci all'interno di una singola pagina (fan-out elevato). Un fan-out maggiore produce un albero molto "largo" e **basso**, riducendo al minimo il numero di accessi al disco (I/O) necessari per raggiungere le foglie partendo dalla radice.

- **Chiavi complesse (es. Stringhe lunghe o chiavi composte):** Occupano molto spazio. Il numero di voci per pagina diminuisce drasticamente (fan-out ridotto), costringendo l'albero a crescere in **altezza**. Ogni livello aggiuntivo dell'albero si traduce in una potenziale operazione di I/O meccanica in più durante la ricerca, degradando le prestazioni.

Il Grado dell'Albero (d) La capacità strutturale di un nodo è formalmente definita dal parametro d , chiamato **grado dell'albero**. In un B+ Tree, per garantire il bilanciamento dinamico e l'efficienza dello spazio, ogni nodo (ad eccezione della radice) deve rispettare un vincolo di riempimento minimo e massimo:

$$d \leq m \leq 2d$$

dove m è il numero di chiavi di ricerca attualmente presenti nel nodo. Di conseguenza, un nodo interno che contiene m chiavi avrà esattamente $m + 1$ puntatori ai sotto-nodi figli.

6.3.2 Compressione delle chiavi basata su prefissi (Prefix Key Compression)

Per mitigare il problema dimensionale delle chiavi complesse e massimizzare il fan-out dei nodi interni, si ricorre a tecniche di compressione. L'obiettivo è memorizzare all'interno delle pagine indice solo l'informazione strettamente necessaria per instradare la ricerca verso la foglia corretta, scartando i dati ridondanti.

Questa tecnica si rivela particolarmente efficace quando le chiavi di ricerca sono **stringhe testuali** (es. VARCHAR), sfruttando l'idea di memorizzare solo il **prefisso discriminante minimo**.

Funzionamento: Invece di copiare l'intera chiave per separare due sotto-alberi, il DBMS calcola e memorizza il prefisso più corto che sia strettamente sufficiente a distinguere il valore massimo del sotto-albero sinistro dal valore minimo del sotto-albero destro.

Esempio pratico di discriminazione Supponiamo che il DBMS debba generare una chiave di instradamento (separatore) in un nodo interno per dividere i record che iniziano per Pall1a da quelli che iniziano per Pallone.

- **Senza compressione:** La chiave di separazione nel nodo interno occuperebbe 7 byte (Pallone).
- **Con compressione:** È sufficiente utilizzare il prefisso Pall1o (5 byte). Durante la discesa dell'albero, il sistema sa che tutto ciò che è $< \text{Pall1o}$ va a sinistra (trovando Pall1a), e tutto ciò che è $\geq \text{Pall1o}$ va a destra (trovando Pallone).
- **Gestione della dinamicità:** Se in futuro inseriamo il record Pallottola e questo causa uno split generando una nuova pagina, il vecchio separatore non sarà più sufficiente. Per discriminare tra la pagina di Pallone e quella di Pallottola, il DBMS dovrà ricalcolare dinamicamente la chiave di routing allungandola a Pallot (poiché $\text{Pallon} < \text{Pallot} \leq \text{Pallottola}$).

Vantaggi, Svantaggi e Trade-off L'adozione di questa tecnica richiede un'attenta valutazione architetturale:

- **Vantaggi (Spazio e I/O):** Aumenta significativamente la densità di chiavi per pagina indice. Questo mantiene l'albero più basso e riduce drasticamente i colli di bottiglia legati all'accesso al disco.
- **Svantaggi (Overhead CPU e Manutenzione):** La compressione introduce un costo computazionale per il processore (CPU), che deve codificare e decodificare le chiavi al volo durante l'attraversamento. Inoltre, in scenari altamente dinamici, il continuo ricalcolo dei prefissi a seguito degli *split* comporta un pesante degrado delle prestazioni in scrittura.
- **Conclusione:** È una soluzione fortemente raccomandata per scenari **statici** o *read-mostly* (es. cataloghi storici, Data Warehouse), mentre la sua convenienza nei sistemi transazionali (OLTP) ricchi di INSERT e DELETE va misurata caso per caso in base all'effettivo risparmio di byte.

6.3.3 Inizializzazione di un B+ Tree: Bulk Loading

Costruire un B+ Tree inserendo i record uno ad uno (inserimento standard) può diventare un processo estremamente costoso dal punto di vista computazionale e delle operazioni di I/O, a causa dei continui attraversamenti dell'albero dall'alto verso il basso e delle frequenti operazioni di *split* e *merge* dei nodi.

Quando si deve popolare un indice con una grande mole di dati preesistenti, l'approccio ottimale è il **Bulk Loading** (caricamento massivo). Invece di modificare continuamente i puntatori in modo casuale, questo algoritmo costruisce l'albero "dal basso verso l'alto" (bottom-up) sfruttando l'ordinamento.

Il procedimento si articola in tre fasi principali:

1. **Ordinamento:** Si estraggono le *Data Entries* (k^*) e si ordinano in base alla chiave di ricerca (ad esempio, tramite un *External Merge Sort*).
2. **Creazione delle foglie:** Si allocano le pagine foglia vuote e si riempiono sequenzialmente con i dati già ordinati. Poiché i dati arrivano in ordine crescente, l'algoritmo riempie una pagina fino a una determinata **soglia di riempimento** (*Fill Factor*, es. 80%, per lasciare spazio a futuri inserimenti organici) e poi passa a una nuova pagina, modificando solo il puntatore della foglia più a destra.
3. **Costruzione dell'indice:** Man mano che le foglie vengono riempite, il sistema estrae il valore minimo della nuova pagina (chiave di separazione) e lo inserisce nel nodo padre al livello superiore. Il processo si ripete ricorsivamente verso l'alto fino a generare un'unica pagina radice.

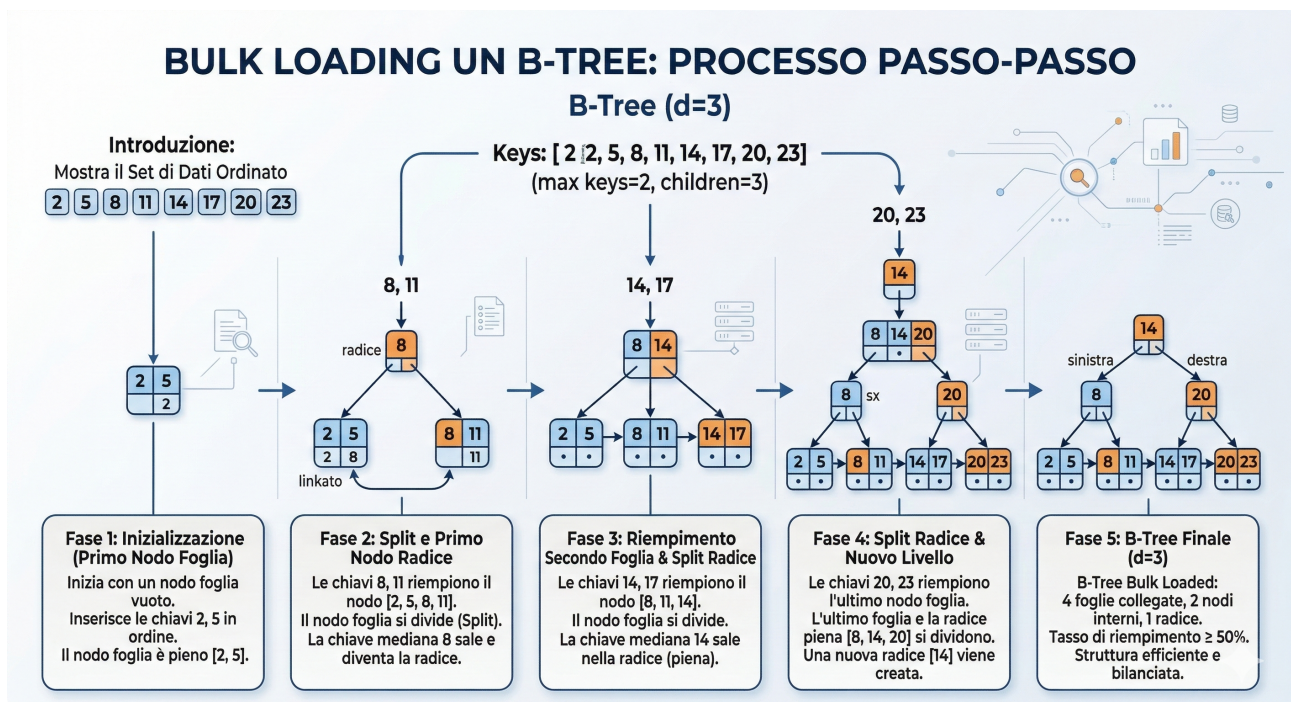


Figure 18: Esempio di bulk loading: costruzione bottom-up partendo dai dati pre-ordinati.

6.4 Classificazione degli indici

Gli indici possono essere classificati in base al loro ruolo logico rispetto allo schema della base di dati:

- **Indice Primario (Primary Index):** È l'indice costruito sulla **chiave primaria** della relazione.
 - Essendo la chiave primaria univoca per definizione, questo indice non contiene chiavi duplicate.
 - Tipicamente, i DBMS relazionali creano automaticamente questo indice di default alla definizione del vincolo PRIMARY KEY, in quanto è il campo più frequentemente utilizzato nelle operazioni di ricerca esatta.
- **Indice Secondario (Secondary Index):** È un indice ausiliario costruito su uno o più attributi che **non** costituiscono la chiave primaria (es. "Cognome", "Dipartimento").

- A differenza di quello primario, può contenere valori duplicati (a meno che non sia imposto un vincolo **UNIQUE**).
- È fondamentale per ottimizzare le query di ricerca che filtrano o ordinano i dati in base a campi secondari.

6.5 Indici Clusterizzati e Non Clusterizzati

Una classificazione critica per le prestazioni fisiche (costo di I/O) riguarda la correlazione tra l'ordine logico dell'indice e l'ordine fisico dei record nel file dati su disco.

- **Indice Clusterizzato (*Clustered Index*):** In un indice clusterizzato, l'**ordine fisico dei record** all'interno del file dati corrisponde (o è fortemente correlato) all'ordine logico della chiave di ricerca nell'indice.
 - **Gestione delle pagine:** Si parla di dati "quasi ordinati" perché, sebbene all'interno di una singola pagina i record possano subire lievi frammentazioni, le pagine nel loro complesso sono ordinate sequenzialmente tra di loro.
 - **Efficienza per Intervalli:** Garantisce prestazioni eccezionali per le **ricerche per intervallo** (*range query*). Scandendo le chiavi ordinate nelle foglie, i puntatori accedono a blocchi fisicamente contigui su disco. Visivamente, i puntatori "non si incrociano".
 - **Page Faults Minimi:** Una volta caricata in RAM la pagina per il primo record dell'intervallo, i successivi si troveranno molto probabilmente nella stessa pagina, ammortizzando drasticamente i costi di I/O (la pagina viene portata in memoria una sola volta).
 - **Unicità del Clustered:** Poiché i dati fisici non possono essere ordinati in più modi contemporaneamente, **può esistere al massimo un solo indice clusterizzato per tabella**.

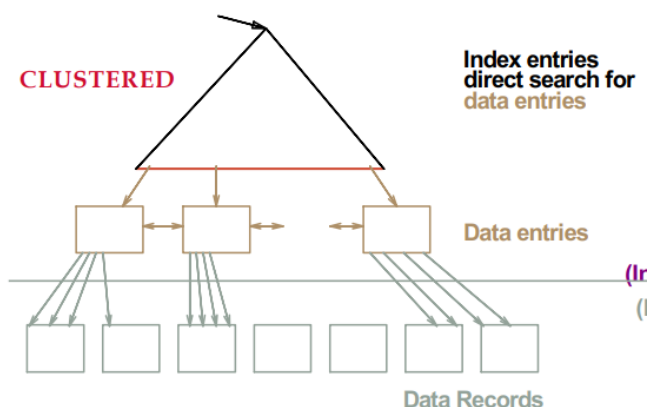


Figure 19: Esempio di indice clusterizzato: i puntatori fluiscono parallelamente verso i blocchi fisici.

- **Indice Non Clusterizzato (*Unclustered Index*):** Non vi è alcun vincolo o correlazione tra l'ordine logico delle chiavi nell'indice e l'organizzazione fisica dei record nel file dati (che tipicamente risiede in un *Heap File* o è clusterizzato su un'altra chiave).
 - **Costo di Scansione:** Una scansione sequenziale delle foglie dell'indice comporta l'inseguimento di puntatori che saltano casualmente da un blocco fisico all'altro sul disco. Visivamente, i puntatori "si incrociano" generando caos.
 - **Collasso delle Prestazioni (Page Faults):** Nel caso peggiore, l'accesso sequenziale a N chiavi nell'indice può provocare il caricamento di N pagine disco differenti. Questo rende gli indici non clusterizzati molto inefficienti per ampi intervalli di ricerca.
 - **Molteplicità:** Essendo slegati dall'ordine fisico, è possibile definire **molteplici indici non clusterizzati** sulla stessa tabella.

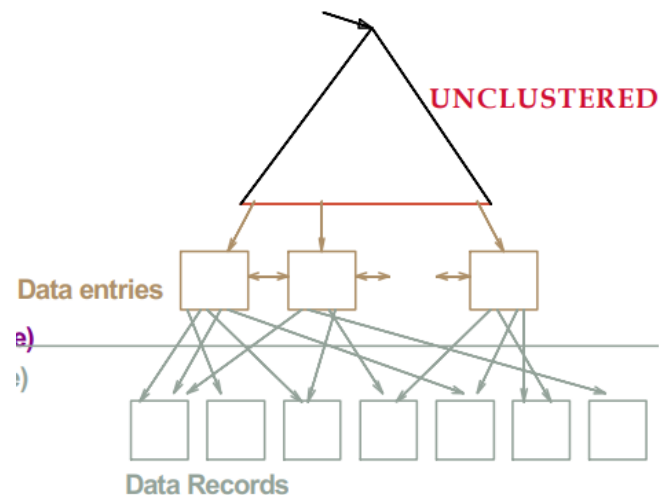


Figure 20: Esempio di indice non clusterizzato: incrocio dei puntatori e conseguente elevato costo di accesso casuale (Random I/O).

6.5.1 Indice Clusterizzato (Clustered Index)

In un indice clusterizzato, l'ordine fisico dei record all'interno del file dati corrisponde all'ordine logico della chiave di ricerca nell'indice.

A livello fisico, l'ordinamento rigoroso dei record all'interno della singola pagina dati non è strettamente necessario; ciò che è fondamentale è l'ordinamento sequenziale tra le pagine. Una volta caricata la pagina in RAM, infatti, il costo per esplorarla e trovare il record internamente è trascurabile.

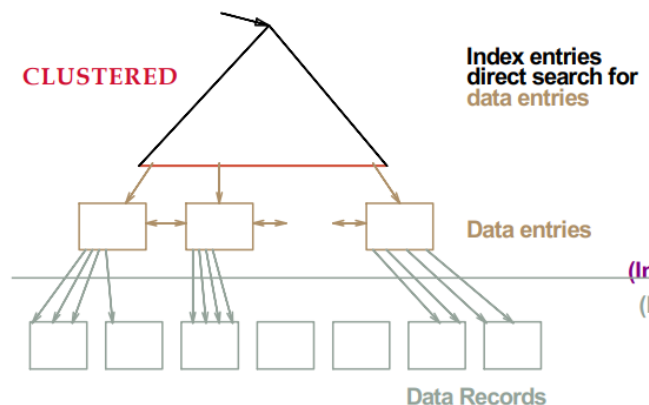


Figure 21: Esempio di indice clusterizzato: i puntatori fluiscono parallelamente verso i blocchi fisici.

Vantaggi

- **Efficienza per Intervalli (Range Query):** Garantisce prestazioni eccellenti per le ricerche di intervalli. Scandendo le chiavi nelle foglie, i puntatori accedono a blocchi fisicamente contigui, riducendo al minimo i salti sul disco.
- **Compressione strutturale:** Facendo leva sull'ordinamento fisico, è possibile mantenere a livello foglia solo gli *estremi degli intervalli* (costruendo un indice sparso). Questo riduce il numero di chiavi necessarie nelle foglie e, di conseguenza, il numero di pagine dell'indice. Il risultato è un albero più piccolo, meno profondo e con tempi di attraversamento più rapidi.

Svantaggi

- **Unicità:** Poiché i dati fisici non possono essere ordinati in più modi contemporaneamente, può esistere un solo indice clusterizzato per tabella.

- **Accesso ai Data File:** Se l'obiettivo è restituire solo i valori della chiave di ricerca, in un indice sparso è comunque necessario accedere alle pagine dei data file, poiché non tutte le chiavi sono esplicitamente rappresentate nelle foglie dell'indice.

6.5.2 Indice Non Clusterizzato (*Unclustered Index*)

Non vi è alcuna correlazione tra l'ordine logico delle chiavi nell'indice e l'organizzazione fisica dei record nel file dati. A causa di questa disconnessione, l'indice **deve** essere denso: tutte le chiavi di ricerca devono essere obbligatoriamente presenti e ordinate a livello delle foglie.

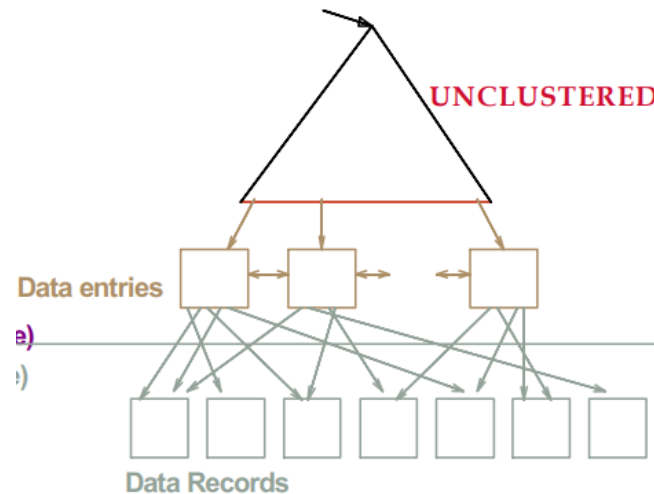


Figure 22: Esempio di indice non clusterizzato: incrocio dei puntatori e conseguente elevato costo di accesso casuale (Random I/O).

Vantaggi

- **Index-Only Query altamente efficienti:** Se la query richiede di restituire solo le chiavi di ordinamento, il DBMS può fermarsi al livello delle foglie dell'indice. Essendo le foglie molto più piccole rispetto ai data record completi, il numero di accessi in lettura è drasticamente inferiore.
- **Molteplicità:** Non essendo vincolati all'ordinamento fisico, è possibile definire molteplici indici non clusterizzati sulla stessa tabella.

Svantaggi

- **Collasso in scansione completa:** Se la query non è *index-only* e richiede l'intero record, l'operazione diventa estremamente costosa. Il DBMS deve posizionarsi sulla foglia e seguire i puntatori, generando accessi casuali al disco. Nel caso peggiore, il numero di page fault è pari al numero di record che soddisfano la query.

6.5.3 Casi di Studio: Domande d'Esame

Domanda 1: Se si sa che la query più frequente sarà la ricerca del valore massimo dell'attributo su cui è costruito l'indice, è preferibile un indice clusterizzato o non clusterizzato?

Risposta: Sono **equivalenti** in termini di prestazioni. Nell'unclustered, si sa che il valore massimo si trova nell'ultima voce dell'ultima foglia. Nel clustered, si segue la medesima struttura fino ad arrivare all'ultima pagina dati (che va portata in RAM per leggerne il massimo). Il numero di letture strutturali (l'altezza dell'albero) è sostanzialmente lo stesso.

Domanda 2: E se la query fosse il calcolo del valore medio (AVG) sull'attributo indicizzato?

Risposta: È nettamente preferibile l'**indice non clusterizzato**. Per calcolare la media è sufficiente leggere i soli valori della chiave. Con l'unclustered si esegue un'efficiente *index-only query* leggendo esclusivamente le foglie dell'indice. Con il clustered (essendo spesso sparso), si è costretti a scandire

tutte le N pagine dell'intero data file per estrarre i valori, comportando un costo di I/O molto più elevato.

Domanda 3: Se si dovesse stampare ordinatamente i valori di un attributo che NON è la chiave di ricerca su cui è costruito l'indice, è meglio un clustered o un unclustered?

Risposta: Meglio il **clusterizzato**. Dovendo in ogni caso scandire l'intera relazione (poiché l'indice esistente non aiuta a filtrare o ordinare quell'attributo specifico), il file clusterizzato garantisce l'accesso sequenziale ai blocchi fisici. Usare un indice unclustered per accedere sequenzialmente a tutti i dati genererebbe un numero inaccettabile di accessi casuali.

6.5.4 Gestione dei Duplicati e Struttura delle Voci (Data Entries)

La presenza di chiavi di ricerca duplicate (più record con la stessa chiave) influisce su come le Data Entries (k^*) vengono formattate nelle foglie dell'indice:

- **Alternativa 1** ($k^* = \langle \text{Intero Record} \rangle$): Non si pone il problema dei puntatori duplicati, poiché le foglie ospitano direttamente i record. Eventuali record con la stessa chiave di ricerca differiranno inevitabilmente per la chiave primaria. Questa alternativa ha senso unicamente per gli indici clusterizzati.

- **Alternativa 2** ($k^* = \langle k, RID \rangle$): Se esistono 5 record con la stessa chiave, si avranno 5 chiavi ripetute e 5 puntatori distinti all'interno dell'indice.

Vantaggio: Garantisce record a *lunghezza fissa*, rendendo la gestione dello spazio nella pagina estremamente semplice e veloce.

Svantaggio: Maggiore occupazione di spazio a causa della ridondanza della chiave.

- **Alternativa 3** ($k^* = \langle k, \text{ListaRID} \rangle$): La chiave compare una singola volta, abbinata alla lista di tutti i puntatori che la soddisfano.

Vantaggio: Ottimizzazione e forte risparmio di spazio.

Svantaggio: Introduce record a *lunghezza variabile*. La gestione strutturale (allocazione, aggiornamenti, frammentazione) risulta molto più complessa da implementare nel DBMS.

6.6 Indici con chiavi di ricerca composte

È possibile definire un indice utilizzando un numero arbitrario di attributi, creando così una **chiave di ricerca composta** (o *composite key*).

Nei database, l'ordinamento lessicografico di una chiave composta è rigorosamente sequenziale e posizionale. Di conseguenza, indicizzare per (Cognome, Nome) è radicalmente diverso dall'indicizzare per (Nome, Cognome): nel primo caso, il DBMS ordina l'intero set di dati primariamente per Cognome e, solo in caso di omonimia (duplicati), utilizza il Nome come criterio di spartizione per l'ordinamento secondario.

Quando utilizzare chiavi composte L'indicizzazione composita diventa cruciale quando l'analisi del *query workload* (il carico di lavoro previsto sul database) rivela la presenza frequente di query che filtrano i dati su più attributi contemporaneamente nella clausola WHERE.

Un esempio tipico è: `SELECT * FROM Studenti WHERE Cognome = 'Rossi' AND AnnoNascita > 2002 AND AnnoNascita < 2006;`

Attenzione al Trade-off: Ogni indice aggiuntivo o complesso impone un onere (*overhead*) sulle operazioni di scrittura (INSERT, UPDATE, DELETE), poiché il DBMS deve aggiornare la struttura dell'albero per ogni modifica. Pertanto, la creazione di indici composti è giustificata solo se i benefici in fase di lettura superano i costi di manutenzione.

6.6.1 Strategie di esecuzione per query multi-attributo

Di fronte alla query d'esempio, il DBMS può adottare diverse strategie (o *piani di esecuzione*) a seconda degli indici effettivamente disponibili:

- **Scenario 1: Indice singolo solo su *Cognome***
 - L'albero naviga rapidamente fino a trovare il primo record con cognome 'Rossi'.
 - Segue sequenzialmente i puntatori per recuperare *tutti* i record dei 'Rossi' dai file dati.
 - Solo dopo averli caricati in RAM, verifica l'anno di nascita, scartando quelli fuori dal range. Il costo dipenderà da quanti 'Rossi' esistono in totale (selettività del cognome).
- **Scenario 2: Indice singolo solo su *AnnoNascita***
 - Effettua una ricerca per intervallo (Range Query) estraendo i record degli anni 2003, 2004 e 2005.
 - Accede ai file dati per tutti questi studenti e verifica se il cognome è 'Rossi'.
 - Questa opzione è tipicamente inefficiente: essendo i range di età poco selettivi, il sistema finirà per caricare un'enorme mole di dati inutili solo per trovare pochi 'Rossi'.
- **Scenario 3: Due indici singoli separati (*Index Intersection*)**
 - Sfrutta entrambi gli indici in modo indipendente: estrae dall'indice dei cognomi la lista dei RID (puntatori) di tutti i 'Rossi' e, dall'indice delle date, la lista dei RID di chi è nato nel range.
 - Il DBMS calcola in memoria l'**intersezione** di questi due insiemi di puntatori.
 - Infine, accede ai file dati *esclusivamente* per recuperare i record che appartengono all'intersezione, azzerando le letture inutili.
- **Scenario 4: Indice Composto (La soluzione ottimale)** Se la query è molto frequente, un indice composto è la scelta vincente. Tuttavia, **l'ordine degli attributi nell'indice è fondamentale**:
 - **Opzione A - Ordinamento** (*AnnoNascita*, *Cognome*): L'attributo primario è soggetto a un range. L'indice individua il primo record > 2002 . Da quel punto in poi, il DBMS è costretto a **scandire sequenzialmente tutto il blocco** di nati nel 2003, 2004 e 2005, saltando continuamente i record con cognomi diversi da 'Rossi'. Molto costoso.
 - **Opzione B - Ordinamento** (*Cognome*, *AnnoNascita*): L'attributo primario è soggetto a un'uguaglianza. L'indice salta direttamente al blocco esatto di tutti i 'Rossi' e, all'interno di questo sottoinsieme già isolato, si posiziona sull'estremo inferiore del range ('Rossi', 2003). Da lì scorre sequenzialmente leggendo **solo record validi**, fino a quando l'anno non supera il 2005.

Regola d'Oro del Design degli Indici Dall'ultimo scenario emerge una regola fondamentale per la progettazione degli indici composti: **Equality First, Range Second** (Prima le uguaglianze, poi gli intervalli).

Se un *query workload* prevede l'uso combinato di predicati di uguaglianza (es. =) e predicati di intervallo (es. <, >, BETWEEN), l'indice composto deve **sempre** avere l'attributo di uguaglianza come chiave primaria e l'attributo di intervallo come chiave secondaria. In questo modo si massimizza la precisione della discesa nell'albero e si minimizzano i record letti scartati.

6.7 Indici Hash-Based

Gli indici basati su tabelle di **Hash** sono strutture progettate per massimizzare l'efficienza delle **ricerche per uguaglianza esatta** (es. `SELECT * FROM Impiegati WHERE Matricola = 12345`).

A differenza degli alberi B+ Tree, le funzioni di hash mirano per loro stessa natura a distribuire i record in modo uniforme e pseudo-casuale su tutto lo spazio degli indirizzi disponibili. Questo comporta una **totale assenza di correlazione** tra l'ordine logico delle chiavi di ricerca e l'ordine fisico dei record su disco.

Caratteristiche Principali

- **Accesso in tempo costante:** Nel caso ideale (assenza di collisioni), per recuperare la pagina contenente il record desiderato è sufficiente **un singolo accesso al disco** (I/O), applicando semplicemente la funzione di hash $h(k)$ alla chiave di ricerca k .
- **Inadeguatezza per Range Query:** Proprio a causa della distribuzione disordinata che distrugge qualsiasi ordinamento logico, gli indici hash **non possono** essere utilizzati per risolvere in modo efficiente query per intervallo (es. WHERE Età > 18). Per queste operazioni, il file dovrebbe essere scandito per intero, rendendo il B+ Tree la scelta obbligata.

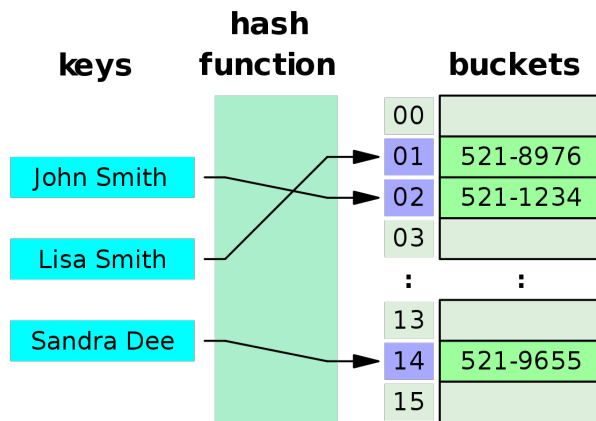


Figure 23: Esempio di hashing

Livelli di Applicazione dell'Hashing L'indicizzazione hash può essere implementata in due modalità architetturali:

1. **Hashing diretto sul Data File:** La funzione di hash calcola l'indirizzo fisico in cui posizionare direttamente il record dati completo (corrispondente all'*Alternativa 1*). Poiché i record fisici possono essere disposti in un solo modo su disco, è possibile avere **un solo** indice di questo tipo per tabella.
2. **Hashing sulle Data Entries:** La funzione di hash organizza esclusivamente le voci di indice (es. *Alternativa 2* o *Alternativa 3*), le quali fungono da directory contenente le chiavi e i puntatori (RID) alla posizione reale dei record nei data file (ad esempio, un *Heap File*). Essendo svincolato dall'ordinamento fisico dei dati veri e propri, questo approccio permette di costruire **molteplici indici hash** secondari sulla stessa tabella.

6.7.1 Hashing Statico

L'hashing statico è la forma architetturale più semplice. Si basa sull'allocazione di un numero **prefissato e immutabile di bucket** (pagine primarie) fin dal momento della creazione del file.

Inizializzazione: Poiché non è possibile conoscere a priori l'esatta distribuzione e il volume dei futuri inserimenti, il DBMS (o il DBA) deve stimare il numero totale di record. Tipicamente si dimensiona la struttura per avere un fattore di riempimento (*Fill Factor*) iniziale ragionevole (es. 80%), lasciando dello spazio vuoto fisiologico per assorbire futuri inserimenti organici.

Il Problema del Degrado: Collisioni e Overflow Il principale punto debole dell'hashing statico è la sua rigidità strutturale di fronte alla crescita dei dati. Se il database subisce numerosi inserimenti non previsti o se la funzione di hash distribuisce i dati in modo non omogeneo (fenomeno dello *skew*), i bucket primari si riempiono rapidamente, generando **collisioni**.

Quando un bucket si satura e non c'è più spazio per il record k^* , il DBMS è costretto ad allocare **pagine di overflow** concatenate dinamicamente al bucket originale. L'impatto sulle prestazioni è severo:

- L'efficienza ideale di un singolo accesso $\mathcal{O}(1)$ **degrada rapidamente** in una **lettura sequenziale** (scansione) di tutte le pagine che compongono la catena di overflow.
- In scenari pessimi (catene molto lunghe), il costo per trovare un record supera quello di un attraversamento di un albero B+ Tree.
- Per ripristinare le prestazioni originali, si rende necessaria una costosa **riorganizzazione periodica** dell'intero file (re-hashing), che richiede di fermare le operazioni, definire una nuova funzione di hash e ridistribuire tutti i dati su un set di bucket più ampio.

6.7.2 Hashing Estendibile (Extendible Hashing)

L'hashing estendibile è una tecnica dinamica progettata per gestire la crescita dei dati senza ricorrere a lunghe catene di overflow. A differenza dell'hashing statico, introduce un livello intermedio di indirizzamento: una **directory** di puntatori ai bucket dati.

Si parte da un numero di bucket dimensionato in modo da contenere tutte le chiavi iniziali con un fattore di riempimento ragionevole.

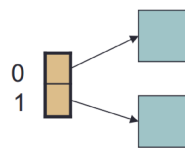


Figure 24: Esempio di hashing estendibile: partenza con un numero di bucket prefissato e directory associata

Meccanismo di funzionamento La struttura fa affidamento su due concetti fondamentali per gestire la dinamicità:

- **Profondità Globale (GD - Global Depth):** Indica il numero di bit generati dalla funzione di hash attualmente utilizzati dalla directory per instradare le chiavi. Una directory con profondità globale GD possiede esattamente 2^{GD} voci.
- **Profondità Locale (LD - Local Depth):** È una proprietà associata al singolo bucket e indica il numero effettivo di bit utilizzati per determinare quali chiavi vi risiedono. Vale sempre $LD \leq GD$.

L'approccio alla gestione degli spazi è strettamente **locale**. Quando un bucket si riempie e riceve un nuovo inserimento, il sistema non crea una pagina di overflow, ma confronta la profondità locale del bucket con quella globale:

1. **Se $LD < GD$:** Non serve ingrandire la directory. Il bucket saturo viene "splittato" (diviso in due), la sua profondità locale LD viene incrementata di 1, e le chiavi in esso contenute vengono ridistribuite tra i due nuovi bucket utilizzando il bit aggiuntivo per discriminarle. I puntatori nella directory vengono semplicemente reindirizzati.
2. **Se $LD = GD$:** La directory ha esaurito i bit per mappare nuovi split e deve essere **raddoppiata** (la profondità globale GD viene incrementata di 1, passando da 2^{GD} a 2^{GD+1} voci). Successivamente, il bucket saturo viene splittato come descritto sopra.

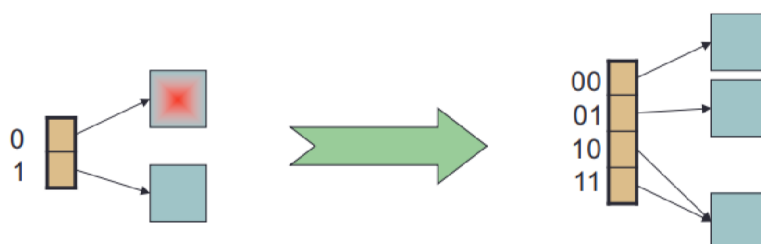


Figure 25: Esempio di hashing estendibile: raddoppio della directory e redistribuzione del bucket saturo

È cruciale notare che quando la directory raddoppia, il numero di bucket effettivi *non* raddoppia con essa. Le nuove voci della directory punteranno inizialmente in modo ridondante ai bucket preesistenti non splittati. Viene creata una nuova pagina fisica solo per ospitare la metà dei dati del bucket originariamente saturo.

Vantaggi e Svantaggi Questa tecnica mantiene intatta l'efficienza dell'accesso in tempo costante (tipicamente due operazioni di I/O: una per la directory, una per il bucket) anche in scenari di forte crescita. Tuttavia, richiede un attento monitoraggio: se la funzione di hash distribuisce i dati in modo non uniforme (*skew*), i continui split locali provocheranno raddoppi a catena della directory, portando a un forte spreco di memoria a causa dei numerosi bucket allocati ma parzialmente vuoti.

6.7.3 Hashing Lineare (Linear Hashing)

L'hashing lineare è un'architettura alternativa che permette alla struttura di crescere gradualmente (un bucket alla volta) **senza la necessità di una directory centrale**. A differenza dell'hashing estendibile, il criterio che guida la crescita della struttura si basa su una valutazione a livello **globale**.

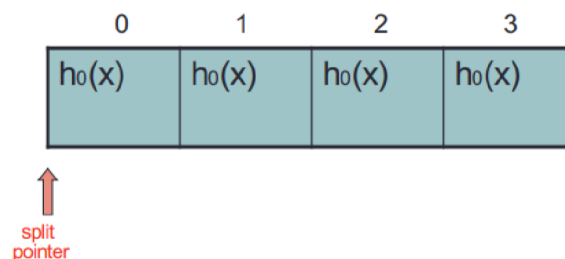


Figure 26: Esempio di hashing lineare: stato iniziale prima delle operazioni di split

Si parte con un numero N di pagine iniziali capaci di garantire un'occupazione accettabile. Il sistema monitora il fattore di riempimento globale (ad esempio, il rapporto totale tra chiavi inserite e capacità complessiva dei bucket).

Meccanismo di funzionamento e Splitting

- **Funzioni di Hash:** Si parte utilizzando una funzione di hash primaria $h_0(k) = k \bmod N$. Per gestire gli split, si prepara una funzione a risoluzione raddoppiata $h_1(k) = k \bmod 2N$.
- **Puntatore di Split (p):** Lo splitting delle pagine procede in modo rigorosamente sequenziale (*Round Robin*), guidato da un puntatore p inizializzato a 0.
- **Gestione dell'Overflow:** Se un inserimento satura un bucket temporaneamente, l'hashing lineare tollera la creazione di una limitata catena di overflow. Tuttavia, se l'inserimento fa superare la soglia di occupazione globale prefissata, si innesca la procedura di split.
- **Lo Split:** Indipendentemente da quale bucket sia andato in overflow, **il sistema splitta sempre e solo il bucket indicato dal puntatore p** . Viene creata una nuova pagina all'indirizzo $p+N$, e i record originariamente nel bucket p vengono ridistribuiti tra la pagina p e la pagina $p+N$ ricalcolandone la posizione tramite la nuova funzione h_1 . Finito lo split, il puntatore avanza di un passo ($p \leftarrow p+1$).

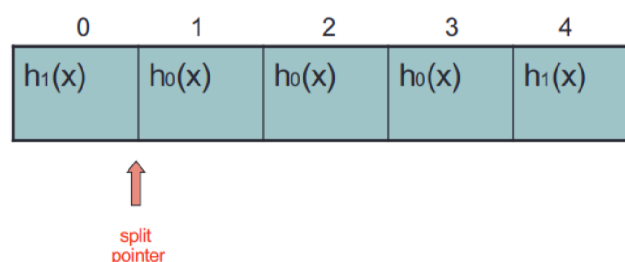


Figure 27: Esempio di hashing lineare: procedura di splitting guidata dal puntatore p

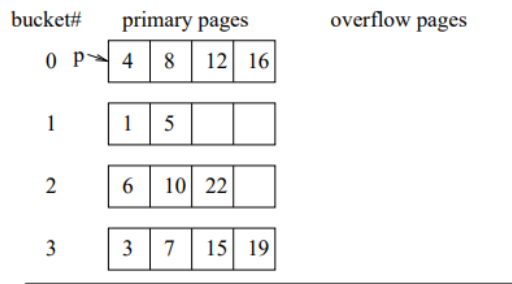


Figure 28: Pre splitting: utilizzo transitorio di un bucket di overflow

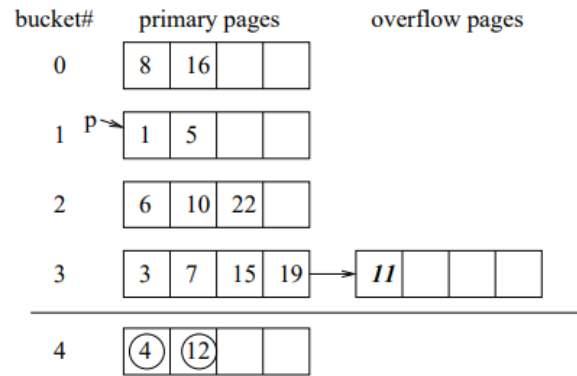


Figure 29: Post splitting: redistribuzione e avanzamento del puntatore

Algoritmo di Ricerca Poiché in un dato momento l'indice contiene sia bucket non splittati (che usano h_0) sia bucket già splittati (che usano h_1), la ricerca di una chiave k segue una logica condizionale:

1. Si calcola l'indirizzo base: $indirizzo = h_0(k)$.
2. Se $indirizzo \geq p$, il bucket bersaglio si trova alla destra del puntatore e non è ancora stato splittato; la chiave si trova in quel bucket.
3. Se $indirizzo < p$, il bucket originale è già stato sottoposto a split. Per individuare in quale delle due metà si trovi ora la chiave, si ricalcola l'indirizzo con la funzione avanzata: $indirizzo = h_1(k)$.

Transizione di Livello Quando il puntatore p completa il suo giro e raggiunge il valore N (es. se si partiva da 4 bucket ed è arrivato a 4, avendone ora 8 totali), significa che l'intera struttura è stata raddoppiata. A questo punto si chiude un "livello" o fase: il puntatore di split viene resettato a $p = 0$, N diventa $2N$, h_1 assume il ruolo della nuova funzione di base h_0 , e si definisce una nuova $h_1(k) = k \bmod 4N$ per gestire i futuri split del nuovo ciclo.

7 Analisi dei Costi e Prestazioni

L'obiettivo di questa sezione è sviluppare un modello analitico per stimare il costo (in termini di operazioni di Input/Output su disco) delle operazioni fondamentali sui database, a seconda della struttura dati fisica adottata per memorizzare i record.

Le operazioni che analizzeremo sono:

- **Scansione (Scan):** Recupero di tutti i record presenti nella tabella.
- **Ricerca per Uguaglianza (Equality Query):** Ricerca di un record specifico tramite una chiave esatta.
- **Ricerca per Intervallo (Range Query):** Recupero di tutti i record le cui chiavi rientrano in un determinato range (es. Età > 18).
- **Inserimento (Insert):** Aggiunta di un nuovo record.
- **Cancellazione (Delete):** Rimozione di un record esistente.

7.1 Parametri del Modello di Costo

Per definire matematicamente i costi, introduciamo le seguenti variabili fondamentali:

- **B:** Numero totale di pagine (*blocks*) che compongono il data file originale (senza considerare lo spazio extra introdotto dagli indici).
- **R:** Numero medio di record ospitati in una singola pagina dati. Il prodotto $B \times R$ fornisce il numero totale di record nella tabella.
- **D:** Tempo medio necessario per leggere o scrivere una singola pagina su disco (costo di una operazione di I/O).

Nota bene: In questo modello semplificato, **ignoriamo il costo computazionale della CPU** (es. il calcolo della funzione di hash, l'ordinamento in RAM o la ricerca binaria in memoria centrale). Il collo di bottiglia nei DBMS è quasi sempre l'accesso meccanico al disco, pertanto misuriamo le prestazioni esclusivamente contando i trasferimenti di pagine (*Page Faults*).

7.2 Assunzioni sulle Strutture Dati

Per rendere i calcoli confrontabili, dobbiamo definire le caratteristiche architetturali e i fattori di riempimento (*Fill Factor / Occupancy*) di ciascuna struttura:

- **Heap Files:** Si assume che, per ricerche su chiavi univoche (chiavi primarie), la scansione possa interrompersi non appena viene trovato il record desiderato.
- **Sorted Files:** Si assume che il file debba essere mantenuto rigorosamente compatto e ordinato fisicamente; di conseguenza, ogni cancellazione comporta una compattazione per tappare il "buco" lasciato.
- **Indici (Voci di Indice):** Utilizziamo l'**Alternativa 2 o 3** per memorizzare le voci di indice (*Data Entries, k^**). Assumiamo che la dimensione di una singola voce di indice (k^*) sia pari al **10% (0.10)** della dimensione dell'intero record dati.
- **Occupazione Indice Hash (Unclustered):**

Spiegazione Frazione di Costo: Si assume l'assenza di bucket di overflow e un fattore di riempimento target delle pagine indice pari all'**80%**. Poiché ogni pagina indice è riempita solo all'80%, significa che il 20% dello spazio è sprecato per assorbire futuri inserimenti. Per calcolare quante pagine fisiche occuperà l'indice rispetto al minimo teorico, si calcola il reciproco del fattore di riempimento: $\frac{1}{0.8} = 1.25$. Inoltre, sapendo

che una singola voce di indice (k^*) pesa il 10% (0.10) del record originale, il numero di pagine destinate alle voci di indice rispetto ai blocchi di dati originali (B) sarà pari a:

$$B_{\text{hash}} = B \times 0.10 \times 1.25 = 0.125B$$

- **Occupazione Indice ad Albero (B+ Tree):**

Per gli alberi B+ (sia clustered che unclustered), si assume un fattore di riempimento tipico del **67%** ($2/3$ della capacità per ogni nodo). Di conseguenza, il moltiplicatore per compensare lo spazio vuoto è $\frac{1}{0.67} \approx 1.5$.

Per un albero Clustered (Alternativa 1, le foglie sono i record dati): il data file stesso funge da indice, occupando il 50% di spazio in più rispetto a un Heap File compatto. Il numero di foglie sarà quindi **1.5 B**. Questo fattore mi consente nel momento in cui devo inserire un nuovo record di avere un margine di spazio sufficiente per evitare di dover spostare tutti i record successivi (come accade nei Sorted Files) e di non dover modificare la struttura dell'albero soprastante.

Per un albero Unclustered (Alternativa 2): le foglie contengono solo le voci di indice (k^*). Tenendo conto del peso del 10% di k^* e del riempimento al 67%, il numero di foglie dell'indice sarà:

$$B_{\text{tree.unclustered}} = B \times 0.10 \times 1.5 = 0.15B$$

Assunzioni sulle Strategie di Scansione

- **Scansione ad Albero:** La scansione sequenziale di un indice ad albero sfrutta sempre il *Sequence Set* (la lista concatenata a livello delle foglie).
- **Scansione Hash:** Gli indici Hash, non avendo ordinamento, vengono del tutto ignorati per le query di *Range*.

7.3 Tabella Riassuntiva dei Costi (in operazioni di I/O)

La seguente tabella riassume i costi (moltiplicati per il tempo D) previsti per le diverse organizzazioni fisiche.

	Scan	Equality Query	Range Query	Insert	Delete
Heap File	BD	$0.5BD$	BD	$2D$	Search + D
Sorted File	BD	$D \log_2 B$	$D(\log_2 B + \text{\#pgs match})$	Search + BD	Search + BD
Clustered Tree	$1.5BD$	$D \log_F(1.5B)$	$D(\log_F(1.5B) + \text{\#pgs match})$	Search + D	Search + D
Unclustered Tree	$BD(R + 0.15)$	$D(1 + \log_F(0.15B))$	$D(\log_F(0.15B) + \text{\#pgs match})$	Search + $2D$	Search + $2D$
Unclustered Hash	$BD(R + 0.125)$	$2D$	BD	Search + $2D$	Search + $2D$

7.4 Analisi Dettagliata per Struttura

7.4.1 Heap File

Una collezione disordinata di record. I nuovi inserimenti avvengono in coda all'ultima pagina disponibile.

- **Scansione (BD):** Occorre leggere tutti i B blocchi dal disco, una sola volta.
- **Ricerca per Uguaglianza ($0.5BD$):** Per trovare una chiave primaria specifica, bisogna effettuare una ricerca lineare sequenziale dall'inizio. Nella migliore delle ipotesi il record è nella prima pagina (1 I/O), nella peggiore è nell'ultima (B I/O). In media, si leggerà la metà del file.
- **Ricerca per Range (BD):** Non essendoci ordinamento, non si può sapere in anticipo dove si trovino i valori cercati. L'intero file deve essere scandito.
- **Inserimento ($2D$):** Il costo è composto dalla lettura della pagina finale per verificare la disponibilità di spazio (1 I/O in lettura) e dalla successiva scrittura della pagina aggiornata su disco (1 I/O in scrittura).

- **Cancellazione** ($Search + D$): Prima occorre individuare il record, pagando il costo della ricerca (mediamente $0.5BD$). Una volta trovata e caricata in RAM la pagina interessata, la si modifica e la si riscrive su disco ($+1D$ in scrittura).

7.4.2 Sorted File

Un file in cui i data record sono mantenuti fisicamente e rigorosamente ordinati in base alla chiave di ricerca.

- **Scansione** (BD): L'ordinamento fisico garantisce che una semplice lettura sequenziale di tutti i blocchi restituisca i record già ordinati. Il costo è proporzionale al numero di blocchi.
- **Ricerca per Uguaglianza** ($D \log_2 B$): Essendo il file ordinato, il DBMS non fa una scansione lineare ma esegue una **ricerca binaria** sulle pagine. Ad ogni passo, si legge il blocco mediano, riducendo dimezzando lo spazio di ricerca. Il numero di accessi al disco è logaritmico in base 2.
- **Ricerca per Range** ($D(\log_2 B + \# \text{pagine con match})$): Altamente efficiente. Il DBMS utilizza la ricerca binaria ($D \log_2 B$) per trovare l'estremo inferiore dell'intervallo. Una volta posizionato, legge sequenzialmente solo il numero di pagine contigue strettamente necessarie per coprire l'intero range.
- **Inserimento e Cancellazione** ($Search + BD$): Questo è il grave difetto dei Sorted Files compatti. Dopo aver pagato il costo di ricerca ($Search$), l'inserimento o la cancellazione di un record a metà del file costringe il sistema a traslare fisicamente (leggere e riscrivere) *tutte* le pagine successive per far posto al nuovo record o per colmare il vuoto, causando un costo pesantissimo pari all'intera dimensione del file (BD).

7.4.3 Indice ad Albero Clustered (Clustered Tree)

In questa struttura, il data file stesso è organizzato come un albero B+. I record dati si trovano direttamente nelle foglie dell'albero (Alternativa 1), mantenendo un ordinamento fisico basato sulla chiave di ricerca.

- **Scansione** ($1.5BD$): Il costo è superiore a quello di un normale Heap File a causa del fattore di riempimento tipico degli alberi B+ (assunto al 67%). Il file occupa quindi circa il 50% di spazio in più, richiedendo la lettura di $1.5B$ pagine anziché B .
- **Ricerca per Uguaglianza** ($D \log_F(1.5B)$): Sfrutta la struttura gerarchica dell'albero. Il costo corrisponde all'attraversamento dei nodi dalla radice fino alla foglia desiderata, con un andamento logaritmico basato sul fattore di ramificazione F dell'albero.
- **Ricerca per Intervallo** ($D(\log_F(1.5B) + \# \text{pagine con match})$): Estremamente efficiente. Una volta individuato l'estremo inferiore del range tramite la ricerca logaritmica, si sfrutta la contiguità fisica dei record (Sequence Set) per leggere sequenzialmente solo le pagine necessarie a coprire l'intervallo.
- **Inserimento e Cancellazione** ($Search + D$): Comporta un costo iniziale di ricerca ($Search$) per individuare la pagina foglia corretta. Una volta trovata, la si modifica e la si riscrive su disco ($+D$). A differenza dei Sorted Files compatti, il margine di spazio libero garantito dal fattore di riempimento evita la necessità di traslare fisicamente i record successivi.

7.4.4 Indice ad Albero Non Clustered (Unclustered Tree)

Le foglie dell'albero contengono solo le voci di indice (Alternativa 2 o 3) modellate come coppie (*chiave, puntatore al record*). I record dati veri e propri si trovano in un Heap File separato e disordinato.

- **Scansione** ($BD(R + 0.15)$): Il costo è **proibitivo**. L'operazione richiede la lettura di tutte le foglie dell'indice ($0.15B$ pagine) più il recupero di ogni singolo record dati. Poiché l'Heap File non è ordinato rispetto all'indice, ogni voce di indice punta potenzialmente a una pagina dati diversa, causando un I/O per ogni record letto ($B \times R$).

Nota: Questa organizzazione è altamente sconsigliata per le scansioni complete. Si rivela invece eccellente per query altamente selettive che restituiscono una frazione piccolissima dei record totali.

- **Ricerca per Uguaglianza** ($D(1 + \log_F(0.15B))$): Si scende lungo l'albero per trovare la voce di indice nella foglia (costo logaritmico). Se la query richiede dati non presenti nella chiave (*non index-only*), occorre un ulteriore accesso al disco ($+D$) per recuperare la pagina dell'Heap File contenente il record effettivo.
- **Ricerca per Intervallo** ($D(\log_F(0.15B) + \text{\#pagine con match})$): Si trova l'estremo inferiore del range scendendo nell'albero e si scorre il Sequence Set delle foglie ($0.15B$ pagine totali). Il termine *\#pagine con match* rappresenta gli accessi diretti all'Heap File per recuperare i record: questo numero può essere altissimo se il range è ampio, rendendo la query inefficiente.
- **Inserimento e Cancellazione** ($\text{Search} + 2D$): Richiede di trovare la posizione corretta (*Search*). L'aggiornamento coinvolge due scritture distinte: una per aggiornare la pagina dell'Heap File contenente il record e una per aggiornare la pagina foglia dell'indice, per un totale di due operazioni di I/O in scrittura ($2D$).

7.4.5 Indice Hash Non Clustered (Unclustered Hash)

L'indice utilizza una funzione di hash per mappare le chiavi in determinati *bucket* (pagine disco). Anche in questo caso, i record dati veri e propri risiedono in un Heap File separato.

- **Scansione** ($BD(R + 0.125)$): Il ragionamento e i costi sono quasi identici a quelli dell'Unclustered Tree, ma con una spesa marginalmente inferiore dovuta alla struttura più compatta dell'indice Hash (occupa $0.125B$ pagine invece di $0.15B$). La scansione non restituisce i record ordinati, ma raggruppa naturalmente quelli che collidono nello stesso bucket (utile per operazioni di *GROUP BY* sulla chiave di hash).
- **Ricerca per Uguaglianza** ($2D$): È il punto di forza assoluto di questa struttura. Trascurando il costo CPU per il calcolo dell'hash, è sufficiente un I/O per leggere la pagina del bucket e un secondo I/O per recuperare la pagina dati dall'Heap File tramite il puntatore (assumendo assenza di bucket di overflow).
- **Ricerca per Intervallo** (BD): **Inapplicabile**. La funzione di hash distrugge qualsiasi ordinamento logico delle chiavi. Il DBMS ignora l'indice e ripiega su una scansione sequenziale completa dell'Heap File sottostante.
- **Inserimento e Cancellazione** ($\text{Search} + 2D$): Simile all'Unclustered Tree. Si individua il bucket tramite l'hash (*Search*), si aggiunge/rimuove il record nell'Heap File (una scrittura) e si aggiorna la voce di indice nel bucket (seconda scrittura), portando il costo a $\text{Search} + 2D$.

Conclusioni sulle Organizzazioni Fisiche Queste valutazioni matematiche, seppur basate su semplificazioni e stime grossolane, mettono in luce i delicati compromessi (*trade-off*) della progettazione fisica di un database. Non esiste una struttura universalmente perfetta: la scelta tra Heap, Sorted, Tree o Hash Index deve essere sempre guidata dall'analisi del *workload* previsto, ovvero dalla frequenza e dalla tipologia delle query (Scan, Equality, Range) e dalle operazioni di aggiornamento (Insert, Delete) che il sistema dovrà sostenere.

7.5 Come scegliere l'indice appropriato

Definizione di Workload: Il *workload* (carico di lavoro) rappresenta l'insieme tipico di operazioni (query di lettura e aggiornamenti di scrittura) che il database deve sostenere durante il suo normale utilizzo, inclusa la frequenza con cui ciascuna operazione viene eseguita. Ottimizzare un database significa progettare le strutture fisiche in funzione del suo specifico workload.

Per determinare la strategia di indicizzazione ideale, occorre analizzare preventivamente ogni componente del workload.

Analisi delle Query (Letture) Per ogni interrogazione frequente o critica, bisogna chiedersi:

- **Relazioni e Attributi:** A quali tabelle si accede e quali colonne vengono recuperate?
- **Condizioni di Ricerca:** Quali attributi sono coinvolti nelle clausole `WHERE` (selezioni) o `JOIN`?
- **Selettività:** Quanto sono restrittive le condizioni di ricerca?
 - Se la condizione è **molto selettiva** (restituisce pochi record, es. `CodiceFiscale = X`), un indice è fondamentale.
 - Se la condizione è **poco selettiva** (restituisce quasi tutta la tabella, es. `Sesso = M`), l'uso di un indice Unclustered costringerebbe a continui salti (Page Faults) verso il data file. In questo caso, ignorare l'indice e fare una scansione sequenziale completa (Scan) dell'intero file risulta spesso più efficiente, poiché ogni pagina dati viene caricata in RAM una sola volta.

Analisi degli Aggiornamenti (Scritture) Per ogni operazione di modifica (`INSERT`, `DELETE`, `UPDATE`), bisogna valutare:

- **Attributi Modificati:** Quali colonne vengono alterate? Se un attributo viene aggiornato frequentemente, mantenere un indice su di esso comporterà un notevole sovraccarico prestazionale (*overhead*).
- **Condizioni di Selezione:** Quali attributi vengono usati per individuare i record da modificare o cancellare?

7.5.1 Il Trade-off Fondamentale e la Scelta dell'Indice

La regola d'oro della progettazione fisica è il bilanciamento (**Trade-off**): gli indici accelerano drasticamente le operazioni di lettura (Query), ma rallentano inevitabilmente le operazioni di scrittura (Update), in quanto ogni modifica ai dati richiede un corrispondente aggiornamento strutturale di tutti gli indici associati, oltre a consumare spazio aggiuntivo su disco.

Metodologia di scelta:

1. Si individua la query più importante o frequente del workload.
2. Si valuta il piano di esecuzione (*Query Plan*) migliore con gli indici attualmente esistenti.
3. Si ipotizza la creazione di un indice addizionale: se il miglioramento prestazionale sulla query giustifica il rallentamento indotto sugli aggiornamenti, si procede con la creazione dell'indice.
4. Si ripete il processo considerando l'impatto globale.

7.5.2 Linee Guida per la Costruzione degli Indici

Gli attributi filtrati nelle clausole `WHERE` o usati nei `JOIN` sono i candidati primari per diventare chiavi di ricerca di un indice.

- **Query di Uguaglianza** (es. `età = 30`): Sugeriscono fortemente l'uso di **Indici Hash**, imbattibili per i match esatti.
- **Query per Intervallo / Range** (es. `età > 18`): Sugeriscono l'uso di **Indici ad Albero (B+ Tree)**, preferibilmente **Clustered**. L'ordinamento fisico garantito dal clustering massimizza l'efficienza della scansione dell'intervallo. Il clustering è eccellente anche per le uguaglianze in presenza di molti duplicati.
- **Chiavi di Ricerca Composte:** Se una clausola `WHERE` filtra costantemente su più attributi contemporaneamente (es. `Nome = X AND Cognome = Y`), è opportuno creare un singolo indice su chiave composta (*Cognome, Nome*).
 - L'ordine degli attributi nella dichiarazione dell'indice è vitale (un indice su (*A, B*) può rispondere a query su *A*, ma non a query solo su *B*).

- Gli indici composti aumentano le probabilità di abilitare le scansioni **Index-Only**: se tutti gli attributi richiesti dalla SELECT sono già presenti nelle voci di indice, il DBMS non ha bisogno di accedere al data file. Per le strategie Index-Only, il clustering diventa irrilevante.

Nota sul Clustering: Poiché i record fisici su disco possono essere ordinati in un solo modo, è possibile avere un solo indice **Clustered** per ogni tabella. Questa "risorsa scarsa" deve essere assegnata all'attributo che beneficia maggiormente dell'ordinamento per le query più critiche.

7.5.3 Esempi di Scelta per Indici Clustered

Di seguito analizziamo alcuni scenari per stabilire se e come allocare l'unico indice Clustered disponibile.

```
SELECT E.dno
FROM Emp E
WHERE E.age > 40;
```

Analisi: Questa è una classica query di Range. Indicizzare su age con un albero B+ è la scelta corretta. Se l'indice è **Clustered**, il DBMS scende nell'albero per trovare il primo valore age > 40 e da lì legge sequenzialmente e in modo contiguo tutte le pagine dati necessarie, minimizzando l'I/O.

```
SELECT E.dno, COUNT(*)
FROM Emp E
WHERE E.age > 10
GROUP BY E.dno;
```

Analisi: In questo caso subentrano due operazioni costose: il filtraggio (WHERE age > 10) e il raggruppamento (GROUP BY dno). La decisione del clustering dipende dalla distribuzione dei dati:

- **Opzione A (Clustering su age):** Conviene se la condizione age > 10 è *altamente selettiva* (ovvero, scarta moltissimi record). Si leggono solo le poche pagine rilevanti. Tuttavia, i record recuperati avranno valori dno sparsi; il DBMS dovrà quindi allocare memoria aggiuntiva per raggruppare e contare i dipartimenti.
- **Opzione B (Clustering su dno):** Se la condizione age > 10 è *poco selettiva* (quasi tutti i dipendenti hanno più di 10 anni), l'indice su age diventa inutile (meglio una scansione completa). In questo scenario, se la **cardinalità** di dno è molto alta (ci sono tantissimi dipartimenti diversi che non entrerebbero nei buffer RAM per il calcolo), avere i dati **clusterizzati su dno** fa sì che il DBMS legga la tabella già ordinata per dipartimento. Questo permette di raggruppare e calcolare il COUNT(*) "al volo" durante la scansione, azzerando i costi di ordinamento temporaneo su disco.

```
SELECT E.dno
FROM Emp E
WHERE E.hobby = 'Stamps';
```

Analisi: Questa è una query di uguaglianza. Un indice Unclustered (Hash o Tree) su hobby andrebbe bene se "Stamps" fosse un hobby raro (alta selettività, pochi accessi al data file). Se invece ci sono moltissimi impiegati con questo hobby (bassa selettività), un indice Unclustered causerebbe un disastroso numero di accessi casuali al disco. In questo caso limite, creare un indice **Clustered su hobby** farebbe sì che tutti gli impiegati che collezionano francobolli si trovino nelle stesse pagine fisiche, riducendo drasticamente il costo dell'operazione.

7.5.4 Esempi di Scelta per Query Index Only

Nota sulle Scansioni Index-Only: Una query si definisce *Index-Only* quando tutti gli attributi richiesti dall'interrogazione (sia per il filtraggio nella clausola *WHERE*, sia per la *SELECT* e il raggruppamento) sono interamente contenuti nelle voci di un singolo indice. In questo scenario, il DBMS può soddisfare l'intera query leggendo esclusivamente le pagine dell'indice, senza mai accedere al data file (*Heap File*), con un drastico risparmio di operazioni di I/O.

```
SELECT E.dno , COUNT(*)
FROM Emp E
GROUP BY E.dno;
```

Analisi: Affinché la query sia *Index-Only*, è sufficiente un indice ad albero B+ sull'attributo *dno*. Poiché tutti i dati richiesti sono presenti nell'indice, non vi è alcun accesso al file dati. L'ordinamento intrinseco dell'albero garantisce che le voci di indice per un dato dipartimento siano fisicamente contigue nelle foglie (*Sequence Set*). Questo permette al DBMS di eseguire il raggruppamento e calcolare il *COUNT(*)* "al volo" tramite una singola scansione sequenziale del livello foglia, eliminando completamente i costosi ordinamenti temporanei in RAM.

```
SELECT E.dno , MIN(E.sal)
FROM Emp E
GROUP BY E.dno;
```

Analisi: La scelta ottimale è un indice ad albero composto su (*dno*, *sal*). L'ordine degli attributi nella dichiarazione dell'indice è vitale: raggruppando primariamente per *dno*, i record dei dipartimenti si trovano già contigui per il *GROUP BY*. Il vero vantaggio deriva dall'ordinamento secondario su *sal*: all'interno di ogni dipartimento, le voci di indice sono crescenti per stipendio. Di conseguenza, il *MIN(sal)* si trova esattamente nella *prima* voce di indice di ciascun gruppo *dno*. Il DBMS può quindi ottimizzare la query saltando direttamente alla prima voce di ogni dipartimento e ignorando le restanti, massimizzando l'efficienza di lettura.

```
SELECT AVG(E.sal)
FROM Emp E
WHERE E.age = 25 AND
      E.sal BETWEEN 3000 AND 5000;
```

Analisi: Un indice composto su (*age*, *sal*) supporta perfettamente questa interrogazione. Il DBMS sfrutta la ricerca gerarchica dell'albero per risolvere la condizione di uguaglianza *age = 25* in tempo logaritmico. Raggiunta la prima foglia utile, l'ordinamento secondario su *sal* permette di scandire in sequenza unicamente le voci che soddisfano il range *BETWEEN 3000 AND 5000*. Il calcolo della media (che a livello logico richiede una somma e un conteggio) viene effettuato elaborando in memoria esclusivamente i dati estratti dalle foglie dell'indice.

```
SELECT E.dno , COUNT(*)
FROM Emp E
WHERE E.age = 30
GROUP BY E.dno;
```

Analisi: L'indice composto ottimale in questo caso è (*age*, *dno*). La condizione altamente selettiva di uguaglianza (*age = 30*) permette di scendere nell'albero per individuare l'inizio del set di dati pertinente. Poiché, a parità di età, le voci di indice sono ordinate rispetto alla seconda chiave (*dno*), il DBMS disporrà dei record dei dipendenti trentenni già raggruppati per dipartimento. Questo abilita la computazione del *COUNT(*)* in sequenza diretta (al volo) per i soli record filtrati, senza gravare sulle risorse CPU e RAM per ordinamenti postumi.

```
SELECT E.dno , COUNT(*)  
FROM Emp E  
WHERE E.age > 30  
GROUP BY E.dno ;
```

Analisi: L'utilizzo di un operatore di disuguaglianza (>) introduce un classico *trade-off* tra il costo di scansione e il costo di ordinamento in RAM. Vi sono due alternative Index-Only:

- **Opzione A (Indice su age , dno):** Si posiziona sul primo valore age > 30 e scorre il Sequence Set in avanti. Il numero di pagine lette è **minimizzato** perché si legge solo la porzione di indice pertinente. Tuttavia, poiché l'ordinamento primario è per età, i valori dei dipartimenti (dno) saranno sparsi; il DBMS dovrà pertanto allocare memoria aggiuntiva (es. tramite Hash Table) per accorpare i gruppi e calcolare i contatori.
- **Opzione B (Indice su dno , age):** L'indice è ordinato primariamente per dipartimento, il che azzerà i costi in memoria per il raggruppamento (il GROUP BY è "gratis"). Tuttavia, per ogni singolo blocco di dipartimento, il DBMS dovrà ricercare il sottoinsieme con age > 30. Questo lo costringerà, nel peggiore dei casi, a leggere (o quantomeno attraversare) gran parte o la totalità delle foglie dell'indice.

In sintesi, l'Opzione A è raccomandabile se la condizione age > 30 è altamente selettiva, mentre l'Opzione B risulta vincente qualora la maggior parte della popolazione impiegatizia superi i 30 anni, giustificando la scansione estesa dell'indice pur di evitare l'overhead di calcolo.

8 Gestione del Buffer (Buffer Manager)

Il **Buffer Manager** è il componente del DBMS responsabile della gestione della memoria centrale (RAM) allocata per ospitare temporaneamente le pagine dati lette dal disco. Poiché la RAM è una risorsa volatile e di capacità intrinsecamente limitata rispetto all'intero database, il compito cruciale di questo modulo è orchestrare in modo ottimale il trasferimento di pagine tra disco e memoria.

8.1 Obiettivi e Metriche di Valutazione

Per garantire alte prestazioni, il Buffer Manager deve essere:

- **Efficiente:** Le sue logiche decisionali interne devono avere un costo computazionale quasi nullo, dato che il gestore entra in gioco ad ogni singola richiesta di accesso ai dati.
- **Efficace:** L'efficacia si misura in base al numero di *Page Fault* (costose letture dal disco) risparmiati. L'obiettivo strategico è trattenere in RAM le pagine utili, evitando l'espulsione (*eviction*) di blocchi che serviranno nell'immediato futuro.

8.2 Politiche di Rimpiazzo (Replacement Policies)

Quando il buffer è pieno e il sistema necessita di caricare una nuova pagina dal disco, deve scegliere quale pagina residente "sacrificare". A priori non esiste una strategia universalmente perfetta, ma si adottano approcci basati su pattern temporali:

- **Principio di Località Temporale:** Assume che una pagina utilizzata di recente abbia un'elevata probabilità di essere richiesta nuovamente a breve.
 - **LRU (Least Recently Used):** Rimpiazza la pagina che non viene utilizzata da più tempo in assoluto.
 - **LFU (Least Frequently Used):** Rimpiazza la pagina che storicamente ha registrato il minor numero di accessi.
- **Principio di Antilocalità Temporale:** Ribalta l'assunzione precedente, ipotizzando che la pagina appena elaborata abbia esaurito il suo scopo e non servirà più a breve termine.
 - **MRU (Most Recently Used):** Rimpiazza la pagina utilizzata più di recente.

8.3 Differenza tra OS e DBMS: Il problema del Sequential Flooding

I normali Sistemi Operativi (OS) gestiscono la memoria virtuale applicando quasi esclusivamente politiche di località temporale (come LRU). Questo perché l'OS non può prevedere il comportamento delle applicazioni e non fa ipotesi sui pattern di utilizzo.

Il DBMS, al contrario, genera e conosce in anticipo il *Query Plan* (es. sa quando dovrà eseguire un loop su una tabella per un JOIN). Sfruttando queste informazioni, il DBMS può adattare la politica di rimpiazzo per prevenire crolli prestazionali gravissimi, come il **Sequential Flooding**.

Sequential Flooding (Allagamento Sequenziale): Si verifica quando un pattern di accesso ciclico e sequenziale agisce su un set di pagine di dimensioni lievemente superiori alla capacità del buffer. In questo scenario, le politiche standard come LRU falliscono sistematicamente, portando il tasso di Page Fault al 100%.

Analisi di un caso di Sequential Flooding Supponiamo che il DBMS debba eseguire un *Nested Loop Join*, accedendo ciclicamente a una relazione interna composta da 4 pagine (p_1, p_2, p_3, p_4). Il nostro buffer, però, ha spazio solo per **3 pagine**.

- **Caso Peggior: Politica LRU**

1. Lettura sequenziale iniziale: il buffer si riempie con $[p_1, p_2, p_3]$.

2. Richiesta di p_4 : LRU espelle p_1 (la meno recente). Buffer: $[p_4, p_2, p_3]$.
3. Inizia il nuovo ciclo, serve p_1 : LRU espelle p_2 . Buffer: $[p_4, p_1, p_3]$.
4. Serve p_2 : LRU espelle p_3 . Buffer: $[p_4, p_1, p_2]$.

Risultato: Ogni singolo accesso genera un Page Fault. Il buffer è reso inutile.

- **Soluzione DBMS: Politica MRU per pattern ciclici (Sequential Looping)**

1. Lettura sequenziale iniziale: buffer pieno con $[p_1, p_2, p_3]$.
2. Richiesta di p_4 : MRU espelle p_3 (l'ultima appena letta). Buffer: $[p_1, p_2, p_4]$.
3. Inizia il nuovo ciclo, serve p_1 : è già in RAM (**Page Hit!**).
4. Serve p_2 : è già in RAM (**Page Hit!**).
5. Serve p_3 : MRU espelle p_2 (l'ultima usata). Buffer: $[p_1, p_3, p_4]$.

Risultato: La politica MRU riesce a stabilizzare parte delle pagine in memoria (p_1 e p_4), garantendo numerosi Page Hit e riducendo drasticamente le operazioni di I/O su disco.

8.4 Algoritmo di Rimpiazzo a Orologio (Clock Algorithm)

L'algoritmo *Clock* è un'approssimazione efficiente della politica LRU. Idealmente, le pagine residenti nel buffer vengono disposte in una lista circolare (come i numeri sul quadrante di un orologio). Una "lancetta" (un puntatore) scorre le pagine per individuare quella da rimpiazzare.

Ogni pagina nel buffer (frame) è associata a due variabili di stato fondamentali:

- **pincount:** Un contatore che indica quanti processi stanno attualmente leggendo o scrivendo quella pagina. Se il **pincount** > 0 , la pagina è "bloccata" (*pinned*) e non può essere espulsa.
- **referenced** (o *Reference Bit*): Un flag booleano (0 o 1). Viene settato a 1 dal sistema ogni volta che la pagina subisce un accesso. Serve a tracciare se la pagina è stata utilizzata di recente.

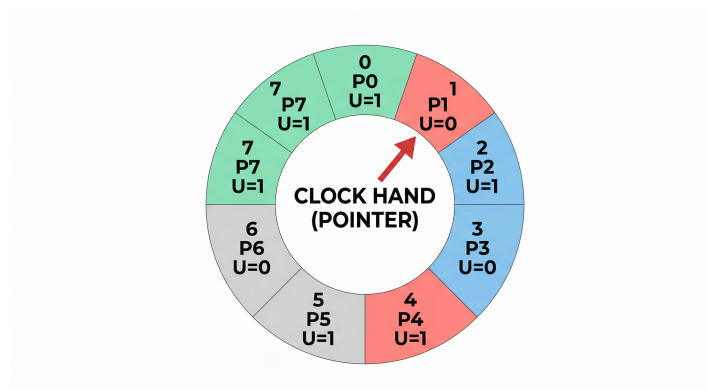


Figure 30: Rappresentazione dell'algoritmo Clock

Logica di rimpiazzamento Quando il buffer è pieno e serve un frame libero, la lancetta dell'orologio inizia a scorrere:

1. Se la pagina corrente ha **pincount** > 0 , viene ignorata (è in uso) e si passa alla successiva.
2. Se la pagina ha **pincount** $== 0$ ma **referenced** $== 1$, significa che è stata usata recentemente. L'algoritmo le dà una "seconda chance": resetta il bit **referenced** $= 0$ e passa alla pagina successiva.
3. Se la pagina ha **pincount** $== 0$ e **referenced** $== 0$, significa che è sopravvissuta a un intero giro di orologio senza essere mai richiesta. Questa è la **pagina vittima** che viene espulsa e rimpiazzata.

```
// Pseudocodice semplificato dell'algoritmo Clock
loop until (found eligible page) or (all pages pinned):
    if page[current].pincount > 0:
        current++
    else if page[current].referenced == 1:
        page[current].referenced = 0
        current++
    else: // pincount == 0 AND referenced == 0
        replace_page(page[current])
        page[current].referenced = 1
        current++
    break
```

Nota sulle Prestazioni: Il Clock Algorithm non implementa una LRU perfetta (l'ordine di espulsione non è strettamente cronologico rispetto all'ultimo utilizzo), ma è enormemente più leggero. La LRU pura richiede la gestione di una coda con priorità, con un costo di aggiornamento computazionale ad ogni singolo accesso. Il Clock, invece, richiede solo la modifica di un bit.

8.5 Evoluzione delle Strategie di Allocazione

Oltre alle politiche di rimpiazzo standard, i DBMS moderni adottano modelli più sofisticati per allocare lo spazio del buffer in base alla natura dei dati e delle query.

8.5.1 Domain Separation (Separazione per Dominio)

Si basa sull'assunto che file di tipo diverso abbiano pattern di accesso e importanza differenti (es. le pagine di un indice B+ Tree sono lette molto più frequentemente delle singole pagine dati).

- **Meccanismo:** Il buffer viene partizionato in "domini" (es. un dominio per gli indici, uno per i dati). Le espulsioni e i rimpiazzamenti avvengono *esclusivamente* all'interno dello stesso dominio.
- **Vantaggi:** Semplicità implementativa; protegge le pagine critiche (come la radice di un indice) dall'essere sovrascritte da massicce scansioni di dati.
- **Svantaggi:** È una strategia statica. Non tiene conto del carico di lavoro effettivo (*workload*), della tipologia di query in esecuzione o della loro frequenza. Migliora la LRU, ma risulta inflessibile in un database altamente dinamico.

8.5.2 Working Set Model

Sposta la granularità dell'allocazione dal "dominio" alla specifica **relazione** (tabella o indice). Assegna maggiore priorità (e più spazio) alle relazioni maggiormente utilizzate.

- **Meccanismo:** Il buffer è ripartito tra le relazioni attive. Per ciascuna relazione è garantito almeno un frame "intoccabile". Ogni relazione mantiene il proprio *resident_set* (l'insieme delle sue pagine in RAM) ed è gestita da una coda di priorità globale basata su MRU (Most Recently Used).
- **Vantaggi:** Scala meglio della LRU pura, poiché si aggiorna la coda a livello di "relazione" e non ad ogni singolo accesso di pagina.
- **Svantaggi:** L'efficacia dipende criticamente da due fattori difficili da calibrare: la frequenza di riavvio della coda di priorità e l'uniformità del ritmo di accesso. Inoltre, l'uso incondizionato della MRU non è ottimale per tutti i pattern.

8.5.3 HotSet Model

Sposta ulteriormente l'attenzione dalle relazioni alle **query**.

- **Meccanismo:** Il modello cerca di allocare a ciascuna query un numero di pagine pari al suo **Hot Point**. L'Hot Point è la soglia critica di pagine allocate oltre la quale il numero di Page Fault crolla drasticamente (es. in un loop di 4 pagine, se ne alloco 3 ho continui Page Fault, se ne alloco 4 scendono a zero. L'Hot Point è 4).
- **Svantaggi:** Essendo calcolato per ottimizzare la singola query nel caso peggiore, tende a **sovrastimare** lo spazio necessario, sprecando memoria. Inoltre, se più query leggono la stessa pagina, l'HotSet rischia di replicarla in aree di buffer diverse, frammentando le risorse.

8.6 Query Locality Set Model (QLSM)

È l'evoluzione definitiva dell'HotSet. Considera simultaneamente **le query, le relazioni e la condivisione delle pagine**. A differenza dell'HotSet, previene la replicazione garantendo che ogni pagina fisica esista in una sola copia nel buffer globale, assegnandone la "proprietà" al *Locality Set* che l'ha richiesta per ultimo.

Il QLSM ottimizza l'allocazione prevedendo in anticipo il comportamento della query in base a specifici **Pattern di Accesso**:

Pattern Sequenziali

- **Straight Sequential (SS):** Scansione sequenziale pura, senza riutilizzo (es. calcolo di una media). Una volta letta, la pagina non serve più.
Risorse necessarie: **1 Frame** (sovrascritto a ogni passo).
- **Clustered Sequential (CS):** Lettura sequenziale a blocchi (cluster) di record.
Risorse necessarie: Numero di frame pari alla **Dimensione del Cluster**.
- **Looping Sequential (LS):** Scansione sequenziale ripetuta ciclicamente (es. Nested Loop Join).
Risorse necessarie: Idealmente **tutte le pagine** della relazione. Se impossibile, il numero massimo di frame disponibili (gestiti tramite MRU per evitare il Sequential Flooding).

Pattern Casuali (Random)

- **Independent Random (IR):** Accessi puntuali e isolati (es. ricerca tramite Hash Index).
Risorse necessarie: **1 Frame**.
- **Clustered Random (CR):** Accessi casuali che estraggono blocchi di dati.
Risorse necessarie: Numero di frame pari alla **Dimensione del Cluster**.

Pattern Gerarchici (Accesso ad Alberi)

- **Straight Hierarchical (SH):** Discesa dalla radice alla foglia di un B+ Tree per una singola ricerca. Ogni nodo genitore può essere scartato dopo aver letto il figlio.
Risorse necessarie: **1 Frame** (sovrascritto scendendo nell'albero).
- **SH seguito da SS (Range Query):** Discesa nell'albero (SH) seguita dalla scansione sequenziale delle foglie (SS).
Risorse necessarie: **2 Frame** (1 per mantenere fissa la radice, 1 per scorrere sequenzialmente le foglie).
- **SH seguito da CS:** Discesa nell'albero seguita da una lettura a blocchi (es. Indice Clustered).
Risorse necessarie: **1 Frame** (radice) + **Dimensione del Cluster**.
- **Looping Hierarchical (LH):** Discese ripetute sullo stesso indice (es. ricerca di molte chiavi tramite un B+ Tree in un Join).
Risorse necessarie: Idealmente **tutte le pagine dell'indice** (o quantomeno i livelli superiori dell'albero per massimizzare i Page Hit). Gestito tramite MRU.

8.6.1 Algoritmo Globale di Gestione QLSM

Il sistema mantiene una tabella globale condivisa, ma assegna policy di rimpiazzo locali (generalmente LRU, tranne per LS e LH che usano MRU) ai singoli *Locality Set* generati dalle query.

Quando una query richiede una pagina P nel suo Locality Set L:

1. **P è nella Tabella Globale ed è assegnata a L:** *Page Hit*. Si aggiorna lo stato secondo la policy locale di L.
2. **P è nella Tabella Globale ma non in L:** La pagina è già in RAM per un'altra query. P viene trasferita/condivisa e assegnata al nuovo Locality Set L.
3. **P non è presente (Page Fault):** La pagina viene caricata dal disco in memoria centrale e assegnata al Locality Set L. Se il buffer di L è pieno, si applica la sua *policy locale* per espellere una pagina e far posto a P.

9 Valutazione degli operatori

Uno dei punti di forza fondamentali del modello relazionale è l'utilizzo di linguaggi dichiarativi (come l'SQL). L'utente specifica *quali* dati desidera ottenere, senza doversi preoccupare di *come* il sistema debba recuperarli. Il compito di tradurre questa richiesta dichiarativa in un piano di esecuzione procedurale (*Query Plan*) efficiente è affidato al **Query Optimizer**.

Poiché il numero di combinazioni possibili (algoritmi e ordine degli operatori) per risolvere una query cresce in modo esponenziale rispetto al numero di operatori coinvolti, l'ottimizzatore utilizza le statistiche memorizzate nel catalogo del DBMS per stimare il costo di ciascun piano e selezionare quello sub-ottimale (il più efficiente in termini di I/O su disco e uso della CPU).

9.1 Operatori Relazionali e Operazioni Fisiche

Il Query Plan è composto da un albero di operatori fisici. Possiamo distinguere tra le operazioni derivate direttamente dall'algebra relazionale e quelle implicite, necessarie per la gestione fisica dei dati.

9.1.1 Operatori dell'Algebra Relazionale

- **Selezione (σ)**: Data una relazione e un predicato (criterio di selezione), agisce orizzontalmente restituendo solo i record (tuple) che soddisfano la condizione.
- **Proiezione (π)**: Data una relazione, agisce verticalmente restituendo una nuova relazione che contiene solo gli attributi specificati. *Nota teorica*: nel modello relazionale puro non esistono duplicati, ma un'operazione di proiezione SQL standard li mantiene a meno che non sia specificata la clausola `DISTINCT`.
- **Join ($\bowtie$)**: Combina due relazioni in base a una condizione che lega i loro attributi. Teoricamente equivale a un prodotto cartesiano seguito da una selezione, ma a livello fisico viene implementato con algoritmi dedicati (es. Nested Loop, Hash Join, Merge Join) per questioni di efficienza.
- **Set-Difference ($-$)**: Date due relazioni compatibili, restituisce i record presenti nella prima ma non nella seconda.
- **Unione ($\cup$)**: Date due relazioni con lo stesso schema, restituisce i record presenti in almeno una delle due.
- **Aggregazione e Raggruppamento ($SUM, MIN, MAX, GROUP BY$)**: Raggruppa i record con valori uguali in determinati attributi e calcola un valore riassuntivo per ciascun gruppo.

9.1.2 Operazioni Fisiche Implicite

L'algebra relazionale lavora su insiemi (non ordinati e senza duplicati), ma i DBMS fisici lavorano su multinsiemi. Diventano quindi cruciali due operazioni di supporto:

- **Ordinamento (Sorting)**: Spesso richiesto esplicitamente dall'utente (`ORDER BY`) o usato come passo intermedio per algoritmi di Join o per l'eliminazione dei duplicati. Poiché i dati superano quasi sempre la capacità della RAM, non è possibile usare algoritmi in-memory (come QuickSort o HeapSort) sull'intero dataset. Si ricorre ad algoritmi out-of-core come l'**External Merge Sort**, che processa i dati a blocchi usando file temporanei su disco.
- **Eliminazione dei Duplicati (`DISTINCT`)**: Può essere implementata in due modi principali:
 - *Tramite Ordinamento*: Si ordinano i record in modo che i duplicati risultino adiacenti. Basterà poi una scansione sequenziale per filtrare i valori identici consecutivi.
 - *Tramite Hashing*: Si partizionano i record in bucket usando una funzione di hash. I duplicati finiranno inevitabilmente nello stesso bucket, dove potranno essere identificati ed eliminati.

9.2 Classificazione degli Algoritmi di Valutazione

Le strategie fisiche per implementare gli operatori si dividono in tre macro-categorie:

- **Iterazione (Scansione):** Si scansiona l'intera relazione (Table Scan), blocco per blocco, valutando la condizione per ogni record. È la soluzione di ripiego (o a volte la migliore, se si deve leggere gran parte della tabella).
- **Indicizzazione:** Si sfruttano strutture dati ausiliarie (B+ Tree, Hash Index) create in fase di progettazione per localizzare rapidamente i record senza leggere l'intero file.
- **Partizionamento:** Si divide l'input in sottoinsiemi più piccoli (es. tramite hashing o ordinamento) che possono essere elaborati in modo indipendente in memoria centrale.

9.3 Implementazione dell'Operatore di Selezione (σ)

9.3.1 Query con Singola Condizione

Obiettivo: Trovare tutti i record che soddisfano un predicato semplice (es. $\text{Età} = 25$ oppure $\text{Salario} > 2000$).

Caso Standard: File Scan Se non esistono indici o se la query richiede la restituzione della quasi totalità della tabella, si opta per una scansione sequenziale pura (*Straight Sequential*).

- **Uso del Buffer:** Sono sufficienti **2 frame**. Un frame funge da buffer di input per caricare le pagine lette dal disco, l'altro da buffer di output per accumulare i record validi da passare all'operatore successivo.

Caso con Indice (Index Scan) Se esiste un indice sull'attributo filtrato, l'algoritmo varia in base alla tipologia:

- **Indice Hash:** Ideale per query di uguaglianza. Si applica la funzione di hash per trovare il bucket e si recuperano direttamente i record.
- **Indice ad Albero (B+ Tree):** Ottimo sia per uguaglianze che per range query (es. $>$, $<$). Si scende l'albero fino alla foglia appropriata. Se l'indice è **Clustered** (i dati fisici sono ordinati come l'indice), le letture successive saranno sequenziali e contigue, massimizzando l'efficienza.

Il Problema dell'Indice Unclustered Se l'indice è *Unclustered*, i record con valori simili nell'indice sono sparsi in pagine dati casuali sul disco. Recuperare ogni singolo record tramite i puntatori dell'indice (Record ID - RID) potrebbe generare una lettura su disco (*Page Fault*) per ogni tuple.

- **Ottimizzazione (Sorting dei RID):** Invece di accedere immediatamente ai dati, il DBMS estrae tutti i RID dalle foglie dell'indice e li **ordina per ID di pagina fisica**. In questo modo, tutte le tuple richieste presenti nella stessa pagina fisica verranno lette insieme, garantendo che ogni pagina su disco venga caricata in RAM **una sola volta**.

9.3.2 Query Complesse (Condizioni Congiunte)

Quando la selezione coinvolge più attributi (es. $\sigma_{A=x \wedge B=y}$), e disponiamo di indici su entrambi.

Approccio 1: Uso dell'indice più selettivo Il DBMS usa le statistiche per determinare quale condizione filtri il maggior numero di dati (la più *selettiva*). Si interroga solo quell'indice, si recuperano le tuple associate e si verifica la seconda condizione direttamente in memoria (*on-the-fly*).

- **Eccezione:** L'ottimizzatore potrebbe preferire un indice meno selettivo ma che fornisce i dati già **ordinati**, nel caso in cui un operatore successivo (es. un Merge Join o un ORDER BY) richieda quel preciso ordinamento, risparmiando così un costoso sorting esplicito.

Approccio 2: Intersezione degli Indici (RID Intersection) Se entrambi gli indici sono unclustered, accedere ai dati usando solo uno dei due potrebbe comportare troppe letture casuali se la singola condizione non è abbastanza selettiva.

- Il DBMS attraversa *entrambi* gli indici ed estrae le due liste di RID (identificatori fisici dei record).
- Esegue in memoria l'**intersezione** di queste due liste, ottenendo solo i RID delle tuple che soddisfano entrambe le condizioni.
- Solo a questo punto i RID risultanti vengono ordinati per pagina fisica e si esegue l'accesso ai dati su disco, riducendo drasticamente l'I/O.

9.4 Implementazione dell'Operatore di Proiezione (π)

La proiezione rimuove colonne indesiderate ed, eventualmente, elimina le tuple duplicate.

Implementazione Standard (Scansione + Eliminazione Duplicati) L'approccio di base prevede di leggere la tabella in modo sequenziale (con **2 frame** di buffer), isolare i campi richiesti e scartare il resto. Tuttavia, se la proiezione genera duplicati (e la query richiede di eliminarli), l'output provvisorio dovrà essere sottoposto a una fase di Hashing o di Sorting esterno, aumentando notevolmente il costo dell'operazione.

Ottimizzazione Massima: Index-Only Scan Se esiste un indice (Clustered o Unclustered non importa) la cui chiave di ricerca **contiene già tutti gli attributi** richiesti dalla proiezione, il DBMS può ignorare completamente la tabella dati (*Heap File*).

- Il sistema legge sequenzialmente solo le **pagine foglia dell'indice**, estraendo da lì i valori necessari. L'operazione è rapidissima perché gli indici occupano una frazione dello spazio rispetto all'intera tabella.
- Se l'indice utilizzato è un **B+ Tree**, le foglie sono già ordinate in base alla chiave. Di conseguenza, eventuali duplicati saranno naturalmente adiacenti, permettendo di implementarli o filtrarli istantaneamente senza costi di sorting aggiuntivi. Se si usa un indice Hash, i duplicati si trovano nello stesso bucket, facilitando anch'esso la deduplicazione, ma non garantisce alcun ordinamento globale utile per query successive.

9.5 L'Operatore di Ordinamento

L'ordinamento (Sorting) è un operatore ausiliario, non formalmente presente nell'algebra relazionale di base, ma fondamentale nell'implementazione dei DBMS. Può essere richiesto esplicitamente dall'utente (tramite la clausola `ORDER BY`) o utilizzato dal sistema come passo intermedio per ottimizzare altri operatori (es. per il Sort-Merge Join o per l'eliminazione dei duplicati in una proiezione con `DISTINCT`).

Questa operazione è particolarmente critica a livello fisico. Per dataset di piccole dimensioni, algoritmi *in-memory* (come il QuickSort) risultano estremamente efficienti poiché lavorano *in-place* (effettuando scambi direttamente nell'array caricato in RAM). Tuttavia, quando la dimensione della relazione supera la capacità della memoria centrale allocata (i buffer frame), gli algoritmi tradizionali generano un numero inaccettabile di Page Fault.

Per questo motivo, i DBMS si affidano a varianti dell'**External Merge Sort**, una famiglia di algoritmi di ordinamento esterno (*out-of-core*) progettata per minimizzare gli accessi al disco. L'approccio generale consiste nel dividere il dataset in blocchi (run) sufficientemente piccoli da essere ordinati in RAM, scriverli temporaneamente su disco, e successivamente fonderli (merge) in modo efficiente. Il limite teorico inferiore per ordinare N pagine è $O(N \log N)$.

9.5.1 2-Way External Merge Sort

Il 2-Way Sort è la forma più basilare di ordinamento esterno. Consente di ordinare un intero file sfruttando il disco come area di appoggio temporanea per i risultati parziali, richiedendo un minimo rigoroso di soli **3 frame di buffer in RAM** (due per l'input, uno per l'output).

L'algoritmo emula l'approccio *divide et impera* del Merge Sort tradizionale. Durante la fase di fusione, si leggono sequenzialmente i dati da due flussi (run) di input precedentemente ordinati. Il DBMS confronta i record correnti dei due flussi (che, essendo ordinati, si trovano in testa), seleziona il minore e lo scrive nel frame di output. Quando la pagina di output è piena, viene scritta su disco (flushed) e il buffer viene svuotato per accogliere i record successivi.

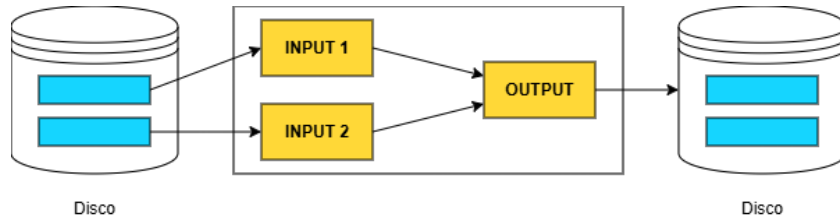


Figure 31: Visualizzazione del 2-Way Sort

L'algoritmo si sviluppa in passi (pass):

- **Passo 0 (Inizializzazione):** Si carica una pagina alla volta in memoria, la si ordina al suo interno (usando algoritmi in-place come il QuickSort) e la si riscrive su disco. Da un file di N pagine, si ottengono N run ordinati di lunghezza pari a 1 pagina.
- **Passo 1:** Si considerano i run a due a due. Si legge la prima (e unica) pagina di ciascun run nei due buffer di input. Si fondono i record estraendo progressivamente il minimo, riempiendo il buffer di output e scaricandolo su disco. Si ottengono così $\lceil N/2 \rceil$ run ordinati, ciascuno lungo 2 pagine.
- **Passo 2:** Si fondono a coppie i run da 2 pagine ottenuti al passo precedente, generando run ordinati da 4 pagine. L'operazione richiede letture puramente sequenziali.
- **Passi successivi:** Si reitera il processo, raddoppiando ad ogni passo la lunghezza dei run, fino a ottenere un unico grande run ordinato contenente l'intero dataset.

Analisi dei Costi (I/O): Il numero totale di passi necessari (incluso il Passo 0) è pari a $1 + \lceil \log_2 N \rceil$. Ad ogni singolo passo, l'intero file (tutte le N pagine) deve essere letto dal disco e, dopo la fusione, interamente riscritto. Il costo di I/O per un singolo passo è quindi $2N$. Il costo totale ammonta a:

$$2N (\lceil \log_2 N \rceil + 1)$$

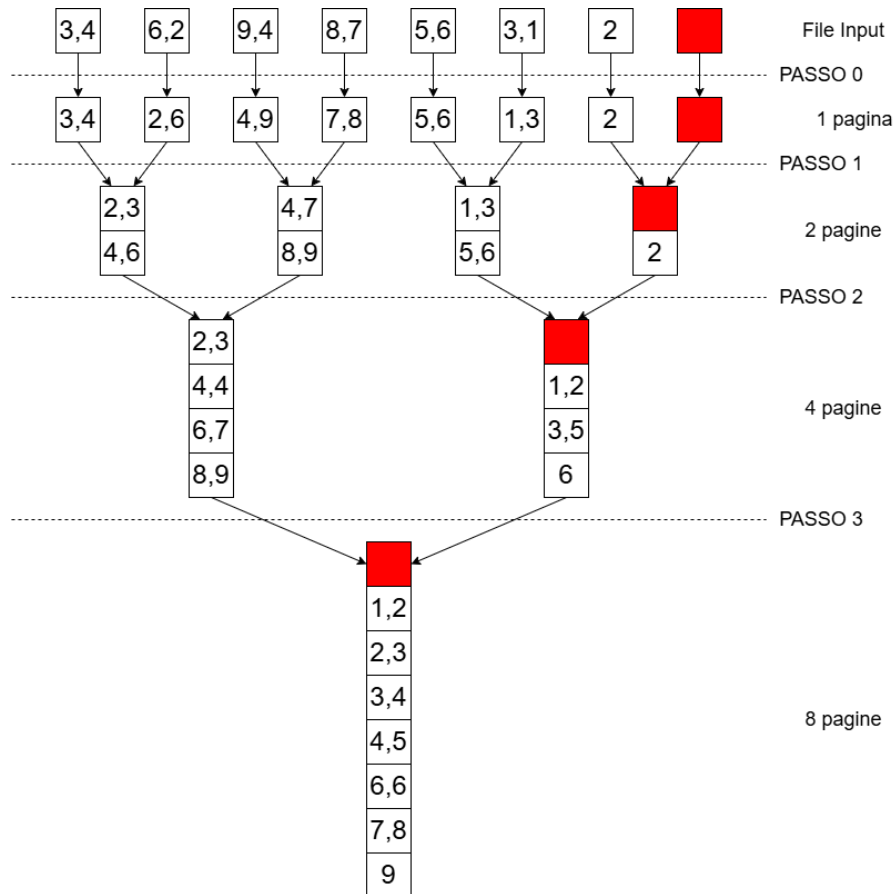


Figure 32: Esempio di applicazione del 2-Way External Merge Sort

9.5.2 General External Merge Sort (con (B) Buffer)

Il 2-Way Sort risulta inefficiente per tabelle di grandi dimensioni a causa del logaritmo in base 2, che genera numerosi passaggi di lettura/scrittura sull'intero file. Nella realtà, i DBMS dispongono di B frame di buffer (dove $B \gg 3$). Possiamo sfruttare questa memoria aggiuntiva per abbattere drasticamente i costi.

Invece di fondere i run a due a due, possiamo allocare $B - 1$ frame per i flussi di input e mantenere 1 frame per l'output. Questo ci permette di fondere $B - 1$ **run ordinati simultaneamente** (*multi-way merge*).

Inoltre, i buffer extra ottimizzano sensibilmente il **Passo 0**: anziché ordinare singole pagine isolate, possiamo caricare contemporaneamente blocchi di B pagine in RAM, ordinarli internamente (es. tramite QuickSort), e scriverli su disco. Questo genera run iniziali molto più lunghi, riducendo le iterazioni successive.

- **Passo 0 (Inizializzazione):** Si leggono gruppi di B pagine per volta. Vengono ordinati in memoria e riscritti. Si generano $\lceil N/B \rceil$ run ordinati, ciascuno di dimensione B pagine.
- **Fase di Merge:** Si fondono $B - 1$ run alla volta. Questo aumenta la base del logaritmo, riducendo l'altezza dell'albero di merge.

Analisi dei Costi (I/O): Il numero di passi di merge si riduce a $\lceil \log_{B-1} \lceil N/B \rceil \rceil$. Sommando il passo iniziale, il costo totale dell'operazione diventa:

$$2N \left(1 + \left\lceil \log_{B-1} \left\lceil \frac{N}{B} \right\rceil \right\rceil \right)$$

N	B=3	B=5	B=9	B=17	B=129	B=257
100	7	4	3	2	1	1
1,000	10	5	4	3	2	2
10,000	13	7	5	4	2	2
100,000	17	9	6	5	3	3
1,000,000	20	10	7	5	3	3
10,000,000	23	12	8	6	4	3
100,000,000	26	14	9	7	4	4
1,000,000,000	30	15	10	8	5	4

Table 1: Numero di passi per External Sort in funzione di N e B

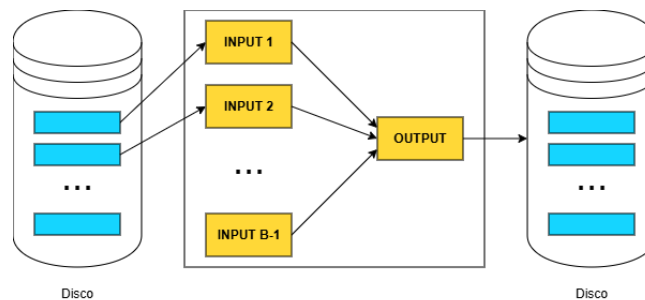


Figure 33: Visualizzazione dell'External Merge Sort con più di 2 flussi di input

9.5.3 Ottimizzazione del Passo 0: Replacement Selection (Internal Sort)

Un'ulteriore e sofisticata ottimizzazione riguarda la generazione dei run iniziali. Con il QuickSort, disponendo di B frame, possiamo creare run ordinati lunghi al massimo B pagine. Tuttavia, adottando una variante dell'**Heap Sort** (tecnica nota come *Replacement Selection*), è possibile generare dinamicamente run più lunghi, migliorando l'efficienza globale.

Funzionamento dell'algoritmo:

1. Si caricano B pagine in RAM e si strutturano in una **Coda con Priorità** (Min-Heap), dove l'elemento minimo è sempre prelevabile in testa a costo costante.
2. Si estrae il valore minimo dalla Heap e lo si sposta nel buffer di output.
3. Essendosi liberato un posto in RAM, si legge un nuovo record dal buffer di input e lo si valuta:
 - Se il nuovo record è $\geq$ all'ultimo record appena scritto in output, rispetta l'ordinamento. Viene quindi inserito nella Min-Heap "attiva" e farà parte del run corrente.
 - Se il nuovo record è $<$ all'ultimo record scritto, non può fisicamente appartenere al run corrente (violerebbe l'ordinamento). Viene perciò inserito in una "Heap congelata" che verrà utilizzata solo per il *prossimo* run.
4. Il processo continua iterativamente. Quando tutti gli elementi nella Heap attiva si esauriscono (ovvero tutti i nuovi record letti erano più piccoli degli ultimi scritti), il run corrente viene chiuso su disco. La Heap congelata diventa la nuova Heap attiva e si inizia un nuovo run.

Risultati:

- **Caso peggiore:** I dati di input sono in ordine rigidamente decrescente. Ogni nuovo record letto andrà nella Heap congelata. I run generati saranno lunghi esattamente B pagine (nessun miglioramento rispetto al QuickSort).
- **Caso migliore:** I dati di input sono già quasi ordinati. La Heap attiva non si svuota mai. Si genera un unico run infinito (l'intero file viene ordinato al Passo 0).

- **Caso medio:** Come dimostrato in letteratura (analogia dello "spazzaneve" di Knuth), la lunghezza media dei run generati con questa tecnica è **pari a $2B$ pagine**, il doppio rispetto all'approccio standard.

9.6 L'Operatore di Join ($\bowtie$)

Date due relazioni, e data una condizione (predicato) che lega gli attributi dell'una a quelli dell'altra, l'operatore di join restituisce tutte le coppie di record che soddisfano tale condizione. Il predicato può essere matematico (es. minore, maggiore, $\neq$, noto come *Theta-Join*) oppure di stretta uguaglianza (noto come *Equi-Join*).

Il Join è tipicamente l'operatore più costoso da eseguire all'interno di un DBMS, pertanto l'ottimizzatore dedica molta attenzione alla scelta dell'algoritmo fisico più appropriato.

L'approccio "Naive" (Inefficiente) Una prima implementazione logica, banale ma corretta, consiste nello sfruttare operatori base già implementati: calcolare prima il prodotto cartesiano ($R \times S$) delle due relazioni e successivamente applicare un operatore di selezione (σ) per filtrare solo le coppie che rispettano il criterio di join.

Questa strada è **estremamente inefficiente** a livello fisico. Il prodotto cartesiano di due relazioni composte rispettivamente da n e m tuple genera un output temporaneo di $n \cdot m$ tuple. Se la condizione di join è molto selettiva (cioè poche coppie matchano), il sistema avrà sprecato enormi quantità di I/O su disco e cicli di CPU per generare e scrivere un file temporaneo gigantesco, per poi scartarne il 99%.

Per questo motivo, i DBMS non usano quasi mai il prodotto cartesiano esplicito, ma si affidano ad algoritmi ad hoc studiati per minimizzare gli accessi al disco (Page Faults).

9.7 Algoritmi basati su Cicli Annidati (Nested Loops)

9.7.1 Simple Nested Loop Join (Tuple-oriented)

È l'approccio più basilare, che implementa il join tramite due cicli iterativi annidati direttamente sulle tuple.

```
foreach tuple r in R do
  foreach tuple s in S do
    if r.i == s.j then // La condizione può essere qualsiasi
      add <r, s> to result
```

Chiamiamo **relazione esterna** (R) quella iterata nel ciclo principale, e **relazione interna** (S) quella iterata nel ciclo secondario.

Analisi del Costo (I/O): Per ogni singola tupla della relazione esterna, il DBMS deve scansionare l'intera relazione interna. Assumendo:

- M : numero di pagine fisiche della relazione R (esterna)
- p_R : numero di tuple per ogni pagina di R
- N : numero di pagine fisiche della relazione S (interna)

Il costo in termini di I/O (pagine lette dal disco) è:

$$Cost = M + (p_R \cdot M) \cdot N$$

Questo costo è proibitivo. Anche per tabelle di medie dimensioni, il numero di letture su disco esplode rapidamente, poiché la tabella interna S viene caricata e scaricata dalla RAM $p_R \cdot M$ volte.

9.7.2 Page-oriented Nested Loop Join

Un miglioramento drastico si ottiene ragionando per pagine fisiche anziché per singole tuple. Invece di ricaricare le pagine della relazione interna per ogni singola tupla di R , le carichiamo una volta sola per ogni *pagina* di R .

Logica: Si carica in un frame del buffer una pagina di R . Per tutte le tuple contenute in quella pagina, si scansiona S pagina per pagina. In questo modo, la relazione interna S viene letta interamente solo una volta per ogni *pagina* di R , non per ogni tupla.

Analisi del Costo (I/O):

$$Cost = M + M \cdot N$$

Questo evidenzia come il join non sia un'operazione simmetrica a livello fisico: $M + M \cdot N$ è profondamente diverso da $N + N \cdot M$. L'ottimizzatore sceglierà sempre la relazione **con il minor numero di pagine** come relazione esterna (R).

Gestione del Buffer: Sono necessari almeno 3 frame di RAM: 1 per la pagina di R , 1 per la pagina di S , 1 per accumulare l'output. Per il rimpiazzamento delle pagine di S nel buffer, è fortemente raccomandata la politica **MRU (Most Recently Used)** anziché LRU, poiché si tratta di letture sequenziali ripetute di un file che supera le dimensioni del buffer.

9.7.3 Block Nested Loop Join (BNLJ)

Se disponiamo di più di 3 frame di buffer in RAM (diciamo B frame in totale), possiamo abbattere ulteriormente le letture della tabella interna. L'idea è allocare $B - 2$ frame per caricare un intero "blocco" della relazione esterna R , lasciare 1 frame per scansionare la relazione interna S , e 1 frame per l'output.

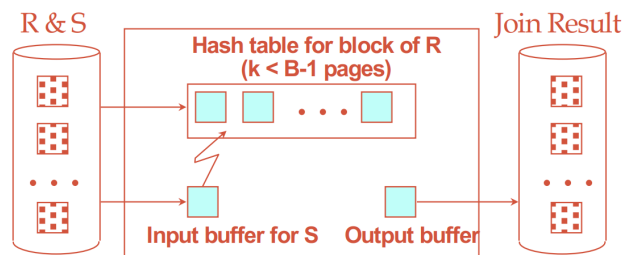


Figure 34: Visualizzazione del Block Nested Loops Join: blocchi in memoria per R confrontati con singole pagine di S

Per ottimizzare la ricerca all'interno della RAM, i record nel blocco di R (costituito da $B - 2$ pagine) vengono spesso organizzati in una **Hash Table in memoria**.

```
For each R-block (size B-2 pages):  
  Build in-memory hash table for R-block  
  For each page s in S:  
    Hash records in s and probe R-block hash table  
    If match, add <r, s> to result
```

Analisi del Costo (I/O): La relazione esterna viene letta tutta (M). La relazione interna viene letta per intero tante volte quanti sono i *blocchi* in cui è stata divisa R .

$$Cost = M + \left\lceil \frac{M}{B-2} \right\rceil \cdot N$$

9.7.4 Index Nested Loop Join

Se la condizione di join è supportata da un **indice** preesistente su una delle due tabelle, l'ottimizzatore forzerà quella tabella a essere la **relazione interna** (S). In questo modo, per ogni record di R , il DBMS non dovrà scansionare tutta S , ma farà una ricerca rapida nell'indice di S .

```
foreach tuple r in R do  
  Lookup r.i in index of S
```

```

foreach matching tuple s in S do
  add <r, s> to result

```

Fattibilità:

- *Indici Hash*: Utilizzabili solo se la condizione di join è di uguaglianza (*Equi-Join*).
- *B+ Tree*: Utilizzabili per uguaglianze e per range (*Theta-Join*, es. $>$, $<$).

Analisi del Costo (I/O):

$$Cost = M + ((M \cdot p_R) \cdot \text{costo_accesso_indice})$$

Il costo di accesso all'indice varia moltissimo se l'indice su S è **Clustered** (solitamente $\sim 1 - 2$ I/O per match) o **Unclustered** (fino a 1 I/O per ogni singola tupla matchante).

9.8 Algoritmi basati su Ordinamento e Partizionamento

9.8.1 Sort-Merge Join

Il Sort-Merge Join è estremamente efficiente, ma **funziona solo per join di uguaglianza**. Si compone di due fasi:

1. **Sort (Ordinamento)**: Entrambe le relazioni R e S vengono ordinate in base all'attributo di join (solitamente tramite External Merge Sort).
2. **Merge (Fusione)**: Si scansionano le due relazioni ordinate in parallelo, avanzando i puntatori come in una cerniera lampo.

Esempio di avanzamento dei puntatori senza duplicati:

$\rightarrow 1$	$0 \leftarrow$	$\rightarrow 1$	0	$\rightarrow 1$	0	$\rightarrow 1$	0	1	0
3	1	3	$1 \leftarrow$	3	1	3	1	$\rightarrow 3$	1
5	1	5	1	5	$1 \leftarrow$	5	1	5	1
6	6	6	6	6	6	6	$6 \leftarrow$	6	$6 \leftarrow$
7	7	7	7	7	7	7	7	7	7
9	8	9	8	9	8	9	8	9	8

Il problema del Backtracking (Duplicati): Se nell'attributo di join sono presenti duplicati in entrambe le relazioni, il semplice avanzamento lineare non è sufficiente. Quando si incrocia una sequenza di valori uguali su entrambi i lati, è necessario formare il prodotto cartesiano di quel sottoinsieme. Per farlo, il puntatore su una delle due relazioni deve "tornare indietro" (*backtracking*) finché l'altra non ha consumato tutti i suoi duplicati.

$\rightarrow 1$	$0 \leftarrow$	$\rightarrow 1$	0	$\rightarrow 1$	0	1	0	1	0
1	1	1	$1 \leftarrow$	1	1	$\rightarrow 1$	$1 \leftarrow$	$\rightarrow 1$	1
1	1	1	1	1	$1 \leftarrow$	1	1	1	$1 \leftarrow$
6	6	6	6	6	6	6	6	6	6
7	7	7	7	7	7	7	7	7	7
9	8	9	8	9	8	9	8	9	8

Caratteristiche: È un operatore **Blocking** (bloccante), poiché la fase di Merge non può iniziare finché l'intero dataset non è stato completamente ordinato.

Analisi del Costo (I/O): Il costo totale è dominato dalla fase di ordinamento. Se le tabelle non sono pre-ordinate:

$$Cost = M \log M + N \log N + (M + N)$$

(I fattori logaritmici dipendono dal numero di passaggi necessari per l'External Merge Sort, che a sua volta dipende dai buffer disponibili). Se le relazioni sono già ordinate (es. tramite indice B+ Tree Clustered sull'attributo di join), il costo crolla a soli $M + N$.

9.8.2 Hash-Join

Come il Sort-Merge, anche l'Hash Join si applica **esclusivamente ai join di uguaglianza**. Si basa sull'idea di partizionare i dati per garantire che le tuple che possono matchare finiscano nello stesso "secchio" (bucket), trasformando un grande problema globale in tanti piccoli problemi indipendenti.

Si compone di due fasi principali (spesso implementate come *Grace Hash Join*):

1. Fase di Partizionamento (Build): Si applica una funzione di hash h sull'attributo di join ad entrambe le relazioni R e S . I record vengono distribuiti su disco in k partizioni. A causa dell'uso della stessa funzione h , se una tupla di R e una tupla di S hanno lo stesso valore di join, verranno inevitabilmente inviate al medesimo bucket.

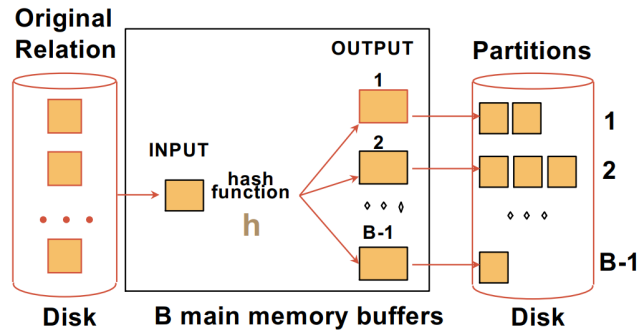


Figure 35: Visualizzazione dell'Hash Join: fase di partizionamento su disco tramite funzione h

2. Fase di Matching (Probing): A questo punto si elaborano le partizioni a coppie indipendenti. Si prende la partizione R_i e la si carica interamente in memoria RAM (costruendo una Hash Table in-memory per velocizzare la ricerca). Successivamente, si scansionano le pagine della corrispondente partizione S_i , una per volta, portandole nel buffer e verificando istantaneamente le uguaglianze sfruttando l'Hash Table in RAM.

Per mitigare ulteriormente i conflitti in RAM qualora un bucket di R sia molto affollato, si può applicare una *seconda* funzione di hash in memoria (h_2 , indipendente da h) per distribuire i record dei bucket all'interno della RAM.

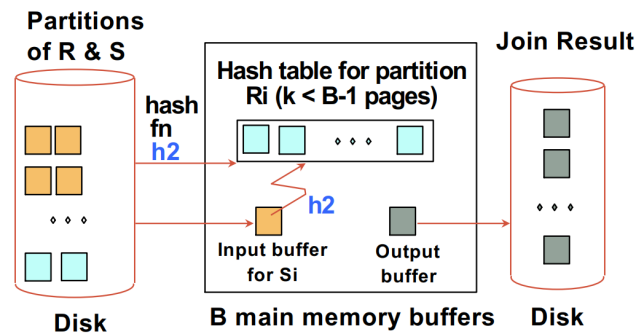


Figure 36: Visualizzazione dell'Hash Join: fase di probing in RAM con eventuale seconda funzione h_2

Analisi del Costo (I/O):

- **Fase 1 (Partizionamento):** Entrambe le relazioni vengono lette e poi scritte frammentate su disco. Costo: $2(M + N)$.
- **Fase 2 (Probing):** Si rileggono dal disco le partizioni a coppie per effettuare il join. Costo: $M + N$.

$$\text{Costo Totale} = 3(M + N)$$

L'algoritmo risulta estremamente efficiente e ha un costo lineare, a patto che il numero di buffer disponibili sia sufficiente per garantire che la partizione più grande di R_i entri completamente in RAM.

10 Ottimizzazione delle query

L'utente interagisce con il sistema relazionale tramite SQL, un linguaggio puramente **dichiarativo**: si specifica *quali* dati ottenere, senza indicare al DBMS *come* recuperarli. Poiché la stessa query può essere tradotta in un numero enorme di piani di esecuzione fisici (variando l'ordine degli operatori e gli algoritmi scelti), il compito del *Query Optimizer* è individuare la strategia più efficiente.

Tuttavia, la ricerca del piano di esecuzione assolutamente ottimo è un problema NP-completo, il cui costo di esplorazione crescerebbe esponenzialmente. Per questo motivo, l'ottimizzatore utilizza algoritmi euristici e statistiche per scartare rapidamente le soluzioni peggiori e individuare un piano **sub-ottimale** (sufficientemente buono) in tempi rapidi. Le statistiche a disposizione del DBMS (salvate nel catalogo) non sono sempre aggiornate in tempo reale per evitare un overhead eccessivo durante le transazioni, ma rappresentano stime affidabili.

Le fasi di ottimizzazione sono tre:

1. **Generazione dei piani logici**: La query dichiarativa viene convertita in un albero di espressioni in algebra relazionale. Il sistema esplora le diverse combinazioni e ordini degli operatori.
2. **Scelta del piano fisico e stima dei costi**: Ad ogni nodo dell'albero logico vengono associati i possibili algoritmi fisici (es. Hash Join, Table Scan). Il DBMS stima il costo di ciascuna alternativa.
3. **Selezione**: Viene scelto il piano con il costo stimato minore.

Il costo è valutato principalmente in termini di accessi I/O su disco (Page Faults) e dimensioni dei risultati intermedi generati. Le statistiche lette dal catalogo includono il numero di record per tabella, la dimensione delle pagine, i valori distinti degli attributi e la presenza di indici o ordinamenti.

```
SELECT t1.a, t2.b
FROM TABLE1 t1, TABLE2 t2
WHERE t1.a = t2.b AND t1.c > 10
```

Per questa semplice query, esistono molteplici espressioni relazionali equivalenti. Eccone alcune:

$$\Pi_{t1.a, t2.b} [\sigma_{t1.c > 10} (TABLE1 \bowtie_{t1.a = t2.b} TABLE2)]$$

$$\Pi_{t1.a, t2.b} [\sigma_{t1.a = t2.b \wedge t1.c > 10} (TABLE1 \times TABLE2)]$$

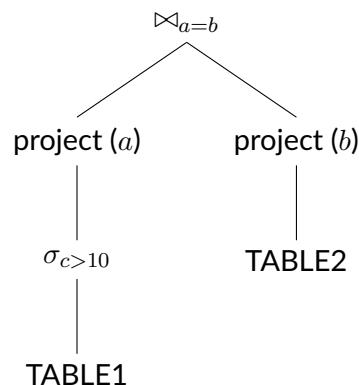
$$\Pi_{t1.a, t2.b} [\sigma_{t1.c > 10} (TABLE1) \bowtie_{a=b} TABLE2]$$

10.1 Alberi di esecuzione: Modello Logico e Modello Fisico

Per formalizzare il piano, l'ottimizzatore utilizza una struttura ad albero (*Query Tree*).

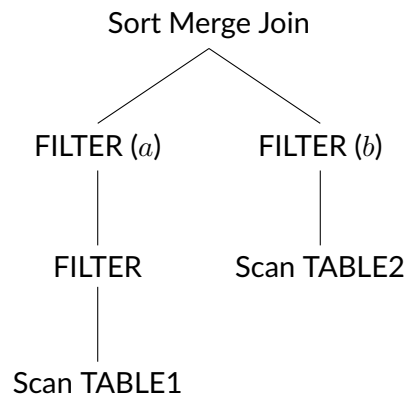
10.1.1 Modello logico

I nodi foglia rappresentano le relazioni di base (le tabelle), mentre i nodi intermedi rappresentano le operazioni algebriche. Non dice nulla sull'algoritmo utilizzato.



10.1.2 Modello fisico

La struttura è isomorfa a quella logica, ma i nodi sono etichettati con l'algoritmo esatto che il motore di esecuzione utilizzerà (es. *Sort-Merge Join*, *Index Scan*).



Nella valutazione fisica, gli operatori si dividono in:

- **Blocking (bloccanti):** Richiedono di leggere per intero il proprio input prima di poter produrre anche un solo record in output (es. ordinamento, GROUP BY).
- **Pipelined:** Elaborano una tupla per volta, passandola immediatamente all'operatore superiore, risparmiando letture e scritture su disco (es. selezioni, proiezioni).

10.2 Approcci all'Ottimizzazione

Ci sono due strategie complementari che l'ottimizzatore applica:

Ottimizzazione Euristica Agisce a livello logico applicando regole algebriche universalmente vantaggiose. L'idea di base è **ridurre al minimo il volume dei dati il prima possibile**. Si cerca di applicare le operazioni restrittive (Selezioni e Proiezioni) spingendole verso il basso dell'albero (*push-down*), in modo da eseguire i costosi Join su relazioni intermedie molto più piccole, a prescindere da quali siano i costi statistici esatti.

Ottimizzazione Basata sui Costi (Cost-Based) Agisce a livello fisico stimando matematicamente il costo delle varie combinazioni, utilizzando la cardinalità e i fattori di selettività calcolati tramite le statistiche a catalogo.

10.3 Modello basato sui costi (Stima)

Per stimare il costo I/O, il sistema valuta il volume dei risultati intermedi prodotti ad ogni nodo. L'ottimizzatore fa affidamento sui seguenti parametri statistici:

- $B(R)$: numero di blocchi fisici (pagine) della relazione R .
- $T(R)$: numero di tuple (record) nella relazione R .
- $V(R, a)$: numero di valori distinti dell'attributo a nella relazione R .

Stima della Scansione Il costo varia in base alle strutture dati:

- $B(R)$: scansione sequenziale di un file non ordinato (Heap File).
- $B(R)/\text{fattore_selettività}$: accesso tramite Indice Clustered (letture minime, dati ordinati).
- $T(R)/\text{fattore_selettività}$: peggior caso con Indice Unclustered, un I/O per ogni tupla.

Stima della Selezione ($S = \sigma(R)$) La selettività stima la frazione di tuple che soddisferà il predicato:

- $\sigma_{a=c}(R)$ (Uguaglianza): Assumendo una distribuzione uniforme, la cardinalità in output è $T(S) = \frac{T(R)}{V(R,a)}$.
- $\sigma_{a \neq c}(R)$ (Disuguaglianza): Solitamente restituisce quasi tutta la relazione. Spesso si approssima con $T(S) \approx T(R)$.
- $\sigma_{c_1 \vee c_2}(R)$ (OR logico): Sfruttando le leggi di De Morgan e assumendo indipendenza statistica tra i predicati, si stima: $T(S) = T(R) \cdot \left[1 - \left(1 - \frac{T(c_1)}{T(R)}\right) \cdot \left(1 - \frac{T(c_2)}{T(R)}\right)\right]$.

Stima del Join L'output di un Join ($R \bowtie S$) può variare drasticamente:

- 0: se la condizione non viene mai soddisfatta (es. domini disgiunti).
- $T(R)$: caso frequente di Equi-Join tra **Chiave Esterna / Chiave Primaria** (ogni tupla di R trova esattamente un match in S).
- $T(R) \cdot T(S)$: prodotto cartesiano (nessun filtro, costo massimo).
- $\frac{T(R)T(S)}{\max(V(R,y), V(S,y))}$: stima generica per equi-join sull'attributo y . Assume il principio di inclusione (i valori della tabella con meno elementi distinti sono inclusi in quella con più elementi distinti).

10.4 Principio di ottimalità delle sotto-query

Il calcolo efficiente dei piani si basa su un approccio di programmazione dinamica, giustificato dal **Principio di Ottimalità**: la dimensione dell'output di un operatore dipende esclusivamente dal suo input, indipendentemente dall'algoritmo fisico scelto.

Dunque, un piano globale ottimo è quasi sempre composto da sotto-piani locali ottimi.

Se $cost(Algoritmo_1) < cost(Algoritmo_2)$ per implementare una selezione, quella scelta resterà la migliore anche quando quella selezione diventerà l'input di un Join sovrastante, poiché il numero di tuple propagate verso l'alto sarà identico in entrambi i casi.

10.4.1 Esempio applicativo: Principio di Ottimalità delle Sotto-query

Il principio di ottimalità (derivato dalla programmazione dinamica) stabilisce che un piano di esecuzione globale ottimo è composto da sotto-piani locali ottimi. Di conseguenza, se due algoritmi fisici diversi producono lo **stesso identico risultato intermedio** (stesse tuple, stessa dimensione), l'ottimizzatore sceglierà l'algoritmo con il costo minore, garantendo così la minimizzazione del costo totale.

Si consideri la seguente query su una tabella *Impiegati* (es. 1.000.000 di record):

```
SELECT *  
FROM Impiegati  
WHERE Dipartimento = 'IT' AND Salario > 3000;
```

In algebra relazionale, l'ottimizzatore suddivide la selezione:

$$\sigma_{Salario > 3000}(\sigma_{Dipartimento = 'IT'}(Impiegati))$$

Concentrandosi sulla sotto-query interna $\sigma_{Dipartimento = 'IT'}(Impiegati)$, il DBMS valuta le possibili implementazioni fisiche:

- **Implementazione 1 (File Scan)**: Lettura sequenziale dell'intera tabella. Costo stimato elevato (es. 10.000 accessi I/O).
- **Implementazione 2 (Index Scan)**: Utilizzo di un indice B+ Tree preesistente sull'attributo *Dipartimento*. Costo stimato ridotto (es. 50 accessi I/O).

Entrambe le implementazioni estraggono esattamente il medesimo sottoinsieme di dati (es. 5.000 impiegati nel dipartimento IT). La dimensione del risultato intermedio, $size(R_{IT})$, è identica.

Il costo per l'applicazione del filtro esterno successivo, $\sigma_{Salario > 3000}$, dipende unicamente dalla dimensione dei dati in ingresso, ed è rappresentato dalla funzione $f(size(R_{IT}), \theta_{Salario})$.

Il calcolo del costo totale per i due piani risulta essere:

$$\begin{aligned} Cost^1 &= Cost(\text{File Scan}) + f(size(R_{IT}), \theta_{Salario}) \\ Cost^2 &= Cost(\text{Index Scan}) + f(size(R_{IT}), \theta_{Salario}) \end{aligned}$$

Poiché il termine $f(size(R_{IT}), \theta_{Salario})$ è costante in entrambe le equazioni, l'ottimizzatore lo elide dal confronto locale, limitandosi a valutare il costo delle singole implementazioni:

$$Cost(\text{Index Scan}) < Cost(\text{File Scan})$$

Scegliendo il sotto-piano locale ottimo (Index Scan), il DBMS ha la garanzia matematica che anche il piano globale risultante sarà quello ottimo, senza dover esplorare interamente l'albero per ogni variante.

Eccezione: Ordinamento Interessante Questo principio decade esclusivamente qualora un'implementazione locale più costosa in termini di I/O restituisca i dati **già ordinati** in modo utile per un operatore di livello superiore (es. per un *Sort-Merge Join* o per una clausola *ORDER BY*). In questo caso, il risultato intermedio, pur essendo logicamente equivalente nei contenuti, differisce nella struttura fisica, costringendo l'ottimizzatore a valutarne il beneficio sul costo dell'intero piano globale. **L'eccezione dell'"Ordinamento Interessante"**: L'unica deroga a questo principio si ha quando un sotto-piano sub-ottimale genera però un risultato *già ordinato* su un attributo che sarà richiesto dagli operatori successivi (es. dal nodo padre che esegue un *Sort-Merge Join*). In tal caso, il risparmio su un futuro ordinamento compensa il maggior costo iniziale.

```
SELECT *
FROM R, S, T
WHERE R.a = S.b AND S.c = T.d
```

Esempio di Programmazione Dinamica: Per risolvere la query a 3 tabelle mostrata sopra, l'ottimizzatore valuterà prima il piano migliore per le combinazioni a due tabelle ($R \bowtie S$, $S \bowtie T$, $R \bowtie T$). Se il metodo meno costoso per unire R ed S è un Hash Join, il motore congelerà questa scelta ottimale e la riutilizzerà quando cercherà il piano migliore per unire il risultato intermedio ($R \bowtie S$) con T .

Propagazione del costo e Ordinamenti Interessanti Nella valutazione dei piani di esecuzione tramite programmazione dinamica, il principio di ottimalità prevede tipicamente la conservazione esclusiva del sotto-piano locale a costo minimo. Tuttavia, per garantire l'individuazione del piano globale ottimo, gli ottimizzatori relazionali (a partire dal modello storico di *System R*) adottano una strategia più sofisticata per la propagazione dei costi. Per ogni nodo dell'albero logico (sotto-query), l'ottimizzatore non conserva una sola implementazione, bensì **due distinte implementazioni fisiche**:

1. **Piano a costo minimo assoluto:** L'algoritmo fisico che minimizza strettamente le operazioni di I/O e l'impiego di CPU per quel nodo specifico, indipendentemente dalla disposizione fisica finale delle tuple in output.
2. **Piano a costo minimo con "ordinamento interessante":** L'algoritmo sub-ottimale più economico tra quelli in grado di restituire un risultato **strutturalmente ordinato** su attributi che saranno presumibilmente richiesti dagli operatori di livello superiore.

Questa ridondanza è giustificata dal fatto che un ordinamento preventivo dei dati, pur richiedendo un maggiore costo computazionale o di I/O a livello locale (es. preferendo un *Index Scan* a un *Table Scan*), può abbattere drasticamente i costi di esecuzione nel resto dell'albero. Quando l'algoritmo valuta il nodo padre, calcola il costo combinato derivante dall'adozione di entrambe le strategie figlie, scegliendo e propagando la combinazione globalmente più efficiente.

Esempio applicativo sull'operatore DISTINCT

Si consideri un nodo superiore dell'albero logico incaricato di eliminare i duplicati (operatore algebrico δ , clausola SQL DISTINCT) su un attributo x . L'efficienza di questo operatore dipende interamente dalla natura del suo input:

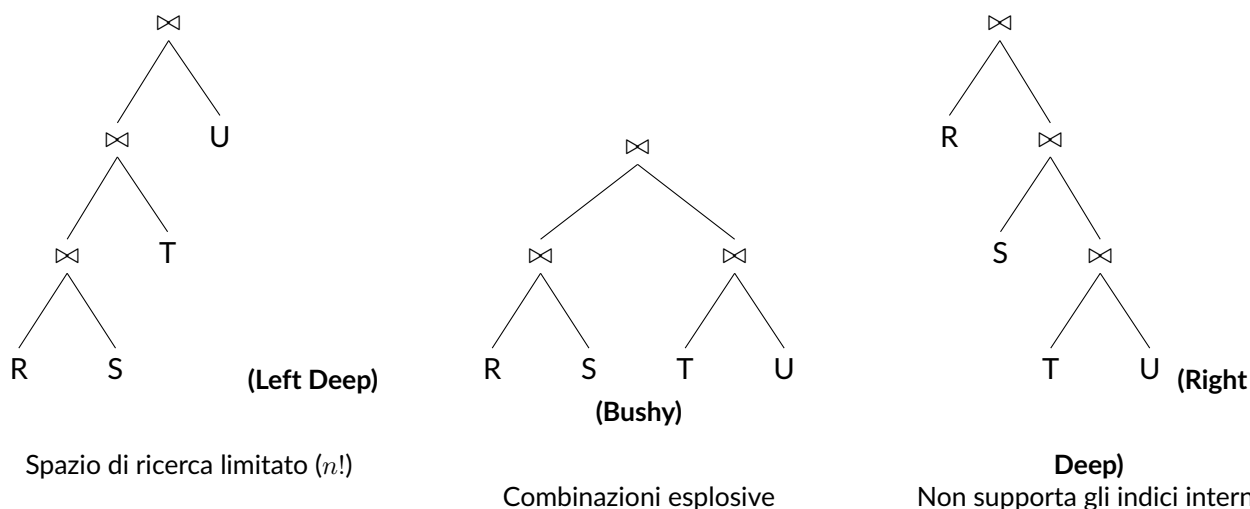
- Se l'operatore riceve il flusso dal **piano a costo minimo assoluto** (dati disordinati), il DBMS è costretto a eseguire un ordinamento esplicito (*Sort*) o a costruire una tabella Hash in memoria (*Hashing*) per raggruppare i valori uguali. Si tratta di operazioni *blocking* ad alto costo (es. $3 \cdot B(R)$ accessi I/O in caso di ordinamento esterno).
- Se l'operatore riceve il flusso dal **piano con ordinamento interessante** (dati già ordinati sull'attributo x), l'eliminazione dei duplicati si trasforma in un'operazione *pipelined* estremamente economica. È sufficiente confrontare ogni tupla con la precedente scartando i match adiacenti, con un costo addizionale pari a zero I/O aggiuntivi.

L'ottimizzatore valuterà se il risparmio ottenuto nel nodo padre compensa il maggior costo iniziale dell'ordinamento. In caso affermativo, l'eccezione dell'ordinamento interessante prevarrà sulla scelta locale strettamente meno costosa.

10.5 Ordine di Valutazione dei Join

Quando la query coinvolge più relazioni (es. $R \bowtie S \bowtie T \bowtie U$), l'ordine in cui vengono aggregati fa una differenza sostanziale nei costi, sia per la dimensione dei risultati intermedi che per le implementazioni fisiche possibili.

Esistono tre principali tipologie di alberi logici per i Join: Left-Deep, Bushy e Right-Deep.



Nella stragrande maggioranza dei casi, l'ottimizzatore **riduce lo spazio di ricerca ai soli Left-Deep Trees**. Sebbene trascurare le configurazioni Bushy precluda potenziali piani paralleli eccellenti, imporre la forma Left-Deep offre vantaggi decisivi dal punto di vista fisico:

1. **Massimizzazione dell'uso degli Indici:** In un Left-Deep Tree, l'argomento destro (la relazione "interna") di *ogni* join è sempre una tabella base originale. Se quella tabella possiede un indice sull'attributo di Join, l'ottimizzatore può sfruttarlo per applicare il performante algoritmo *Index Nested Loop Join*. Negli alberi Right-Deep, l'argomento interno è invece il risultato calcolato da un operatore precedente: essendo una tabella generata al volo, non possiede indici.
2. **Gestione della Memoria e Pipelining:** Molti algoritmi di iterazione (es. Nested Loop o la fase di Probe dell'Hash Join) leggono un record alla volta dalla relazione esterna (ramo sinistro) per confrontarlo con quella interna (ramo destro). In un Left-Deep Tree, i record generati dal primo join in basso fluiscono verso l'alto ed entrano subito in parallelo (in *pipeline*) nel ramo sinistro del join di livello superiore, senza dover essere scritti su disco. Questo minimizza i Page Faults e mantiene i requisiti di memoria bassi e costanti.

Una volta ristretto lo spazio di ricerca ai soli Left-Deep Trees, l'ottimizzatore si trova di fronte alla necessità di valutare le $n!$ permutazioni possibili per una query che coinvolge n relazioni. Il problema centrale diventa quindi individuare una strategia algoritmica efficiente per esplorare questo spazio e selezionare il piano globale a costo minimo.

10.6 Approccio Ricorsivo (Top-Down)

Sia data una query costituita dal join di n relazioni: $Q = \{R_1, R_2, \dots, R_n\}$. Un approccio naturale per esplorare le combinazioni è l'utilizzo della **ricorsione** (ricerca *top-down*). L'algoritmo procede valutando, per ogni relazione $R_i \in Q$, il costo di eseguire prima il join ottimo di tutte le altre relazioni (ossia $Q \setminus \{R_i\}$), per poi unire il risultato intermedio proprio con R_i .

Formalmente, il costo per risolvere l'insieme Q scegliendo R_i come relazione da aggregare per ultima è definibile come:

$$Cost(Q, R_i) = Cost(Ottimo(Q \setminus \{R_i\})) + Cost-Join(Q \setminus \{R_i\}, R_i)$$

Il piano ottimo globale per Q sarà quello che minimizza questo valore esplorando tutte le possibili relazioni R_i .

10.6.1 Il collo di bottiglia della ricorsione ingenua

Il limite critico di una ricorsione pura risiede nell'assenza di memoria strutturale, che porta al problema dei **sottoproblemi sovrapposti** (*overlapping subproblems*). Analogamente a quanto accade nell'implementazione ricorsiva ingenua per il calcolo della sequenza di Fibonacci, l'algoritmo esplora e ricalcola ripetutamente le stesse identiche sotto-query.

Ad esempio, il piano ottimo per il sotto-insieme $\{R_1, R_2, R_3\}$ verrebbe ricalcolato innumerevoli volte in rami differenti dell'albero di esecuzione. Ciò genera un'esplosione combinatoria esponenziale del lavoro ridondante e un forte consumo di memoria (stack delle chiamate) per il mantenimento degli ambienti di esecuzione aperti. Una soluzione teorica consisterebbe nell'introdurre una *Look-up Table* (tecnica di *memoization*) per salvare i risultati intermedi appena calcolati, ma nei DBMS relazionali si preferisce storicamente capovolgere la direzione di calcolo.

10.7 Programmazione Dinamica (Bottom-Up)

Per risolvere le inefficienze computazionali della ricorsione, gli ottimizzatori moderni (a partire dall'architettura storica di *System R*) adottano la **Programmazione Dinamica**. Questo paradigma opera secondo una logica *bottom-up*: costruisce, memorizza e riutilizza sistematicamente i piani ottimi per sotto-insiemi di relazioni di dimensioni crescenti. In questo modo, il DBMS garantisce matematicamente che ogni sotto-problema venga calcolato **esattamente una volta sola**.

Il procedimento algoritmico si struttura in fasi basate sul numero di relazioni coinvolte (k):

1. **Fase $k = 1$:** Si determina il metodo di accesso ottimale (e l'eventuale sotto-piano con ordinamento interessante) per leggere ogni singola relazione di base R_i (es. *File Scan* vs *Index Scan*).
2. **Fase $k = 2$:** Si combinano le relazioni a due a due sfruttando le condizioni di Join. Per ogni coppia (R_i, R_j) , si calcola il costo di $R_i \bowtie R_j$ e $R_j \bowtie R_i$, memorizzando per quel set di dati solo il piano fisico a costo minimo.
3. **Fasi successive ($k = i$):** Per generare i piani ottimali di dimensione i , l'algoritmo non riparte da zero, ma preleva i risultati ottimi di dimensione $i - 1$ (già memorizzati) e vi aggiunge una singola relazione rimasta (mantenendo rigorosamente la forma Left-Deep). Si valutano i costi delle nuove combinazioni e si congela nuovamente solo la migliore per ogni specifico sotto-insieme di i tabelle.
4. **Terminazione ($k = n$):** L'iterazione si conclude quando si calcola l'aggregazione di tutte le n tabelle, restituendo il piano globale ottimo.

Questo approccio riduce drasticamente i tempi di ottimizzazione (passando da una complessità di $O(n!)$ a circa $O(2^n)$, o persino meno se il grafo dei Join è denso di vincoli). Va sottolineato che l'impianto logico della programmazione dinamica è **indipendente dalla topologia dell'albero**: sebbene sia comunemente vincolato alla costruzione di Left-Deep Trees per i vantaggi fisici già illustrati, lo stesso algoritmo può essere esteso per includere nello spazio di ricerca anche i Right-Deep e i Bushy Trees, alterando semplicemente le logiche combinatorie nelle fasi $k > 2$.

11 Concorrenza e Transazioni

La gestione della concorrenza è un aspetto critico nei DBMS (Database Management System). L'esecuzione simultanea di più operazioni da parte di utenti diversi migliora drasticamente il throughput e i tempi di risposta del sistema, ma rischia di compromettere la correttezza dei dati se le operazioni si sovrappongono in modo incontrollato.

11.1 Il concetto di Transazione

L'unità logica di elaborazione in un DBMS non è la singola query, ma la **transazione**. Una transazione è una sequenza finita di operazioni di lettura e scrittura che, dal punto di vista logico e applicativo, costituisce un'unità di lavoro **indivisibile** (atomica).

Il ciclo di vita di una transazione prevede due soli esiti possibili:

- **Successo (Commit):** Tutte le operazioni della transazione sono andate a buon fine. Le modifiche vengono rese definitive e visibili agli altri utenti.
- **Fallimento (Abort / Rollback):** Si è verificato un errore (es. crash di sistema, violazione di vincoli, stallo/deadlock) o l'utente ha interrotto l'operazione. Il DBMS deve garantire che nessuna delle operazioni parziali venga applicata, effettuando un *undo* per riportare il database allo stato originale.

L'obiettivo fondamentale è garantire che il database transiti sempre da uno **stato consistente** iniziale a un nuovo **stato consistente** finale, nascondendo le inconsistenze temporanee che si verificano durante l'esecuzione della transazione stessa.

11.2 Le Proprietà ACID

Per garantire la correttezza del database, il DBMS assicura che ogni transazione rispetti il paradigma ACID. Il **Recovery Manager** (gestore del recupero) si occupa di Atomicità e Durabilità, mentre il **Concurrency Control Manager** (gestore della concorrenza) garantisce l'isolamento.

Atomicity (Atomicità): *"Tutto o niente"*. Le azioni di una transazione non possono essere eseguite parzialmente. In caso di fallimento in qualsiasi punto dell'esecuzione, il sistema annulla tutte le modifiche apportate fino a quel momento (tramite i file di *Log*), ripristinando lo stato precedente all'inizio della transazione.

Consistency (Consistenza): L'esecuzione isolata di una transazione deve preservare la validità semantica dei dati. Se il database rispetta tutti i vincoli di integrità prima dell'inizio della transazione, dovrà rispettarli rigorosamente anche al termine (sia in caso di Commit che di Abort).

Isolation (Isolamento): L'esecuzione concorrente di più transazioni deve produrre un risultato identico a quello che si otterrebbe eseguendole in modo strettamente sequenziale (una dopo l'altra). Una transazione in corso non deve subire interferenze né vedere i risultati parziali di altre transazioni non ancora concluse.

Durability (Durabilità): Una volta che una transazione termina con un *Commit*, le modifiche apportate ai dati diventano permanenti e sopravvivranno a qualsiasi malfunzionamento hardware o software successivo (garantito dalla scrittura sincrona dei *Log* su memoria non volatile).

11.3 Esecuzione Concorrente e Anomalie

Per massimizzare l'efficienza, il DBMS **interfoglia** (interleaves) le operazioni di I/O (Read e Write) di transazioni diverse. L'ordine cronologico di esecuzione di queste operazioni è detto **Schedule** (o storia).

Si consideri il seguente esempio con due transazioni applicate a due conti bancari *A* e *B*:

$$T_1 : \text{BEGIN } A = A + 100; B = B - 100 \text{ END}$$
$$T_2 : \text{BEGIN } A = 1.06 \cdot A; B = 1.06 \cdot B \text{ END}$$

L'esecuzione corretta impone che il risultato finale sia equivalente a T_1 seguita da T_2 , oppure a T_2 seguita da T_1 . Qualsiasi interleaving che produca un risultato ibrido è considerato errato.

11.3.1 Esempi di Schedule

Valutiamo due possibili interfogliamenti, rappresentando le operazioni come istruzioni generiche per chiarezza.

Schedule Seriale (Corretto per definizione):

T_1 :	$A = A + 100,$	$B = B - 100$
T_2 :	$A = 1.06 \cdot A$	$B = 1.06 \cdot B$

Questo schedule è corretto poiché T_1 termina completamente prima che T_2 inizi ad operare sulle stesse variabili.

Schedule Interfogliato Inconsistente (Errato):

T_1 :	$A = A + 100,$	$B = B - 100$
T_2 :	$A = 1.06 \cdot A$	$B = 1.06 \cdot B$

Questo schedule produce uno stato inconsistente. T_2 applica l'interesse del 6% al conto A *dopo* l'accredito di 100, ma lo applica al conto B *prima* dell'addebito di 100. Il risultato non equivale a nessuna delle due esecuzioni sequenziali possibili.

A livello formale, l'ottimizzatore astrae queste operazioni studiando unicamente le operazioni di **Read (R)** e **Write (W)** sugli item (es. A e B).

T_1 :	$R(A), W(A)$	$R(B), W(B)$
T_2 :	$R(A), W(A), R(B), W(B)$	

11.4 Conflitti e Anomalie di Concorrenza

Si ha un **conflitto** quando due transazioni diverse accedono allo stesso dato simultaneamente e almeno una delle due esegue un'operazione di scrittura. Si distinguono tre pattern anomali principali:

11.4.1 Conflitto W-R: Lettura Sporca (Dirty Read)

Si verifica quando una transazione T_2 legge un dato che è stato appena modificato da una transazione T_1 la quale **non ha ancora effettuato il commit**. Se T_1 dovesse fallire e subire un *Abort*, il valore letto da T_2 cesserebbe di esistere formalmente: T_2 avrebbe basato i propri calcoli su un dato "sporco" e inconsistente.

T_1 :	$R(A), W(A)$	$R(B), W(B) \rightarrow \text{Abort}$
T_2 :	$R(A), W(A) \rightarrow \text{Commit}$	

In questo scenario, T_2 ha propagato nel sistema un valore calcolato a partire dall'aggiornamento temporaneo di A fatto da T_1 , causando un'inconsistenza permanente.

11.4.2 Conflitto R-W: Lettura Non Ripetibile (Unrepeatable Read) o Update Fantasma

Si verifica quando una transazione T_1 legge un dato due volte durante la sua esecuzione, ma ottiene due valori diversi perché, nel frattempo, una transazione concorrente T_2 ha modificato e committato quel dato.

T_1 :	$R(A)$	$R(A), W(A) \rightarrow \text{Commit}$
T_2 :	$R(A), W(A) \rightarrow \text{Commit}$	

Un corollario pratico di questa anomalia è la **violazione della logica di controllo**. Si supponga $A = -1$:

T1: if $A < 0$ then $A = A + 7$
T2: $A = A + 5$

Se T_1 valuta la condizione ($A < 0 \rightarrow \text{Vero}$) ma, subito prima di eseguire l'aggiornamento, T_2 modifica A portandolo a 4, T_1 eseguirà $A = 4 + 7 = 11$, applicando una logica su un presupposto che non è più valido, corrompendo la semantica dell'operazione.

11.4.3 Conflitto W-W: Scrittura Cieca (Lost Update / Overwriting Uncommitted Data)

Si verifica quando una transazione T_2 sovrascrive il valore di un item che era stato precedentemente modificato da T_1 , prima che T_1 abbia terminato. Questo può portare alla **perdita di aggiornamenti** o a stati misti incoerenti con qualsiasi esecuzione seriale.

$T_1:$	$W(A)$	$W(B) \rightarrow \text{Commit}$
$T_2:$	$W(A), W(B) \rightarrow \text{Commit}$	

In questo caso, il valore finale di A rifletterà le modifiche di T_2 , mentre il valore finale di B rifletterà quelle di T_1 (o viceversa), rompendo i vincoli interni tra A e B garantiti dall'isolamento.

11.5 Teoria dello Scheduling e Serializzabilità

Per evitare matematicamente tutte queste anomalie, il DBMS deve accettare solo determinati *schedules*.

Schedule Seriale Uno schedule si dice seriale se le transazioni non sono interfogliate in alcun modo. Le operazioni di T_1 vengono eseguite interamente dall'inizio alla fine, seguite da quelle di T_2 (o viceversa). Garantisce l'assenza di conflitti per definizione, ma offre prestazioni pessime poiché non sfrutta il parallelismo.

Equivalenza tra Schedule Due schedule S_1 e S_2 sono considerati **equivalenti** (nello specifico, *conflict-equivalent*) se l'ordine di ogni coppia di operazioni in conflitto (appartenenti a transazioni diverse) è identico in entrambi gli schedule. In sintesi, se due schedule processano i conflitti nello stesso ordine, produrranno il medesimo stato finale del database.

Schedule Serializzabile È il compromesso ottimale ricercato dal DBMS: uno schedule interfogliato si dice **serializzabile** se e solo se è equivalente a *almeno uno* degli schedule seriali possibili con le stesse transazioni. Se uno schedule è serializzabile, il DBMS sa di poterlo eseguire in parallelo garantendo contemporaneamente le alte prestazioni della concorrenza e l'integrità logica garantita dall'esecuzione seriale. Il controllo di serializzabilità viene solitamente effettuato in tempo reale tramite protocolli di *Locking* (es. Two-Phase Locking, 2PL).

11.5.1 Schedule Conflict Serializable (Serializzabilità basata sui conflitti)

La serializzabilità basata sui conflitti (Conflict Serializability) è un criterio fondamentale per stabilire la correttezza di un'esecuzione concorrente. Due schedule si definiscono **conflict equivalent** (equivalenti nei conflitti) se:

- Contengono esattamente lo stesso insieme di operazioni (Read e Write) appartenenti alle stesse transazioni.
- L'ordine cronologico di esecuzione per *ogni singola coppia di operazioni in conflitto* rimane inalterato in entrambi gli schedule.

Uno schedule S è **conflict serializable** se e solo se è conflict equivalent a un qualsiasi schedule seriale composto dalle medesime transazioni.

Essere conflict serializable è una condizione **sufficiente ma non necessaria** per garantire la consistenza del database. Esistono infatti schedule che producono risultati corretti pur non essendo conflict serializable (rientrano nella classe più ampia della *View Serializability*). Tuttavia, verificare la View Serializability è un problema computazionalmente intrattabile (NP-completo). Per questo motivo, i DBMS moderni limitano l'attenzione alla Conflict Serializability: essa garantisce la correttezza, permette un elevato throughput e parallelismo, e può essere imposta in tempo reale con algoritmi efficienti e poco onerosi.

Esempio di schedule NON Conflict Serializable

T_1 :	$R(A), W(A)$	$R(B), W(B)$
T_2 :	$R(A), W(A), R(B), W(B)$	

In questo scenario si verificano due conflitti di tipo R-W (o W-W). Sul dato A , l'ordine del conflitto impone che T_1 preceda T_2 ($T_1 \rightarrow T_2$). Sul dato B , tuttavia, T_2 opera prima di T_1 , imponendo l'ordine inverso ($T_2 \rightarrow T_1$). Questa contraddizione rende lo schedule non equivalente ad alcun ordine seriale (T_1 seguita da T_2 , oppure T_2 seguita da T_1), portando a un'esecuzione errata.

Il Grafo delle Dipendenze (Precedence Graph) Per verificare formalmente la serializzabilità sui conflitti si costruisce il **Grafo delle Dipendenze**. È un grafo orientato in cui:

- I **Nodi** rappresentano le transazioni in esecuzione ($T_1, T_2, \dots, T_n$).
- Gli **Archì** (diretti da T_i a T_j) indicano che esiste almeno un conflitto tra un'operazione di T_i e una di T_j , e che l'operazione di T_i è stata eseguita temporalmente prima.

La regola aurea afferma che: **Uno schedule è conflict serializable se e solo se il suo grafo delle dipendenze è aciclico (privo di cicli)**. Costruire e analizzare questo grafo a runtime è costoso. Pertanto, i DBMS non controllano a posteriori la presenza di cicli, ma utilizzano protocolli (come il Locking) per **prevenirne** la formazione in modo strutturale.

11.5.2 Gestione della Concorrenza basata sul Locking

Il **Lock** è un primitivo di sincronizzazione che regola l'accesso in mutua esclusione (o condiviso) a una risorsa. Richiedere un lock significa imporre un'attesa ad altre transazioni potenzialmente in conflitto, ritardandole ed evitando così l'interfogliamento anomalo.

Esistono due modalità principali di Lock:

- **Lock Condiviso (Shared - S):** Richiesto per le operazioni di **Lettura**. Più transazioni possono detenere contemporaneamente un lock shared sullo stesso oggetto. Viene concesso se l'oggetto è libero o se vi sono solo altri lock shared attivi.
- **Lock Esclusivo (Exclusive - X):** Richiesto per le operazioni di **Scrittura** (modifica/cancellazione). Garantisce mutua esclusione assoluta: viene concesso solo se nessun'altra transazione detiene un lock (né S né X) su quell'oggetto.

Strict Two-Phase Locking (Strict 2PL) È il protocollo di controllo della concorrenza più utilizzato. Si basa su due fasi rigorose:

1. **Fase di Espansione (Growing Phase):** La transazione acquisisce tutti i lock necessari (S o X) man mano che procede nell'esecuzione. Non può rilasciare alcun lock.
2. **Fase di Restringimento (Shrinking Phase) - Modalità Strict:** La transazione **non rilascia alcun lock** finché non giunge alla sua conclusione (Commit o Abort). Al termine, rilascia tutti i lock simultaneamente.

Vantaggi: Garantisce matematicamente che il grafo delle dipendenze sia aciclico (assicurando la serializzabilità) e *previene totalmente gli aborti a cascata*. **Svantaggi:** Riduce sensibilmente il grado di concorrenza del sistema, poiché i lock esclusivi vengono trattenuti per un tempo maggiore rispetto allo stretto necessario.

Two-Phase Locking non strict (2PL Base) Simile allo Strict 2PL, mantiene l'obbligo delle due fasi: una volta che la transazione rilascia il suo *primo* lock, entra nella fase di restringimento e **non può più acquisirne di nuovi**. A differenza della variante Strict, permette di rilasciare i lock sui singoli dati non appena l'operazione su di essi è conclusa, prima del termine della transazione.

Ciò aumenta la concorrenza, ma espone al rischio di **Aborti a Cascata** (Cascading Rollbacks).

T_1 :	$W(A), \text{Unlock}(A) \dots \rightarrow \text{Abort}$
T_2 :	$\text{Lock}(A), R(A), W(A) \rightarrow \text{Commit}$

Se T_1 modifica A , rilascia il lock e successivamente fallisce, T_2 potrebbe nel frattempo aver letto il valore "sporco" di A e aver committato. Per preservare l'Atomicità, il sistema dovrebbe annullare anche T_2 (che ha basato i suoi calcoli su un dato inconsistente), generando una reazione a catena disastrosa per le performance.

11.5.3 Deadlock (Stallo)

Il protocollo 2PL genera **Deadlock**: situazioni di blocco circolare in cui due o più transazioni attendono reciprocamente il rilascio di un lock detenuto dall'altra, impedendo a entrambe di procedere. I DBMS adottano due filosofie per gestire i deadlock:

1. Rilevamento (Detection): Consiste nel riconoscere i deadlock dopo che si sono formati. Si può utilizzare il **Grafo delle Attese** (Wait-for Graph), dove i nodi sono le transazioni e gli archi indicano chi aspetta chi. Un ciclo nel grafo certifica il deadlock. Poiché l'analisi dei grafi è onerosa, nella pratica i DBMS implementano meccanismi di **Timeout**: se una transazione attende un lock per un tempo superiore a una soglia prefissata, si assume l'esistenza di un deadlock e la transazione viene forzatamente interrotta (Abort) e riavviata.

2. Prevenzione (Prevention): Si utilizzano i **Timestamp** (marche temporali associate all'istante d'inizio della transazione) per assegnare una priorità: transazioni più vecchie hanno priorità maggiore. Quando $T_{richiedente}$ chiede un lock detenuto da $T_{detentore}$, il DBMS valuta due possibili politiche:

- **Wait-Die (Non-preemptive):** Se $T_{richiedente}$ è più vecchia (priorità alta), viene messa in attesa (Wait). Se è più giovane (priorità bassa), viene "uccisa" (Die - Abort) e riavviata.
- **Wound-Wait (Preemptive):** Se $T_{richiedente}$ è più vecchia, "ferisce" il detentore costringendolo all'aborto (Wound) e rubando il lock. Se è più giovane, si mette in attesa (Wait).

In entrambi i casi, per evitare fenomeni di *starvation*, una transazione che subisce un aborto per deadlock e viene riavviata **conserva il suo timestamp originale**, diventando progressivamente la transazione più vecchia e garantendosi l'esecuzione.

11.5.4 Locking con Multigranularità (MGL)

Un database è organizzato in modo gerarchico:

Database $\rightarrow$ Tabella $\rightarrow$ Pagina (Blocco) $\rightarrow$ Record (Tupla) $\rightarrow$ Campo

Il Concurrency Control Manager deve bilanciare due esigenze:

- **Granularità Fine (es. Record):** Alta concorrenza (molti utenti accedono alla stessa tabella contemporaneamente), ma alto overhead gestionale (milioni di lock da tracciare).
- **Granularità Grossolana (es. Tabella):** Basso overhead gestionale, ma concorrenza nulla (un'intera tabella bloccata per la modifica di una sola riga).

Acquisire un lock a un livello (es. Tabella) blocca implicitamente tutti i suoi discendenti (Pagina, Record). Per risolvere il problema del conflitto tra granularità diverse (es. T_1 vuole leggere l'intera Tabella, mentre T_2 sta modificando un singolo Record all'interno), si introducono i **Lock Intensionali**.

Lock Intensionali (IS, IX) Prima di richiedere un lock (S o X) su un nodo foglia (es. un record), la transazione deve "scendere" lungo la gerarchia acquisendo lock intensionali su tutti gli antenati del nodo. L'intensionale funge da "avviso": indica ai livelli superiori che la transazione sta operando a un livello inferiore.

- **IS (Intention Shared):** Si intende richiedere lock condivisi sui nodi discendenti.
- **IX (Intention Exclusive):** Si intende richiedere lock esclusivi (e potenzialmente condivisi) sui nodi discendenti.

	-	IS	IX	S	X
-	✓	✓	✓	✓	✓
IS	✓	✓	✓	✓	
IX	✓	✓	✓		
S	✓	✓		✓	
X	✓				

Table 2: Matrice di Compatibilità per la Multigranularità (Righe: Lock esistente, Colonne: Lock richiesto)

Tabella di Compatibilità e Gestione dei Casi L'utilità di questo approccio è evidente analizzando l'interazione tra i vari lock allo **stesso livello gerarchico** (es. sulla Tabella):

Caso IS + IS: Compatibile. Due transazioni dichiarano di voler leggere record di livello inferiore. Possono operare contemporaneamente; chiederanno lock (S) solo sui record specifici.

Caso IS + IX (e viceversa): Compatibile. Una transazione vuole leggere alcuni record (IS) e un'altra vuole scriverne di nuovi o modificarne altri (IX). Essendo un avviso a livello di tabella, la compatibilità permette l'accesso concorrente. Eventuali conflitti reali (es. modifica dello stesso record letto dall'altro) saranno gestiti a livello di tupla, dove i lock S e X si scontreranno.

Caso IX + IX: Compatibile. Molteplici transazioni dichiarano di voler scrivere a livello inferiore (es. inserimento di righe diverse). Il parallelismo in scrittura sulla tabella è massimizzato; i conflitti esploderanno solo in caso di operazioni sulla medesima tupla.

Caso IS + S (e viceversa): Compatibile. Una transazione vuole leggere l'intera tabella (S), l'altra vuole leggere solo un record specifico (IS). Operando in sola lettura, i dati rimangono invariati.

Caso IX + S (e viceversa): Incompatibile. Non si può acquisire un lock in lettura sull'intera tabella (S) se un'altra transazione sta modificando un suo record interno (IX). Si leggerebbero dati inconsistenti o non committati (violazione dell'Isolamento). Viceversa, se qualcuno sta leggendo la tabella (S), nessuna operazione interna può subire modifiche (IX).

Caso X con qualsiasi altro (IS, IX, S, X): Incompatibile. Se una transazione detiene un lock X sulla tabella (es. 'DROP TABLE' o 'ALTER TABLE'), nessuno può né leggere né scrivere a nessun livello. Se vi sono lock attivi, il lock X sulla tabella deve attendere che tutti vengano rilasciati.

Per garantire la correttezza del sistema, l'acquisizione dei lock (a partire da quelli intensionali) deve seguire un percorso gerarchico rigorosamente *top-down*, mentre il loro rilascio deve essere effettuato in ordine *bottom-up*.

11.5.5 Database Dinamici e il Problema dei Dati Fantasma (Phantom Reads)

Non tutti i sistemi affrontano automaticamente le anomalie legate all'inserimento dinamico dei dati. Un problema classico si verifica con gli aggiornamenti o le letture di tipo **dichiarativo** (es. interrogazioni basate su condizioni 'WHERE').

Se una transazione T_1 richiede: "Per ogni studente con voto uguale a 29, aggiorna il voto a 30", il DBMS individua i record esistenti che soddisfano la condizione, applica i lock necessari e procede alla modifica. Tuttavia, se prima del commit di T_1 , una transazione concorrente T_2 inserisce un **nuovo studente** (o modifica uno studente esistente) portando il suo voto a 29, questo nuovo record sfugge al controllo di T_1 .

Questi record aggiuntivi, che compaiono o scompaiono in modo imprevisto durante l'esecuzione di una transazione, prendono il nome di **Dati Fantasma** (Phantom Records). Al termine di T_1 , la condizione della query risulterà violata: esisterà ancora uno studente con voto 29, contravvenendo alla logica della transazione.

Strategie di Risoluzione

- **Re-esecuzione iterativa:** Alcuni DBMS, prima di effettuare il commit, verificano se sono comparsi dati fantasma che soddisfano i predicati delle query eseguite. Se presenti, la transazione applica nuovamente la logica su questi nuovi dati, iterando finché lo stato non è stabile. (Approccio raro e costoso).
- **Index Locking (o Next-Key Locking):** È la soluzione più diffusa. Se il campo della condizione (es. voto) è indicizzato, il DBMS non mette il lock solo sui record esistenti che valgono 29, ma blocca l'**intervallo logico** nell'indice associato a quel valore. In questo modo, qualsiasi transazione T_2 che tenti di inserire una nuova voce nell'indice con valore 29 verrà messa in attesa, prevenendo la nascita di dati fantasma.
- **Ignorare il problema:** Ai livelli di isolamento inferiori a *Serializable* (come *Read Committed* o *Repeatable Read*), molti DBMS tollerano nativamente le letture fantasma per favorire le prestazioni.

11.5.6 Gestione della Concorrenza sugli Indici Gerarchici

Ci concentriamo su indici basati su alberi (come i **B+ Tree**), che possiedono stringenti requisiti strutturali: ogni nodo (pagina) deve avere un fattore di riempimento minimo (tipicamente 50%). L'inserimento o la cancellazione di un record in una foglia può violare questi vincoli, innescando rispettivamente operazioni di **Split** (sdoppiamento del nodo) o **Merge** (fusione dei nodi).

Queste operazioni strutturali possono propagarsi **a cascata** verso l'alto (verso gli antenati o la radice). Pertanto, la modifica di una foglia rischia di alterare la struttura dei livelli superiori.

Perché non usare il 2PL Standard? Applicare il protocollo *Strict 2PL* a un B+ Tree avrebbe effetti disastrosi. Poiché ogni operazione deve necessariamente attraversare la radice per scendere lungo l'albero, il 2PL imporrebbe di acquisire un lock sulla radice e mantenerlo fino al termine della transazione. Questo serializzerebbe di fatto tutti gli accessi all'indice, azzerando la concorrenza.

Si ricorre quindi a tecniche di locking specifiche, dette di **Lock Coupling** (o *Crabbing*), basate sul concetto di nodo sicuro. **Definizione di Nodo Safe (Sicuro):** Un nodo è "safe" se è garantito che non subirà split né merge in seguito all'operazione in corso (es. per un inserimento, ha meno del 100% di riempimento; per una cancellazione, ha più del 50% di riempimento). Se un nodo è safe, eventuali modifiche ai suoi discendenti non si propagheranno al di sopra di esso.

Si distinguono tre protocolli principali:

Algoritmo 1 (Pessimistico / Cautivo):

- **Ricerca (Lettura):** Si parte dalla radice acquisendo un lock condiviso (S). Si scende al figlio acquisendo un lock S e rilasciando il lock S sul padre (*hand-over-hand locking*).
- **Inserimento/Cancellazione:** Si parte dalla radice acquisendo un lock esclusivo (X). Scendendo, si acquisisce un lock X sul nodo figlio. Se il figlio è **safe**, si è certi che nessuna modifica risalirà oltre; pertanto, **si rilasciano immediatamente i lock X su tutti gli antenati**. Si procede così fino alla foglia.

Algoritmo 2 (Ottimistico): L'Algoritmo 1 pone lock X sui livelli alti, bloccando inutilmente i lettori. Poiché i nodi "non safe" sono statisticamente rari, si sfrutta un approccio ottimistico.

- **Inserimento/Cancellazione:** La discesa avviene esattamente come nella fase di Ricerca (usando solo lock S e rilasciandoli man mano), scommettendo che non ci saranno split/merge a cascata.
- Arrivati alla foglia da modificare, si richiede un lock X solo su di essa.
- **Verifica:** Se la foglia è *safe*, l'operazione va a buon fine. Se *non è safe*, l'ipotesi ottimistica è fallita: la transazione rilascia i lock, abortisce l'operazione sull'albero e **ricomincia da capo applicando l'Algoritmo 1** (pessimistico).

Algoritmo 3 (Basato sui Lock Intensionali):

- **Inserimento/Cancellazione:** Si scende lungo l'albero acquisendo lock intensionali esclusivi (IX) sui nodi intermedi, senza rilasciarli, fino ad arrivare alla foglia.
- I lock IX permettono ad altre transazioni di scendere per leggere altre parti dell'albero (compatibilità IX con IS).
- Arrivati alla foglia bersaglio, il lock viene convertito in esclusivo (X). Se la foglia non è *safe* e necessita di propagare modifiche verso l'alto, i lock IX detenuti sugli antenati vengono progressivamente convertiti in lock X. Questo blocca temporaneamente il cammino, ma garantisce la massima concorrenza iniziale in lettura.

11.6 Controllo della Concorrenza Ottimistico (Kung-Robinson)

I protocolli basati sul locking (2PL) sono di tipo **pessimistico**: prevengono attivamente i conflitti mettendo in attesa le transazioni, ma introducono un notevole *overhead* per la gestione della tabella dei lock e il rischio di *deadlock*.

Se l'applicazione è caratterizzata da operazioni prevalentemente di lettura e i conflitti di scrittura sono statisticamente **rari**, si preferisce l'approccio **ottimistico** (Optimistic Concurrency Control - OCC), ideato da Kung e Robinson. Non si usano lock; le transazioni procedono liberamente e i conflitti vengono verificati solo alla fine.

Il ciclo di vita di una transazione è diviso in tre fasi sequenziali:

1. **Fase di Lettura (Working Phase):** La transazione opera su una **copia privata** e locale dei dati. Tutte le letture provengono dal database, ma tutte le scritture vengono memorizzate esclusivamente nel *workspace* temporaneo della transazione. Il database pubblico non subisce alcuna alterazione.
2. **Fase di Validazione:** Terminato il lavoro locale, la transazione chiede il permesso di effettuare il commit. Il DBMS verifica se, durante la fase di lettura, altre transazioni hanno alterato i dati invalidando il lavoro corrente. Se rileva un conflitto, la transazione subisce un abort (le copie private vengono scartate). Se non ci sono conflitti, si passa all'ultima fase.
3. **Fase di Scrittura:** Le modifiche calcolate nel workspace privato vengono copiate e rese permanenti nel database centrale.

Svantaggio: Notevole impiego di memoria RAM per mantenere le copie private e rischio di "starvation" (aborti continui) se l'assunzione di "conflitti rari" si rivela falsa. Non posso neanche condividere il buffer, perché ogni transazione ha la sua copia privata dei dati, quindi avrò dei duplicati in memoria.

Se io avessi una politica di gestione del buffer che non consente di avere duplicati, allora non potrei usare l'OCC.

Logica della Fase di Validazione La fase di validazione rappresenta il nucleo dell'algoritmo ottimistico (OCC). A ciascuna transazione T_j viene assegnato un **Timestamp** (TS) unicamente nel momento in cui **inizia la fase di validazione**, e non all'inizio della sua esecuzione. Questo TS sancisce l'ordine di serializzazione formale. Poiché i timestamp sono generati sequenzialmente dal sistema, garantiscono un ordinamento univoco (al più si verificherà una latenza di un ciclo di clock per transazioni concorrenti).

Per gestire i controlli, il sistema mantiene per ogni transazione due insiemi:

- $ReadSet(T_j)$: L'insieme degli identificatori degli oggetti letti dal database.

- $WriteSet(T_j)$: L'insieme degli identificatori degli oggetti modificati (presenti unicamente nella copia privata e locale della transazione).

Durante la validazione, la transazione corrente T_j deve dimostrare di essere consistente rispetto a tutte le transazioni precedenti T_i che hanno già superato la validazione (o hanno effettuato il commit) e che possiedono un $TS(T_i) < TS(T_j)$.

La validazione di T_j ha successo se e solo se, confrontandola con **ogni singola** transazione T_j , risulta soddisfatta **almeno una** delle seguenti tre condizioni:

Condizione 1 (Esecuzione Seriale Pura): T_i ha completato la sua *Fase di Scrittura* **prima** che T_j iniziasse la sua *Fase di Lettura*. Non vi è alcuna sovrapposizione temporale, escludendo matematicamente ogni possibile conflitto.

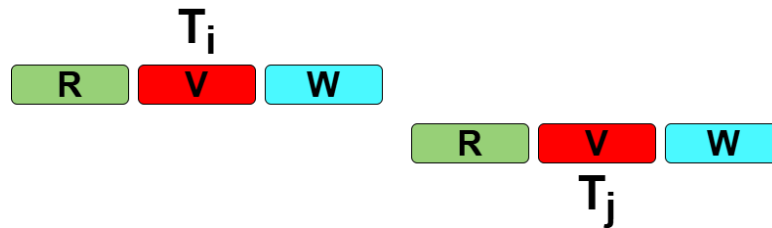


Figure 37: Condizione 1: Assenza di sovrapposizione temporale

Condizione 2 (Scritture non interferenti): Le transazioni presentano una sovrapposizione temporale, ma si verifica che:

$$WriteSet(T_i) \cap ReadSet(T_j) = \emptyset$$

E, inoltre, T_i conclude la sua *Fase di Scrittura* prima che T_j inizi la propria *Fase di Scrittura*. In termini pratici, l'intersezione nulla garantisce che T_j non legga dati "sporchi" o modificati da T_i . L'ordinamento temporale delle fasi di scrittura previene la sovrascrittura incrociata dei dati. Sono ammessi solo conflitti di tipo R-W innocui, preservando la correttezza logica.

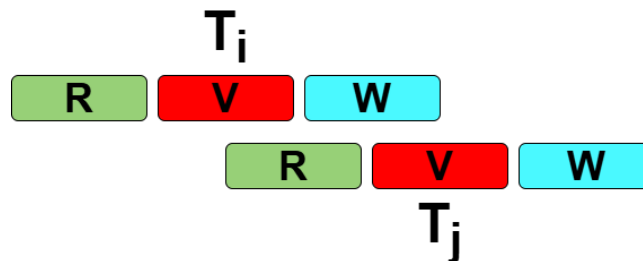


Figure 38: Condizione 2: Sovrapposizione parziale con dati disgiunti in lettura

Condizione 3 (Dati totalmente disgiunti): Le transazioni sono fortemente sovrapposte (esecuzione quasi parallela), ma si verifica che:

$$WriteSet(T_i) \cap ReadSet(T_j) = \emptyset$$

$$WriteSet(T_i) \cap WriteSet(T_j) = \emptyset$$

E, inoltre, T_i completa la sua *Fase di Lettura* prima che T_j completi la propria. Operando su insiemi di dati totalmente indipendenti per quanto riguarda le scritture, l'interfogliamento non causa anomalie. Questa condizione previene alla radice sia i conflitti R-W che i conflitti W-W (Lost Update).

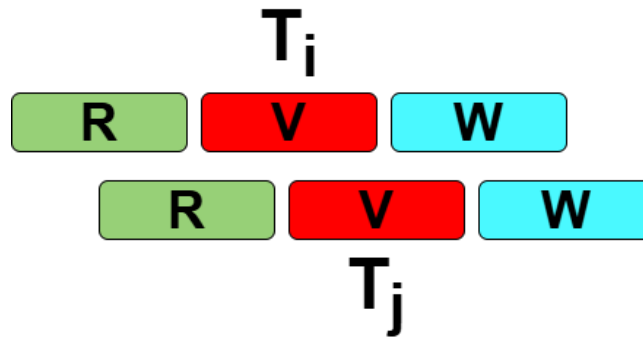


Figure 39: Condizione 3: Sovrapposizione totale con indipendenza dei dati

Se per anche una sola transazione T_i nessuna delle tre condizioni risulta verificata, la transazione in validazione T_j viene immediatamente interrotta (Abort), le sue copie locali eliminate e la transazione viene riavviata dall'inizio.

Questo approccio garantisce l'isolamento senza lock, ma introduce un potenziale *overhead* computazionale in scenari con un elevato numero di transazioni, poiché la validazione di T_j richiede la scansione degli insiemi di tutte le T_i precedenti.

Validazione Seriale (Ottimizzazione dell'Algoritmo) Per semplificare la complessità dei controlli (evitando di dover verificare la Condizione 3), i DBMS possono implementare una **Validazione Seriale**. Questa tecnica prevede che l'intera fase di validazione e la successiva fase di scrittura vengano eseguite all'interno di una **Regione Critica**. Solo una transazione alla volta può accedere a questo blocco logico, garantendo mutua esclusione.

Sia S l'insieme delle transazioni che hanno effettuato il commit *dopo* l'inizio della transazione corrente T . L'algoritmo verifica esclusivamente la Condizione 2:

```
// Inizio della Regione Critica
Lock(Validation_Mutex);

bool valid = true;
// S = insieme delle transazioni Ts committate dopo Begin(T)
foreach (Ts in S) {
    // Verifico che T non abbia letto dati modificati da Ts
    if (ReadSet(T) INTERSECT WriteSet(Ts) != EMPTY) {
        valid = false;
        break;
    }
}

if (valid == true) {
    Install_Updates(WriteSet(T)); // Fase di Scrittura nel DB reale
    Commit(T);
} else {
    Restart(T); // Abort e riavvio
}

Unlock(Validation_Mutex);
// Fine della Regione Critica
```

Vantaggi: L'algoritmo è drasticamente semplificato. L'impossibilità di eseguire scritture parallele (grazie alla regione critica) elimina alla radice i conflitti W-W. Inoltre, le transazioni *read-only* sono esentate dall'entrare in regione critica, massimizzando la loro efficienza. **Svantaggi:** La regione critica crea un collo di bottiglia che riduce il grado di parallelismo globale del sistema. Si introduce inoltre un grave rischio di **Starvation**: una transazione di lunga durata potrebbe essere sistematicamente abortita se, ogni volta che tenta di validarsi, trova in S una transazione più breve che ha causato un'intersezione.

11.7 Two-Phase Locking Ottimistico (O2PL)

Il protocollo O2PL rappresenta una soluzione ibrida che unisce l'alta concorrenza dell'OCC con la sicurezza strutturale del 2PL, riducendo l'overhead del locking preventivo in scrittura.

Il funzionamento si articola in due fasi:

1. **Fase di Lavoro (Lettura e Modifica Locale):** Per leggere un oggetto, la transazione acquisisce un classico **Lock Condiviso (S)**. Se la transazione intende modificare l'oggetto, non richiede subito un lock esclusivo; al contrario, ne crea una **copia privata** in memoria locale ed esegue le scritture unicamente su quest'ultima. Durante questa fase, le altre transazioni possono continuare a leggere l'oggetto originale.
2. **Fase di Validazione e Scrittura (Upgrade dei Lock):** Giunta al termine, la transazione tenta di effettuare il commit. Per rendere le modifiche persistenti, il DBMS tenta di **promuovere (upgrade)** i Lock Condivisi (S) associati agli oggetti modificati in **Lock Esclusivi (X)**. Se nessun'altra transazione detiene un lock S su quegli oggetti, l'upgrade ha successo, le scritture vengono applicate al database e i lock rilasciati. Se vi sono altri lock S attivi, la transazione viene sospesa e messa in attesa che l'oggetto si liberi.

L'O2PL ritarda l'acquisizione dei lock esclusivi all'ultimo istante utile. Questo riduce significativamente la probabilità di bloccare il sistema, ma non elimina il rischio di **Deadlock** (presente nella fase di upgrade se due transazioni tentano simultaneamente di promuovere un lock S in lock X sullo stesso oggetto). Pertanto, il sistema deve comunque implementare meccanismi di rilevamento o prevenzione degli stalli.

11.8 Timestamp Concurrency Control (Timestamp CC)

Invece di basarsi sui lock o sulla validazione a posteriori, questo protocollo utilizza i **Timestamp** per imporre a priori un ordinamento seriale equivalente basato sull'anzianità delle transazioni. Ad ogni transazione T viene assegnato un timestamp $TS(T)$ al momento dell'avvio. A livello di database, ad ogni **singolo oggetto** O vengono associati due marcatori temporali per tracciare lo storico degli accessi:

- **$RTS(O)$ (Read Timestamp):** Il timestamp della transazione **più giovane** (con TS maggiore) che ha letto con successo l'oggetto.
- **$WTS(O)$ (Write Timestamp):** Il timestamp della transazione **più giovane** che ha modificato con successo l'oggetto.

L'assioma fondamentale è garantire che l'ordine di esecuzione delle operazioni in conflitto coincida sempre con l'ordine cronologico dei timestamp. Se una transazione tenta un'operazione anacronistica rispetto allo stato del dato, viene abortita.

Regola di Lettura (Read) Quando T richiede di leggere l'oggetto O :

- **Se $TS(T) < WTS(O)$:** La transazione T è "in ritardo". L'oggetto O è già stato sovrascritto da una transazione più giovane. Leggerlo comporterebbe una lettura inconsistente rispetto alla linea temporale. La lettura viene rifiutata, T subisce un **Abort** e viene riavviata con un nuovo TS.
- **Se $TS(T) \geq WTS(O)$:** L'operazione è cronologicamente coerente. T legge il valore di O . Il DBMS aggiorna il Read Timestamp dell'oggetto per riflettere l'accesso: $RTS(O) = \max(RTS(O), TS(T))$.

Regola di Scrittura (Write) Quando T richiede di scrivere (modificare) l'oggetto O :

- **Se $TS(T) < RTS(O)$:** L'oggetto è già stato letto da una transazione più giovane, che presumibilmente ha usato quel valore per altre operazioni. Se T scrivesse ora, modificherebbe il passato, invalidando le letture già avvenute. T subisce un **Abort** e viene riavviata.

- **Se $TS(T) < WTS(O)$:** L'oggetto è stato già sovrascritto da una transazione più giovane. Sebbene T sia in ritardo, la sua scrittura è "obsoleta" e verrebbe comunque oscurata dalla scrittura successiva. In questo scenario si applica la **Thomas Write Rule**: l'operazione di scrittura di T viene silenziosamente ignorata (non viene eseguita), **senza** provocare l'aborto di T . L'esecuzione può proseguire.
- **Se $TS(T) \geq RTS(O)$ e $TS(T) \geq WTS(O)$:** L'operazione è lecita. T modifica l'oggetto e si aggiorna il Write Timestamp: $WTS(O) = TS(T)$.

11.8.1 View Serializability e Thomas Write Rule

L'applicazione della Thomas Write Rule produce schedule che violano la stretta Conflict Serializability, ma che rimangono corretti in quanto **View Serializable**. Due schedule S_1 e S_2 sono *view equivalent* se garantiscono la stessa visione dei dati:

1. **Letture Iniziali:** Se T_i legge il valore iniziale di A in S_1 , deve leggerlo anche in S_2 .
2. **Flusso dei Dati:** Se T_i legge un valore di A scritto da T_j in S_1 , la medesima dipendenza deve esistere in S_2 .
3. **Scritture Finali:** Se T_i esegue l'ultima scrittura (che definirà lo stato finale) su A in S_1 , deve eseguire l'ultima scrittura anche in S_2 .

Esempio di Blind Write Si considerino i seguenti due schedule. La transazione T_3 è più giovane di T_2 .

Schedule Seriale (S_1)			Schedule Interfogliato (S_2)		
T_1 :	$R(A)$	$W(A)$	T_1 :	$R(A), W(A)$	
T_2 :	$W(A)$		T_2 :		$W(A)$
T_3 :		$W(A)$	T_3 :	$W(A)$	

Nello schedule interfogliato (S_2), T_3 scrive A prima di T_2 , invertendo l'ordine dei conflitti W-W rispetto allo schedule seriale (S_1). Questo rende S_2 **non conflict serializable**. Tuttavia, poiché T_2 esegue una **Blind Write** (scrittura cieca, senza aver letto A in precedenza) e T_3 sovrascrive a sua volta A definendone lo stato finale, il DBMS può ignorare l'operazione di T_2 tramite la Thomas Write Rule. Il risultato finale (dettato da T_3) e le letture (effettuate solo da T_1) rimangono inalterati. S_2 è quindi *View Serializable*.

11.9 Multiversion Timestamp Concurrency Control (MVCC)

Nei sistemi con alta concorrenza, le continue aborts del Timestamp CC dovute ai conflitti R-W degradano le performance. L'approccio **Multiversion** (MVCC) risolve il problema alla radice: le scritture non sovrascrivono l'oggetto originale, ma creano una **nuova versione** (storica) del dato. Ogni oggetto logico O è rappresentato da una lista concatenata di versioni fisiche: $O_1, O_2, \dots, O_k$.

Ciascuna versione O_k conserva autonomamente i propri marcatori:

- $WTS(O_k)$: Timestamp della transazione che ha *creato* la versione.
- $RTS(O_k)$: Timestamp della transazione più giovane che ha *letto* la versione.

Il vantaggio fondamentale del MVCC è che **le operazioni di lettura non vengono mai bloccate né abortite**.

Gestione delle Letture Quando T_i richiede di leggere l'oggetto O : Il DBMS ispeziona lo storico delle versioni di O e seleziona la versione O_k avente il **massimo WTS tale che $WTS(O_k) \leq TS(T_i)$** . In altre parole, la transazione legge l'ultima versione consolidata che era disponibile al momento del suo avvio, garantendo un *Snapshot Isolation* perfetto. Se la versione prescelta è stata generata da una transazione non ancora committata, T_i viene semplicemente messa in attesa per evitare *Cascading Aborts*, ma **non** fallisce. Una volta letta, si aggiorna $RTS(O_k) = \max(RTS(O_k), TS(T_i))$.

Gestione delle Scritture Quando T_i richiede di modificare l'oggetto O : Il DBMS individua la versione O_k che T_i "dovrebbe" leggere (la più recente rispetto a $TS(T_i)$).

- **Se $RTS(O_k) > TS(T_i)$:** Significa che una transazione T_j *più giovane* di T_i ha già letto questa specifica versione O_k . Se T_i creasse ora una nuova versione, altererebbe il passato logico di T_j . In questo caso, T_i **subisce un Abort** e viene riavviata.
- **Altrimenti ($RTS(O_k) \leq TS(T_i)$):** L'operazione è lecita. T_i crea una nuova versione O_{new} (mantenuta nel buffer fino al commit) e aggiorna i puntatori. Vengono impostati i marcatori: $WTS(O_{new}) = TS(T_i)$ e $RTS(O_{new}) = TS(T_i)$.

12 Crash Recovery

Il **Recovery Manager** (gestore del ripristino) è il componente del DBMS incaricato di garantire le proprietà di **Atomicità** (in caso di fallimento parziale) e **Durabilità** (a fronte di salvataggi definitivi), assicurando che il database transiti sempre verso stati consistenti anche in presenza di anomalie hardware o software.

Affinché il sistema possa reagire correttamente, è necessario distinguere tra due tipologie principali di interruzione:

Abort (Fallimento di Transazione) L'abort è un'interruzione **controllata**. Si verifica quando una singola transazione viene annullata (dal sistema per risolvere un deadlock, dalla violazione di un vincolo, o per scelta dell'utente). In questo scenario, la memoria volatile (RAM/Buffer) è perfettamente intatta: il DBMS conosce esattamente lo stato del sistema e possiede tutti i metadati in memoria per effettuare il *rollback* (disfacimento) delle operazioni in modo sicuro.

Crash (Fallimento di Sistema) Il crash è un'interruzione **incontrollata** (es. calo di alimentazione, kernel panic). Comporta la **perdita totale e istantanea di tutto il contenuto della memoria volatile** (Buffer Pool, metadati, tabelle temporanee). Al riavvio, il DBMS si trova di fronte a un disco che può contenere uno stato inconsistente. L'obiettivo del crash recovery è ricostruire un database coerente partendo dalle tracce fisiche sopravvissute. Nello specifico, il sistema dovrà:

- **UNDO (Disfare):** Annullare tutte le modifiche scritte su disco appartenenti a transazioni che, al momento del crash, *non* avevano ancora effettuato il commit.
- **REDO (Rifare):** Riapplicare tutte le modifiche appartenenti a transazioni che avevano effettuato il commit prima del crash, ma i cui aggiornamenti si trovavano ancora nel buffer volatile e non erano stati fisicamente salvati su disco.

Il crash recovery opera in stretta sinergia con il Concurrency Control Manager. L'utilizzo di protocolli come lo **Strict 2PL** risulta fondamentale: mantenendo i lock esclusivi fino al commit o all'abort, si garantisce che nessuna transazione possa leggere dati temporanei. Questo evita il fenomeno degli *aborti a cascata* e garantisce che l'operazione di UNDO riporti sempre il dato al suo stato originale corretto. Inoltre, si assume che il DBMS operi con aggiornamenti **in-place**: i valori su disco vengono fisicamente sovrascritti dai nuovi dati, non si creano file separati per le modifiche.

12.1 Gestione del Buffer Pool: Politiche Steal e No-Force

La necessità di effettuare operazioni di REDO e UNDO deriva direttamente dalle politiche di interazione tra il Buffer Manager (in RAM) e il disco. Le scelte progettuali si basano su due assi cartesiani:

Force vs No-Force (Gestione del Commit):

- **Force:** Al momento del commit, si forza la scrittura sincrona su disco di tutte le pagine modificate dalla transazione. Garantisce la durabilità immediata, ma degrada drasticamente le prestazioni a causa dell'eccessivo I/O.
- **No-Force:** Al commit, le pagine modificate rimangono nel buffer (permettendo ad altre transazioni di leggerle velocemente) e verranno scritte su disco in futuro, in modo asincrono. **Richiede il REDO** in caso di crash.

Steal vs No-Steal (Gestione dello Spazio):

- **No-Steal:** Una pagina modificata da una transazione non ancora committata non può mai essere scritta su disco per liberare spazio nel buffer. Assorbe molta RAM.
- **Steal:** Se il buffer è pieno, il gestore può "rubare" spazio salvando su disco una pagina contenente modifiche di una transazione *ancora in corso*. **Richiede l'UNDO** se tale transazione dovesse poi fallire.

I DBMS moderni adottano universalmente il paradigma **Steal / No-Force** per massimizzare il throughput e le performance, accettando la complessità di dover implementare meccanismi di recovery avanzati.

12.2 Logging e Write-Ahead Logging (WAL)

Per supportare le operazioni di REDO e UNDO, il DBMS registra la storia sequenziale e cronologica di tutte le operazioni di aggiornamento (le letture non vengono tracciate) all'interno di una struttura dedicata: il **File di Log**.

12.2.1 Struttura del File di Log e LSN

Il log è una sequenza strutturata come file *append-only*. Ad ogni nuovo record di log viene assegnato un identificatore univoco crescente, il **LSN (Log Sequence Number)**, che definisce l'ordinamento cronologico assoluto degli eventi.

Ogni pagina dati in memoria contiene un campo speciale, il **pageLSN**, che memorizza l'LSN dell'ultimo record di log che ha apportato una modifica a tale pagina. Similmente, il sistema mantiene in una variabile sicura il **flushedLSN**, che indica l'LSN dell'ultimo record di log che è stato fisicamente scritto sul disco.

I record di log principali sono formati dai seguenti campi:

- **prevLSN**: Puntatore all'LSN del record precedente generato *dalla medesima transazione*. Permette di navigare a ritroso nel log per le operazioni di UNDO.
- **XID**: Identificatore univoco della transazione.
- **Type**: Tipo di operazione (*Update, Commit, Abort, End, Checkpoint, CLR*).
- **pageID, offset, length**: Coordinate fisiche per localizzare il dato modificato.
- **old data (Before Image)**: Valore del dato prima della modifica (necessario per l'UNDO).
- **new data (After Image)**: Valore del dato dopo la modifica (necessario per il REDO).

12.2.2 La Regola del Write-Ahead Logging (WAL)

Il log viene generato nel buffer volatile. Affinché sia utile per il recovery, le informazioni del log devono essere persistenti (su disco) *prima* dei dati veri e propri. La regola del **Write-Ahead Logging** stabilisce due vincoli imprescindibili:

1. **Per la proprietà di Atomicità (Regola per l'UNDO)**: Prima che una pagina dati venga scaricata dal buffer al disco (azione di *Steal*), tutti i record di log relativi alle modifiche di quella pagina devono essere stati salvati su disco. Matematicamente: prima del flush della pagina deve valere $pageLSN \leq flushedLSN$.
2. **Per la proprietà di Durabilità (Regola per il REDO)**: Quando una transazione effettua il commit, il DBMS deve assicurarsi che tutti i record di log prodotti da quella transazione (incluso il record di *Commit*) vengano salvati sincronicamente su disco prima di notificare il successo all'utente.

Nota sull'ottimizzazione del flushing: Il salvataggio dei log su disco non avviene salvando il singolo record, ma scaricando blocchi di log. Se la transazione T_2 modifica un dato e poco dopo la transazione T_5 effettua il commit, il flush richiesto da T_5 porterà su disco anche i log generati precedentemente da T_2 , ammortizzando il costo delle operazioni di I/O.

12.3 Strutture Dati di Supporto al Recovery

Per gestire efficientemente il ripristino, il DBMS mantiene in RAM due strutture dati critiche (che tuttavia vanno perse durante un crash):

Transaction Table (Tabella delle Transazioni): Contiene un record per ogni transazione attualmente attiva nel sistema. Traccia:

$\langle \text{XID}, \text{status}, \text{lastLSN} \rangle$

Dove *status* può essere Running, Committed o Aborted, e *lastLSN* è il puntatore all'ultimo log record scritto da quella specifica XID. Al riavvio dopo un crash, le transazioni contrassegnate come in esecuzione subiranno l'UNDO, quelle in commit subiranno il REDO.

Dirty Page Table (Tabella delle Pagine Sporche): Contiene una riga per ogni pagina nel buffer che è stata modificata rispetto alla sua controparte su disco. Traccia:

$\langle \text{pageID}, \text{recLSN} \rangle$

Il **recLSN** (Recovery LSN) è l'identificatore del *primo* record di log che ha "sporcato" la pagina (la modifica più vecchia non ancora salvata). Questo valore è vitale per ottimizzare il REDO, indicando da quale punto del log è necessario iniziare a leggere per ripristinare quella specifica pagina.

12.4 Checkpointing

Scorrere l'intero file di log dalla sua creazione per un'operazione di recovery impiegherebbe ore. Per limitare e stabilizzare i tempi di ripristino, il DBMS genera periodicamente dei **Checkpoint**. Il checkpointing è un'operazione asincrona che scatta istantanee dello stato interno del gestore della transazione:

1. Si scrive nel log un record di **begin_checkpoint**.
2. Si copiano nel log i contenuti attuali della *Transaction Table* e della *Dirty Page Table*.
3. Si scrive nel log un record di **end_checkpoint**.

In caso di crash, il DBMS leggerà il log all'indietro fino a trovare l'ultimo *end_checkpoint* completo, per ricostruire immediatamente lo stato delle due tabelle al momento del checkpoint, integrando poi le modifiche successive lette dal log. Questo limita l'analisi alla sola porzione recente della storia del database.

12.5 La Procedura di Abort (Disfacimento Logico)

Se una transazione deve essere annullata (abort), il sistema procede con un rollback controllato utilizzando la catena dei log, accessibile tramite il campo *lastLSN* nella *Transaction Table*.

La procedura avviene seguendo rigorosi passi logici:

1. **Inizio:** Viene scritto un log record di tipo *Abort* per certificare l'inizio della fase di disfacimento. Grazie allo Strict 2PL, il DBMS detiene ancora tutti i lock esclusivi necessari, operando in totale isolamento.
2. **Rollback a catena:** Utilizzando i puntatori *prevLSN*, il sistema ripercorre a ritroso tutti i record generati dalla transazione.
3. **Scrittura dei CLR:** Prima di ripristinare fisicamente il valore *old_data* nella pagina, per ogni modifica annullata il DBMS genera e salva un **Compensation Log Record (CLR)**. Il CLR descrive l'azione di ripristino in corso ed è dotato di un campo fondamentale: il **undoNextLSN**. Questo campo punta al prossimo record da disfare (il *prevLSN* del record originario).
4. **Fine:** Quando la catena a ritroso è esaurita (la transazione è stata totalmente annullata), viene scritto il record *End* e i lock vengono rilasciati.

Il ruolo vitale dei CLR: La scrittura dei CLR è cruciale per prevenire anomalie nel caso in cui il sistema subisca un crash proprio mentre sta eseguendo un Abort. Poiché i CLR sono record di log a tutti gli effetti, essi vengono riapplicati dalla fase di REDO. Se il sistema si interrompe a metà di un UNDO, al successivo riavvio leggerà il CLR e il suo campo *undoNextLSN*. Questo indica al DBMS di non ripetere i disfacimenti già avvenuti, ma di riprendere l'abort esattamente da dove si era interrotto. Per definizione strutturale: **un'operazione di UNDO (registrata tramite CLR) non viene mai disfatta**. Non è possibile abortire un aborto.

12.6 La Procedura di Crash Recovery

Al riavvio del sistema a seguito di un crash, il DBMS deve ispezionare il log ed eseguire una procedura di ripristino per riportare il database a uno stato consistente. Questa procedura, formalizzata universalmente dall'algoritmo **ARIES** (*Algorithm for Recovery and Isolation Exploiting Semantics*), si articola in tre fasi sequenziali:

1. **Analisi (Analysis):** Ricostruisce lo stato della memoria volatile (Transaction Table e Dirty Page Table) al momento esatto del crash.
2. **REDO (Rifacimento):** Ripete l'intera storia a partire da un punto specifico del log per riportare il database esattamente allo stato in cui si trovava un istante prima dell'interruzione, ricreando anche lo stato delle transazioni che poi dovranno essere annullate.
3. **UNDO (Disfacimento):** Scandisce il log a ritroso per annullare le modifiche delle transazioni *loser* (quelle attive al momento del crash e non committate).

Nota preliminare: Eventuali operazioni che al momento del crash si trovavano in esecuzione, ma non avevano ancora prodotto alcun record nel buffer di log né modifiche su disco, sono semplicemente e irrimediabilmente perse senza compromettere la consistenza del sistema.

12.6.1 Fase di Analisi

L'obiettivo è ricostruire le tabelle di supporto. Il sistema individua nel log l'ultimo record di **begin_checkpoint** e il relativo **end_checkpoint**, ricaricando in memoria la *Transaction Table* e la *Dirty Page Table* (DPT) valide in quell'istante. Da questo punto, si scandisce il log **in avanti** (forward pass) fino alla fine, analizzando sequenzialmente i record incontrati per aggiornare le due tabelle:

Record di End (Commit o Abort completato): Se il log riporta la fine definitiva di una transazione, questa viene rimossa dalla Transaction Table (se presente). Il suo ciclo di vita è concluso, indipendentemente dal fatto che abbia avuto successo o meno.

Record di Controllo (Begin, Commit, Abort in corso): Se si incontra un record appartenente a una transazione non ancora presente nella Transaction Table, la transazione viene inserita con status *Running*. Il campo *lastLSN* della transazione viene costantemente aggiornato all'LSN del record che si sta leggendo. Se il record letto è un *Commit*, lo status viene aggiornato a *Committed*; se è un *Abort*, lo status diventa *Aborted*.

Record di Update (Modifiche ai dati): Oltre ad aggiornare il *lastLSN* nella Transaction Table, occorre verificare la Dirty Page Table. Se la pagina modificata dal record non è presente nella DPT, viene aggiunta impostando il suo **recLSN** (Recovery LSN) pari all'LSN di questo specifico record di update (poiché è la modifica più vecchia non salvata che ha sporcato la pagina). Se la pagina è già nella DPT, il suo *recLSN* rimane inalterato.

12.6.2 Fase di REDO

La fase di REDO ha lo scopo di riapplicare le operazioni per sincronizzare il disco allo stato del buffer perso. L'operazione inizia **in avanti** partendo dal più piccolo valore di *recLSN* presente nella Dirty Page Table (il *FirstLSN*). Questo punto rappresenta la modifica più remota nella storia del database che potrebbe non essere stata ancora scritta su disco. Le pagine non presenti nella DPT sono garanzia di essere già allineate e persistenti su disco.

Per ogni record di log (di tipo Update o CLR) incontrato dal *FirstLSN* in poi, il sistema **tenta di riapplicare l'azione**, a meno che non si verifichi una delle seguenti tre eccezioni (che garantiscono l'idempotenza e l'efficienza del REDO):

- **Condizione 1 (Pagina sicura):** La pagina oggetto della modifica non è presente nella Dirty Page Table al termine dell'analisi. Significa che la modifica era già stata forzata su disco prima del crash. *L'azione non viene rifatta.*
- **Condizione 2 (Modifica obsoleta):** La pagina è nella DPT, ma il suo *recLSN* è strettamente maggiore dell'*LSN* del record in esame ($recLSN > LSN$). Significa che la pagina è stata "sporcata" per la prima volta da una modifica successiva a quella attuale, pertanto l'operazione in esame è certamente già su disco. *L'azione non viene rifatta.*
- **Condizione 3 (Controllo fisico su disco):** Se le prime due condizioni non scattano, il DBMS è costretto a leggere fisicamente la pagina dal disco e controllare il suo **pageLSN**. Se $pageLSN \geq LSN$, significa che la pagina su disco contiene già questa modifica (o una successiva). *L'azione non viene rifatta.*

Se nessuna delle tre eccezioni si verifica, **l'azione viene rifatta**:

1. Si riapplicano le modifiche presenti nel record di log sulla pagina.
2. Si aggiorna il *pageLSN* della pagina ponendolo uguale all'*LSN* del record appena riapplicato.
3. **Importante:** Durante il REDO non viene generato alcun nuovo record di log.

12.6.3 Fase di UNDO

Ripristinato lo stato al momento del crash, il sistema deve annullare le operazioni delle transazioni che risultano ancora attive nella Transaction Table (transazioni *loser*). Viene inizializzato un insieme contenente i puntatori alle operazioni da annullare:

$$ToUndo = \{l \mid l \text{ è il } lastLSN \text{ di una transazione in stato Running}\}$$

La fase procede **all'indietro** (backward pass) estraendo iterativamente il record più recente, finché l'insieme *ToUndo* non è vuoto:

- Si estrae da *ToUndo* il record con l'**LSN più grande** (garantendo un annullamento cronologico a ritroso in caso di transazioni concorrenti).
- **Se il record è un Update:** L'operazione viene annullata riportando il dato al suo stato originale (*Before Image*). Si scrive un **CLR** (Compensation Log Record) nel file di log per tracciare l'avvenuto annullamento. Infine, si inserisce in *ToUndo* il *prevLSN* del record appena annullato, per proseguire l'analisi della catena della transazione.
- **Se il record è un CLR (e $undoNextLSN \neq NULL$):** Significa che stiamo ripercorrendo un abort che era già in corso e si era interrotto a metà. L'azione non viene annullata (non si disfa un disfacciamento), ma si inserisce in *ToUndo* il valore **undoNextLSN** contenuto nel CLR, saltando in modo sicuro le operazioni già compensate.
- **Se si raggiunge l'inizio della transazione:** Che avvenga tramite un CLR con $undoNextLSN == NULL$ o arrivando al primo *begin* della transazione, l'annullamento per quella transazione è terminato. Si scrive un record di **End** nel log.

Quando l'insieme *ToUndo* è completamente vuoto, il database ha eliminato ogni traccia delle transazioni *loser* e il crash recovery è considerato concluso con successo. Il sistema è ora pronto ad accettare nuove transazioni.

12.6.4 Il Paradigma "Repeating History": Perché il REDO precede l'UNDO?

Una delle caratteristiche architetturali fondamentali dell'algoritmo ARIES è l'obbligo di completare interamente la fase di REDO prima di poter iniziare la fase di UNDO. Questo principio, noto come **Repeating History** (ripetizione della storia), è giustificato da tre motivi cruciali:

- **Ripristino del contesto esatto:** La fase di REDO ha lo scopo di portare fisicamente il database *esattamente* allo stato in cui si trovava nell'istante in cui è avvenuto il crash, ricostruendo fedelmente anche lo stato delle transazioni destinate a fallire. Grazie a questo riallineamento, la successiva fase di UNDO non deve operare in una complessa modalità "di emergenza", ma può limitarsi a richiamare le stesse identiche procedure standard utilizzate per un normale *Abort* quando il sistema è a regime.
- **Gestione sicura dei crash durante un Abort:** Se il sistema si arresta mentre stava già annullando una transazione, il log conterrà dei **CLR** (Compensation Log Record) per le operazioni già disfatte. Eseguendo prima il REDO, il DBMS riapplica passivamente questi CLR. In questo modo, quando inizierà l'UNDO, il sistema troverà la catena dei log perfettamente aggiornata, leggerà il campo *undoNextLSN* e riprenderà il disfacimento esattamente da dove si era interrotto, evitando il rischio catastrofico di annullare due volte la stessa operazione.
- **Coesistenza di dati nella stessa pagina:** Con le politiche *Steal/No-Force* e i lock a grana fine (es. a livello di record), una singola pagina su disco può contenere contemporaneamente modifiche di transazioni committate (che necessitano del REDO) e modifiche di transazioni *loser* (che necessitano dell'UNDO). Il REDO garantisce che la struttura fisica della pagina sia coerente e aggiornata all'ultima versione nota prima che l'UNDO intervenga a modificarne i singoli record per le compensazioni logiche.